

N I B M

Degree of Master of Science in Data Science

NIB7150CEM - Data Science Project

**Low-Cost Offline Voice Assistant for Hospitality Services in
Sri Lanka Using Small-Scale Neural Models**

Author: M. D. P. Wijesuriya

SID: COMScDS241P-002

Supervisor: Mr. Dumira Athukorala

Submitted in partial fulfilment of the requirements for the Degree of Master of Science in Data Science

Academic Year: 2024/25

Declaration of Originality

I declare that this project is all my own work and has not been copied in part or in whole from any other source except where duly acknowledged. As such, all use of previously published work (from books, journals, magazines, internet etc.) has been acknowledged by citation within the main report to an item in the References or Bibliography lists. I also agree that an electronic copy of this project may be stored and used for the purposes of plagiarism prevention and detection.

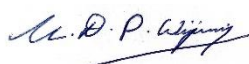
Statement of copyright

I acknowledge that the copyright of this project report, and any product developed as part of the project, belong to Coventry University. Support, including funding, is available to commercialise products and services developed by staff and students. Any revenue that is generated is split with the inventor/s of the product or service. For further information please see www.coventry.ac.uk/ipr or contact ipr@coventry.ac.uk.

Statement of ethical engagement

I declare that a proposal for this project has been submitted to the Coventry University ethics monitoring website (<https://ethics.coventry.ac.uk/>) and that the application number is listed below (Note: Projects without an ethical application number will be rejected for marking)

Signed:



Date: 25/04/2026

Please complete all fields.

First Name:	Dinesh
Last Name:	Wijesuriya
Student ID number	COMScDS241P-002
Ethics Application Number	-
1 st Supervisor Name	Mr. Dumira Athukorala
2 nd Supervisor Name	-

This form must be completed, scanned and included with your project submission to Turnitin. Failure to append these declarations may result in your project being rejected for marking.

Abstract

This research presents the design, development, and evaluation of a low-cost, offline voice assistant system prototype for hotel room service operations, targeting hotels in Sri Lanka. The system addresses a practical gap in hospitality technology for developing economies: in most small Sri Lankan hotels, room service requests are handled through telephone calls or in-person visits, a manual process that is slow, staff-intensive, and error-prone. Existing voice assistant solutions could automate this, but they depend on cloud infrastructure, incur recurring subscription costs, and raise privacy concerns. An on-device solution removes the connectivity dependency and the cost burden, while keeping guest voice data private.

The system prototype runs entirely on a commodity Android tablet in the \$50-\$150 price range. All voice processing happens on-device using Vosk for speech recognition and a fine-tuned MobileBERT intent classifier exported as a TFLite model. A hybrid natural language understanding pipeline combines rule-based keyword matching with neural model inference to handle 18 hotel service intent categories covering common room service operations. A FastAPI backend with SQLite persistence and WebSocket communication manages the full request lifecycle, from guest voice input through to real-time staff notification and service tracking all within a local hotel network, with no external data transmission.

Evaluation addressed three dimensions of viability. For NLU accuracy, a three-variant training experiment demonstrated that noise-aware training on paired clean and Vosk-transcribed data closes the accuracy gap introduced by real speech recognition. The baseline model dropped from 98.07% on clean text to 89.34% on Vosk output, while the noise-aware model recovered to 99.06% on the same Vosk-transcribed input. For speech recognition, the Vosk pipeline achieved an overall word error rate of 11.43% on hotel-domain phrases. For system performance, end-to-end latency reached a P95 of 2,827ms, within the 5-second target set for practical usability.

The results confirm that a fully offline voice assistant system prototype, built on commodity Android hardware, achieves sufficient technical accuracy and performance to be a viable alternative to traditional room service communication in Sri Lankan hotels.

Table of Contents

Abstract.....	2
Table of Contents	3
Acknowledgements.....	9
1 Introduction	10
1.1 Introduction.....	10
1.2 Background to the Project	10
1.3 Project Objectives	11
1.3.1 Design and Develop a Low-Cost, Offline Voice Assistant Prototype	11
1.3.2 Build a Hospitality-Domain Dataset and Train a Noise-Aware NLU Model	11
1.3.3 Build a Lightweight Backend with Real-Time Staff Communication	11
1.3.4 Evaluate System Accuracy, Latency, and Cost-Effectiveness	11
1.4 Overview of This Report.....	11
1.5 Research Contributions	12
1.6 Summary	13
2 Literature Review	14
2.1 Introduction.....	14
2.2 Challenges Hotel Businesses Face in Sri Lanka	14
2.3 Impact of the Challenges	15
2.4 Solutions Proposed in Existing Research and Industry	16
2.4.1 Commercial Cloud-Based Voice Assistants	16
2.4.2 Edge-Based and Offline Speech Recognition	16
2.4.3 Compact NLU Models for On-Device Intent Classification.....	16
2.4.4 Privacy-Preserving Edge Computing	16
2.5 How Existing Solutions Have Helped Address the Challenges.....	17
2.6 Knowledge Contribution of This Research and Model Performance.....	17
2.6.1 Identified Research Gaps.....	17
2.6.2 Empirical Approach to Addressing Gap 1.....	19
2.7 Technology Tools and Their Suitability	19
2.7.1 Vosk for On-Device Speech-to-Text.....	19
2.7.2 MobileBERT and TensorFlow Lite for On-Device NLU.....	19
2.7.3 Android Tablets for Hardware Deployment	19
2.7.4 FastAPI and WebSocket for Backend Communication.....	20
2.7.5 Summary of Tool Suitability	20
2.8 Summary	20
3 Methodology.....	21
3.1 Introduction.....	21
3.2 Research Approach.....	21
3.3 Development Approach.....	21

3.4	System Architecture	22
3.5	Technology Selection	23
3.6	Dataset Preparation	23
3.6.1	Intent Category Design	23
3.6.2	Dataset Construction	24
3.6.3	Vosk Transcription Pairing - The Core Experimental Dataset	24
3.6.4	Three Training Datasets	24
3.6.5	Train / Validation / Test Split	24
3.7	Model Training	25
3.7.1	MobileBERT Fine-Tuning	25
3.8	Evaluation Methodology	25
3.8.1	NLU Accuracy – Three Model Comparison	25
3.8.2	Speech Recognition Accuracy	26
3.8.3	System Latency	26
3.8.4	Cost-Effectiveness	26
3.9	Summary	26
4	Requirements	27
4.1	Introduction	27
4.2	Requirements Elicitation	27
4.2.1	Hotel Operational Requirements - Literature Review	27
4.2.2	Guest Survey	28
4.3	Functional Requirements	29
4.4	Non-Functional Requirements	30
4.5	Requirements Prioritisation	31
4.6	Summary	31
5	Analysis	32
5.1	Introduction	32
5.2	High-Level System Conceptual View	32
5.3	Use Case Model	33
5.3.1	Use Case Descriptions	33
5.4	Process Flow Analysis	35
5.4.1	Voice Request Pipeline	35
5.4.2	Staff Request Handling Flow	37
5.5	Entity-Relationship Model	38
5.6	Technology Evaluation and Selection	38
5.6.1	Speech Recognition: Vosk vs Whisper vs Cloud APIs vs CMU Sphinx	39
5.6.2	Intent Classification: MobileBERT vs DistilBERT vs Rasa DIET vs Rule-Based	41
5.6.3	Backend Framework: FastAPI vs Flask vs Django vs Express.js	42
5.6.4	Database: SQLite vs PostgreSQL vs MySQL vs MongoDB	42
5.6.5	Real-Time Communication: WebSocket vs HTTP Polling vs Server-Sent Events vs MQTT	43
5.6.6	Mobile Platform: Android Native vs iOS vs Flutter vs React Native	44
5.6.7	Text-to-Speech: Android Native TTS vs Piper TTS vs Amazon Polly	45

5.7	Summary	45
6	Design	46
6.1	Introduction.....	46
6.2	System Architecture	46
6.2.1	Three-Tier Architecture	46
6.3	Voice Processing Pipeline Design	47
6.3.1	Pipeline Overview	47
6.3.2	Hybrid NLU Pipeline – Core Design Feature	49
6.4	Database Design.....	51
6.5	API and Communication Design.....	52
6.5.1	REST API Endpoints.....	52
6.5.2	WebSocket Communication Design	52
6.5.3	Request Lifecycle State Management.....	54
6.6	Department Routing Design.....	55
6.7	User Interface Design	55
6.7.1	Guest Application Interface	55
6.7.2	Staff Dashboard Interface	56
6.8	Summary	58
7	Implementation.....	59
7.1	Introduction.....	59
7.2	Development Environment	59
7.3	Iterative Implementation Process	60
7.3.1	Iteration 1: On-Device Speech Recognition.....	60
7.3.2	Iteration 2: Intent Classification Pipeline.....	60
7.3.3	Iteration 3: Backend and Real-Time Communication	61
7.3.4	Iteration 4: Integration and Refinement	62
7.4	Component Implementation Details	62
7.4.1	Android Guest Application.....	62
7.4.2	NLU Training Pipeline	63
7.4.3	Backend Server	64
7.4.4	Staff Dashboard.....	65
7.5	Deployment.....	66
7.6	Current Status.....	66
7.7	Summary	67
8	Testing	68
8.1	Introduction.....	68
8.2	Testing Approach	68
8.3	NLU Model Testing	68
8.3.1	Test Design.....	68
8.3.2	Results.....	69
8.3.3	Per-Intent Results (Model C on Vosk)	71
8.4	Speech Recognition (WER) Testing	72

8.4.1	What Was Tested and Why	72
8.4.2	Results.....	72
8.5	Backend API Testing	73
8.6	End-to-End Integration Testing	74
8.7	System Latency Testing.....	74
8.7.1	Measurement Approach.....	74
8.7.2	Results.....	75
8.8	What Was Not Tested	75
8.9	Summary	76
9	Project Management.....	77
9.1	Introduction.....	77
9.2	Project Schedule.....	77
9.2.1	Work Breakdown Structure	77
9.2.2	Schedule Adherence.....	78
9.3	Risk Management	78
9.3.1	Risk Register	79
9.3.2	Lessons from Materialised Risks.....	80
9.4	Quality Management.....	80
9.4.1	Code Quality	80
9.4.2	Model Quality.....	81
9.4.3	System Quality.....	81
9.5	Social, Legal, Ethical and Professional Considerations	81
9.5.1	Data Protection and Privacy.....	81
9.5.2	Ethical Considerations	82
9.5.3	Legal Considerations	82
9.5.4	Professional Considerations	82
9.6	Summary	83
10	Critical Appraisal.....	84
10.1	Introduction	84
10.2	Achievement of Research Objectives.....	84
10.3	Strengths of the Project.....	84
10.3.1	Identifying and Quantifying the Real-World Accuracy Gap	84
10.3.2	How Mixed Training Closes the Gap and Outperforms Both Baselines	85
10.3.3	Consistent Noise Profile Between Training and Deployment.....	85
10.3.4	Balanced and Reproducible Evaluation Design.....	85
10.3.5	Privacy by Architecture	85
10.3.6	Cost-Effectiveness and Viability at Scale	85
10.4	Limitations and Weaknesses	89
10.4.1	WER Measured on Synthesised Speech, Not Real Guests.....	89
10.4.2	Synthetically Generated Training Data.....	89
10.4.3	Tokenizer Mismatch on Android.....	89
10.4.4	No Field Testing in a Real Hotel.....	89
10.4.5	Prototype-Scale Architecture	89
10.4.6	English Language Only	89

10.4.7 Summary of Limitations	90
10.5 Technical Decisions - What Worked and What Could Be Improved.....	90
10.5.1 What Worked Well	90
10.5.2 What Could Be Improved	91
10.6 Knowledge and Expertise Gained.....	91
10.7 Summary	92
11 Conclusions.....	93
11.1 Introduction	93
11.2 Achievements.....	93
11.3 Future Work.....	94
11.3.1 Field Deployment and User Study.....	94
11.3.2 Real Speech Data Collection and Model Retraining.....	94
11.3.3 Proper WordPiece Tokenizer on Android	94
11.3.4 Multilingual Support	94
11.3.5 Further NLU and Dialogue Improvements.....	94
11.4 Summary	95
12 Student Reflections	96
12.1 Introduction	96
12.2 Personal Growth and Skills Development.....	96
12.3 Problems Encountered and How They Were Resolved.....	96
12.3.1 The Vosk Model Memory Problem	96
12.3.2 The TFLite Conversion and Label Ordering Bug	96
12.3.3 The Tokeniser Compromise.....	97
12.3.4 The WebSocket Screen-Off Disconnection	97
12.3.5 Keeping Scope Under Control	97
12.4 What I Would Do Differently	97
12.4.1 Start Writing Earlier.....	97
12.4.2 Collect Real Hotel Data Earlier.....	97
12.4.3 Build in Automated Tests from the Start.....	98
12.4.4 Use Structured Logging from the Beginning.....	98
12.5 Reflections on the Research Process	98
12.5.1 Iterative Development Was the Right Approach.....	98
12.5.2 The Gap Between Papers and Working Systems	98
12.5.3 Working Across Disciplines.....	98
12.6 Summary	99
References	100
Appendix A – Project Specification	1
Appendix B – Interim Progress Report and Meeting Records.....	2
Appendix C – Requirements Specification Document	3
C.1 Survey Questionnaire	3
C.2 Respondent Demographics	3
C.3 Key Findings Summary.....	4

Appendix D – Code Base	5
Appendix E – User Manual.....	6
Appendix F – Project Presentation	7
Appendix G – List of Figures.....	8
Appendix H – List of Tables.....	9
Appendix I – Glossary of Abbreviations and Acronyms	11

Acknowledgements

I would like to thank my supervisor, Mr. Dumira Athukorala, for his guidance and support throughout this project. His feedback shaped the direction of this research at every stage. I also thank Course Director Mr. Dileepa Samayawardena for his guidance on the overall research process and methodology.

Sincere gratitude is extended to the academic staff at Coventry University, UK and the National Innovation Centre (NIC), National Institute of Business Management (NIBM), for their valuable guidance, constructive feedback, academic mentorship, and for providing the resources and environment that made this research possible.

I am grateful to Mr. Nandana Kaluarachchi, Executive Chef at Cinnamon Wild Yala, and his staff, for their time and insights during the requirements gathering stage. Their practical perspective on hotel operations was invaluable in defining the scope and direction of this research.

I also thank my friends who shared their hotel stay experiences, which helped me better understand the guest perspective and the practical limitations of current room service communication in Sri Lanka.

Finally, I am grateful to my family for their encouragement and patience throughout my studies.

1 Introduction

1.1 Introduction

This chapter introduces the research context, motivation, and scope of the study. It covers the background, core problem, four research objectives, and the overall structure of the dissertation.

1.2 Background to the Project

The hospitality industry is a significant contributor to the Sri Lankan economy. Over 2 million international tourists were visited Sri Lanka in 2024, with India accounting for approximately 20% of all arrivals (Sri Lanka Tourism Development Authority (SLTDA), 2024). Hotels are always looking for ways to improve guest experience while managing operational costs. Despite this, room service operations in most hotels still rely on traditional methods like telephone calls to the front desk and manual request handling. A practitioner consultation conducted as part of this research confirmed that this is the usual way in Sri Lankan hotels (Section 4.2.1). These approaches are slow, prone to miscommunication, and hotel management sometimes find it challenging to track what was requested, when it was fulfilled, or how long it took, especially during busy periods.

With the rise of voice-based virtual assistants such as Amazon Alexa for Hospitality and Google Nest for Hotels has introduced new possibilities for automating guest service operations. However, these commercial solutions have major drawbacks for hotels in developing countries. They rely entirely on cloud infrastructure, require continuous internet connectivity, and send guest voice data to third-party servers raising privacy concerns. Buhalis and Moldavska (2022) found that 67% of guests feel uncomfortable with cloud-connected microphones in their hotel rooms, and documented risks of third-party accessing guest audio without consent. Beyond privacy, the cost structure of proprietary hardware at USD 99 - 200 per device combined with ongoing subscription fees is not feasible for most hotels in Sri Lanka.

Internet connectivity adds another layer of difficulty. Though Sri Lanka's infrastructure has improved in urban areas, hotels outside Colombo regularly experience outages or poor connectivity (Wickramasinghe & Ratnayake, 2022). A system that stops working whenever the internet is unavailable is not suitable for this market. These limitations make existing commercial voice assistants impractical for the majority of Sri Lankan hotels, despite the clear operational need.

This project addresses that gap directly. It proposes and builds “**Sera**” a voice assistant designed specifically for hotel room service that runs entirely on a standard Android tablet without any internet dependency. All speech recognition and intent classification happen on the guest's device using two small, open-source AI models: Vosk for offline speech-to-text and MobileBERT for intent classification. No voice data leaves the room. Requests are routed automatically to the correct hotel department over the local network, and staff manage all incoming requests through a web-based dashboard in real time. The total hardware cost per room sits below USD 150 within reach of small independent hotels.

The research question guiding this study is: ***Can a low-cost, fully offline voice assistant system, built on a standard commodity device, achieve sufficient technical accuracy and performance to be a viable alternative to traditional room service communication in Sri Lankan hotels?***

1.3 Project Objectives

The main objectives of this project are:

1.3.1 Design and Develop a Low-Cost, Offline Voice Assistant Prototype

Design and build a fully offline, end-to-end voice assistant prototype covering on-device speech recognition, intent classification, and the complete request lifecycle including submission, confirmation, cancellation, and guest rating which is deployable on commodity Android hardware in the USD 50-150 price range, without any cloud or internet dependency.

1.3.2 Build a Hospitality-Domain Dataset and Train a Noise-Aware NLU Model

Create a labelled intent classification dataset covering the most common room service request categories in Sri Lankan hotels and train a MobileBERT-based intent classifier that can handle the accuracy challenges introduced by on-device speech recognition, specifically the transcription noise produced by the Vosk Speech-to-Text (STT) engine under real deployment conditions.

1.3.3 Build a Lightweight Backend with Real-Time Staff Communication

Implement a lightweight FastAPI and SQLite backend with automated department routing and a browser-accessible staff dashboard, supporting real-time bidirectional communication between guest devices and hotel staff via WebSocket, without any cloud infrastructure.

1.3.4 Evaluate System Accuracy, Latency, and Cost-Effectiveness

Evaluate the prototype's intent classification accuracy, speech recognition word error rate, end-to-end response latency, and per-room hardware cost against the requirements that defined for a viable room service alternative and demonstrate that a fully privacy-preserving, offline voice processing architecture is achievable on low-cost commodity hardware.

1.4 Overview of This Report

The remainder of this dissertation is structured as follows.

Chapter 2 - Literature Review surveys the academic and commercial background of voice assistants in hospitality, offline speech recognition systems, small-scale Natural Language Understanding (NLU) models for resource-constrained devices, and privacy-preserving edge AI architectures. It identifies the research gaps that this project addresses.

Chapter 3 - Methodology describes the overall research approach. A Design Science Research (DSR) framework combined with iterative software development, and a controlled three-model NLU experiment to evaluate the accuracy impact of real on-device STT conditions.

Chapter 4 - Requirements covers requirements elicitation from the academic literature, a practitioner consultation, and a guest survey, and defines the functional and non-functional requirements used to evaluate the prototype.

Chapter 5 - Analysis translates the requirements into a system architecture and technology choices, justifying the selection of Vosk, MobileBERT, FastAPI, and Android as the core technology stack.

Chapter 6 - Design details the system design. The three-tier architecture, data model, API contracts, Android UI flows, and the hybrid NLU pipeline combining rule-based keyword matching with MobileBERT neural inference.

Chapter 7 - Implementation documents how the system was built across four development iterations, covering the dataset construction pipeline, model training, Android application, and backend service.

Chapter 8 - Testing presents the evaluation results: the three-model NLU accuracy comparison, Vosk word error rate analysis, end-to-end latency measurements, and Non-Functional Requirement (NFR) compliance assessment.

Chapter 9 - Project Management reflects on the development process, timeline, risk management, and the iterative decisions made during implementation.

Chapter 10 - Critical Appraisal honestly assesses the limitations of the prototype, including the synthetic evaluation data, tokenizer mismatch on Android, and the constraints on generalizability.

Chapter 11 - Conclusions summarizes the findings, revisits the research question, and outlines the most important directions for future work.

Chapter 12 - Student Reflections offers a personal account of the learning experience, challenges encountered, and insights gained throughout the project.

1.5 Research Contributions

The individual techniques used in this project are not new; each has been used in prior research. The novelty lies in the combination; applying these techniques together in a specific context: offline hospitality voice assistance on low-cost Android hardware for the Sri Lankan hotel market. No prior work addresses all of these together.

The specific contributions are:

1. **Hospitality-domain intent dataset:** 10,080 labelled phrases across 18 intent categories, paired with Vosk-transcribed versions for noise-aware training. To the best of the author's knowledge, this is the first publicly documented intent classification dataset for Sri Lankan hotel service operations.
2. **Noise-aware NLU training for offline STT pipelines:** a controlled three-model experiment demonstrating that mixing clean and Vosk-transcribed training data significantly closes the accuracy gap introduced by real on-device speech recognition, in the specific context of offline hospitality deployment.
3. **Fully offline, low-cost voice assistant prototype for the hospitality domain:** a working end-to-end system on commodity Android hardware, with no cloud dependency and no voice data leaving the device; targeting a deployment context not addressed by existing commercial solutions.
4. **Hybrid NLU pipeline for resource-constrained on-device deployment:** a two-tier classification approach combining rule-based keyword matching with MobileBERT neural inference, designed for the specific constraints of offline Android deployment in the hospitality domain, where keyword matching handles clear requests and the neural model handles everything else.

1.6 Summary

The Sri Lankan hospitality sector has a clear operational gap: commercial voice assistants are too expensive, too cloud-dependent, and too privacy-invasive for most hotels in the market. The four objectives in Section 1.3 address this gap through the design, development, and evaluation of a fully offline, low-cost voice assistant system. The research question asks whether such a system can achieve sufficient technical accuracy and performance to be a viable alternative to traditional room service communication. The remaining chapters build toward answering that question. Chapter 2 surveys the existing academic and commercial work and identifies the specific gaps this project addresses.

2 Literature Review

2.1 Introduction

The global hotel industry is one of the largest service sectors in the world, generating over USD 1.5 trillion in revenue annually and employing more than 300 million people across its supply chains (World Travel and Tourism Council, 2023). Guest experience is one of the most important factors that determines a hotel's success, and in recent years hotels have been investing in digital technologies such as mobile check-in systems, smart room controls, and AI-powered voice assistants to improve the quality of their services.

Voice-based AI assistants in hotel rooms have received growing attention from both researchers and the industry. Buhalis and Moldavska (2021) conducted a qualitative study through 28 semi-structured interviews with hospitality technology providers and hotel guests to examine how in-room voice AI assistants affect hotel services and guest experiences. The study found that voice assistants were used for room environment controls (lighting, temperature), room service ordering, wake-up alarms, and general information queries. Guests reported benefits including convenience, accessibility, and faster service. Buhalis and Moldavska (2022) further published a detailed analysis in the *Journal of Hospitality and Tourism Technology*, confirming that voice assistants can reduce the burden on front desk staff and allow guests to access services without making phone calls or visiting reception.

Amazon's Alexa for Hospitality (now Alexa Smart Properties), launched in 2018, became the most widely deployed commercial voice assistant in the hotel industry. Major hotel chains including Marriott International, Hilton and Best Western, deployed Amazon Echo devices in guest rooms with customized skills for room service ordering, housekeeping requests and facility information (Marriott International, 2018). A 2025 study published in the *Journal of Theoretical and Applied Electronic Commerce Research* used structural equation modelling on 529 survey responses and found that voice assistant attributes including connectivity, information provision, and interactivity positively influence guest evaluations and online hotel reputation.

In Sri Lanka, the tourism industry is an important part of the national economy. Sri Lanka attracted over 2 million international tourists in 2024, with continued growth across cultural, wellness, and eco-tourism segments (SLTDA, 2024). A 2022 study published by Springer looked at the digital readiness of Sri Lanka's four major tourism sectors hoteliers, restaurateurs, airlines, and tour operators and found that while leading hotel chains have adopted basic digital tools such as online booking systems and mobile applications, the use of advanced AI-powered guest services is still very limited. This shows that there is a significant opportunity to introduce affordable and practical AI solutions in Sri Lanka's hotel industry.

2.2 Challenges Hotel Businesses Face in Sri Lanka

Sri Lankan hotels face several practical challenges that make it difficult to adopt mainstream commercial voice assistant solutions.

High cost of commercial solutions Alexa for Hospitality requires proprietary Echo hardware (USD 99–200 per device) plus AWS subscription fees that grow with usage. For a mid-range Sri Lankan hotel deploying across 50 rooms, the combined cost is not financially sustainable. Buhalis and Moldavska (2022) noted this barrier but did not suggest a more affordable alternative.

Accent and language mismatch. Commercial speech recognition systems are mainly trained on North American and British English and perform poorly on non-native accents. This is a direct

problem in Sri Lanka where both guests and hotel staff typically speak South Asian English. India is consistently Sri Lanka's largest source of tourists (416,974 arrivals in 2024, approximately 20% of all international visitors - SLTDA, 2024), making South Asian accent robustness a direct operational requirement.

Internet connectivity dependency and cost. Cloud voice assistants like Alexa for Hospitality rely entirely on a live AWS connection; if the internet drops, the system stops working. While Sri Lanka's connectivity has improved (Starlink satellite internet launched in 2023, reaching rural tourism areas such as hill country resorts and the Arugam Bay and Pasikuda coasts), internet remains a managed cost rather than a free utility. A fully offline system removes this dependency entirely.

Limited IT expertise. The 2022 Springer study found that most Sri Lankan hotel operators lack in-house technical staff capable of setting up and maintaining cloud-connected AI systems, creating vendor dependency and adding cost and downtime risk.

Operational inefficiency. Without automated routing, every guest request, whether intended for housekeeping, maintenance, or the kitchen, passes through the front desk first. This creates bottlenecks during busy periods and increases miscommunication risk. Hwang and Erdem (2025) confirmed that voice assistants reduce repetitive front desk calls, but only when the system is well-designed for the operational context.

Privacy risks in cloud-connected deployments. Buhalis and Moldavska (2022) found 67% of guests uncomfortable with cloud-connected microphones in hotel rooms and documented risks of third-party skills accessing guest audio without consent. Hotels serving European guests face GDPR exposure, and this risk grows as data protection legislation expands across Asia.

2.3 Impact of the Challenges

These challenges have direct consequences for guest experience, operational efficiency, and market competitiveness.

- Slow service delivery reduces guest satisfaction. Research consistently identifies response speed as one of the most important factors in hotel guest satisfaction (Buhalis & Moldavska, 2022), and the 2025 JTAECR study confirmed that hotels with responsive voice systems receive higher guest evaluation scores than those relying solely on telephone-based request handling.
- Without automated routing, every request passes through the front desk before reaching the right department; delaying resolution and removing any management visibility into service performance or recurring problems.
- As global booking platforms increasingly list smart-room features as searchable amenities, hotels without technology-enhanced services, risk losing higher-spending international guests.
- Combined with expanding data protection legislation across Asia and Europe, cloud-based deployments that send guest audio to overseas servers carry growing legal exposure even without an actual breach.

2.4 Solutions Proposed in Existing Research and Industry

Researchers and technology companies have proposed various solutions to address voice-driven hotel service delivery. These can be grouped into three categories: commercial cloud platforms, offline speech recognition, and compact NLU models for on-device processing.

2.4.1 Commercial Cloud-Based Voice Assistants

Amazon's Alexa for Hospitality is the most widely deployed commercial solution. The platform allows hotels to manage devices at a property level, create custom voice skills, and includes some privacy features such as a microphone disconnect button. Hwang and Erdem (2025) studied how this system affects hotel staff workload and found it can reduce repetitive tasks and improve response times. The 2025 JTAECR study also confirmed that voice assistant features such as interactivity and information provision positively affect hotel reputation through improved guest evaluations.

2.4.2 Edge-Based and Offline Speech Recognition

Vosk (Alpha Cephei) is an open-source offline speech recognition toolkit supporting over 20 languages with model sizes ranging from 50MB to 1.8GB. Korkmaz et al. (2025) built a prototype edge-based STT system using Vosk on a Raspberry Pi and showed it can provide real-time, low-latency speech processing without cloud connectivity. Chantrapornchai and Suchato (2022) similarly demonstrated that offline speech recognition on a Raspberry Pi is practical for command-style IoT control tasks.

Radford et al. (2023) introduced Whisper, a large model trained on 680,000 hours of multilingual audio that handles diverse accents and background noise well. However, its computational requirements; especially for the medium and large variants; make it unsuitable for running on commodity mobile hardware.

2.4.3 Compact NLU Models for On-Device Intent Classification

Bidirectional Encoder Representations from Transformers BERT (Devlin et al., 2019) established bidirectional pre-training as the standard for natural language understanding, but its 110 million parameters (~440MB) make it too large for mobile deployment. Several compressed alternatives have been developed:

Sanh et al. (2019) introduced DistilBERT, which retains 97% of BERT's language understanding at 40% smaller size and 60% faster inference using knowledge distillation. Sun et al. (2020) proposed MobileBERT, designed specifically for mobile devices, achieving 4.3× compression and 5.5× faster inference versus BERT-BASE with only 0.6 points lower on the GLUE benchmark; running at 62ms per inference on a Google Pixel 4.

For server-side NLU, Bunk et al. (2020) introduced Rasa DIET, which handles both intent classification and entity recognition in a single model while training six times faster than fine-tuned BERT. Bujel et al. (2021) showed that fine-tuning BERT on as few as 1,000 domain-specific examples produces better results than general pre-trained models for specialized intent classification.

2.4.4 Privacy-Preserving Edge Computing

Shi et al. (2016) established the concept of edge computing in their widely-cited IEEE IoT Journal paper, showing how processing data locally reduces latency, improves privacy, and decreases bandwidth usage. Korkmaz et al. (2025) demonstrated that fully offline on-device models can

reach competitive accuracy for domain-specific tasks while ensuring voice data never leaves the user's device; something cloud systems cannot guarantee regardless of their privacy policies.

2.5 *How Existing Solutions Have Helped Address the Challenges*

Existing solutions have made progress in addressing individual parts of the problem, and there is good evidence from the literature to support their effectiveness.

Cloud-based commercial voice assistants have clearly shown that hotel guests find value in voice interaction. Buhalis and Moldavska (2021) found that the benefits of hotel voice assistants consistently outweigh the drawbacks for both guests and hotel operators. Hwang and Erdem (2025) reported concrete improvements: fewer repetitive calls to front desk staff, faster delivery of information, and a more modern perception of the hotel's service. The 2025 JTAECR study also confirmed that interactive voice assistants positively affect hotel reputation scores.

Edge-based speech recognition research has shown that offline STT is technically possible on standard hardware. Korkmaz et al. (2025) demonstrated that Vosk-based edge STT can match cloud solutions in terms of response speed for short command-style inputs. Custom domain language models for Vosk have been shown to reduce Word Error Rate by up to 40% compared to generic offline models (Alpha Cephei, 2023), which substantially closes the accuracy gap with cloud-based alternatives. Chantrapornchai and Suchato (2022) confirmed that Raspberry Pi devices can handle real-time offline speech recognition for command recognition tasks.

Compact NLU models have shown that on-device natural language understanding is feasible. MobileBERT's 62ms inference latency on a Google Pixel 4 (Sun et al., 2020) shows that transformer-based intent classification is practical for real-time use on mobile devices. DistilBERT (Sanh et al., 2019) demonstrated that knowledge distillation can produce smaller models without a large drop in accuracy. Wang et al. (2025) survey on-device AI optimisation techniques, including parallel decoder architectures that achieve significant inference speedups with substantially fewer parameters.

Privacy-preserving design has also been technically validated. Korkmaz et al. (2025) showed that fully local processing can reach competitive accuracy while eliminating the data exposure risk that comes with cloud-based systems.

However, while each of these solutions addresses one aspect of the problem; whether it is offline STT, on-device NLU, or privacy; none of them combines all of these aspects into a complete, integrated system designed specifically for hospitality, running on affordable hardware, and tested in a developing-country context.

2.6 *Knowledge Contribution of This Research and Model Performance*

2.6.1 Identified Research Gaps

Table 2.1 compares existing solutions across seven important dimensions to show where this research contributes.

Solution	Offline Capable	On-Device Processing	Hospitality Domain	Low-Cost Hardware	Privacy Preserving	Staff Integration	End-to-End System
Buhalis & Moldavska (2021, 2022)	No	No	Yes	No	No	No	No
Alexa for Hospitality	No	No	Yes	No	Partial	Yes	Yes
Korkmaz et al. (2025) – Edge STT	Yes	Yes	No	Yes	Yes	No	No
Custom Vosk LM (Alpha Cephei, 2023)	Yes	Yes	No	Yes	Yes	No	No
Chantrapornchai & Suchato (2022)	Yes	Yes	No	Yes	Yes	No	No
Whisper (Radford et al., 2023)	Partial	No	No	No	Partial	No	No
MobileBERT (Sun et al., 2020)	Yes	Yes	No	Yes	Yes	No	No
DistilBERT (Sanh et al., 2019)	Yes	Partial	No	Partial	Yes	No	No
Rasa DIET (Bunk et al., 2020)	Partial	No	No	No	Partial	No	Partial
This Research	Yes	Yes	Yes	Yes	Yes	Yes	Yes

Table 2-1: Comparison Matrix of Existing Solutions

As shown in the table, none of the existing solutions satisfies all seven dimensions at the same time. Four research gaps are identified:

Gap 1 - STT Noise Impact on NLU (Core Research Gap): No published research has measured how much accuracy is lost when offline STT transcription noise (specifically from Vosk) is passed to a downstream NLU intent classifier, and no research has shown that training the NLU model on STT-transcribed data can recover this accuracy. NLU models are typically trained and evaluated on clean text, but in real offline deployments the model receives noisy, error-prone STT output. This mismatch between training conditions and deployment conditions has not been studied or addressed in the hospitality domain. This is the primary research gap that this study addresses.

Gap 2 - On-Device NLU for Hospitality in a Real STT Pipeline: No published research evaluates fine-tuned compact transformer models (MobileBERT/TFLite) on a hospitality-specific intent dataset within a real offline STT pipeline. Existing NLU research either uses clean text inputs or operates in a server-based environment, neither of which reflects how such a model actually performs when receiving Vosk-transcribed speech on a mobile device.

Gap 3 - Integrated Offline Hospitality System: No existing work combines offline STT, on-device NLU, privacy-preserving local processing, and real-time staff dashboard integration into a single end-to-end system designed for hospitality. Individual components have been studied in isolation, but no research has integrated them into a complete, working hotel voice assistant that runs without cloud connectivity and routes requests to department-specific staff in real time.

Gap 4 - Developing Economy Context: No such system has been designed, built, and evaluated for the specific operational and economic conditions of hotels in developing economies such as Sri Lanka, where low hardware cost, local serviceability, and freedom from recurring cloud fees are essential requirements for practical adoption.

2.6.2 Empirical Approach to Addressing Gap 1

To address Gap 1, three MobileBERT models were trained on a custom hospitality intent dataset covering 18 intent categories; using clean text only (Model A), Vosk-transcribed text only (Model B), and a mixed clean+Vosk dataset (Model C). All three were converted to TensorFlow Lite and evaluated on the same held-out test set of Vosk-transcribed prashes. The full results, including accuracy figures, confusion matrices, and per-intent breakdowns, are presented in Chapter 8.

2.7 Technology Tools and Their Suitability

Each technology was selected based on specific performance, cost and deployment requirements identified in Sections 2.2 and 2.3.

2.7.1 Vosk for On-Device Speech-to-Text

Vosk was chosen because it works fully offline and the vosk-model-small-en-in-0.4 variant (Indian English, ~50MB) is suited to the South Asian English accents of Sri Lankan hotel guests and staff. At 50MB it fits comfortably on a commodity Android tablet. Korkmaz et al. (2025) showed that Vosk achieves comparable latency to cloud STT for short command-style inputs, and custom domain language models can reduce Word Error Rate by up to 40% (Alpha Cephei, 2023), providing a clear path for further accuracy improvement.

2.7.2 MobileBERT and TensorFlow Lite for On-Device NLU

MobileBERT was chosen over DistilBERT and Rasa DIET because it was designed specifically for mobile inference. It converts to TFLite with a 26MB footprint and runs at 62ms per inference on a Pixel 4. DistilBERT has not been optimised for TFLite on Android; Rasa DIET requires a server process, which conflicts with the on-device requirement. MobileBERT's TFLite conversion produces a 26MB model that runs at 62ms per inference on a Pixel 4 (Sun et al., 2020); meeting the on-device latency requirement without cloud dependency.

2.7.3 Android Tablets for Hardware Deployment

Android tablets were chosen over Amazon Echo or Raspberry Pi for three reasons: **cost** (USD 50 - 150 in Sri Lanka vs USD 200+ for smart speakers), **availability** (local technicians can source and repair them without specialist knowledge or importing parts), and **platform support** (well-established TFLite, Vosk, and WebSocket tooling). The 2022 Springer study on Sri Lankan digital tourism found that locally serviceable hardware is a key requirement for technology adoption in this market.

2.7.4 FastAPI and WebSocket for Backend Communication

FastAPI handles both HTTP and WebSocket communication efficiently with low overhead, suitable for running on standard hotel hardware within a local network. WebSockets provide real-time two-way communication between guest devices and the staff dashboard; a capability not offered by Alexa for Hospitality. Running the backend on hotel hardware means no recurring hosting fees and all guest data stays on-premises, consistent with the edge computing privacy model described by Shi et al. (2016).

2.7.5 Summary of Tool Suitability

Table 2.2 shows how each selected tool addresses the challenges identified in Section 2.2.

Challenge (Section 2.2)	Selected Tool	Research Gap Addressed
Internet dependency and cost	Vosk offline STT	Gap 3 - fully offline, no cloud dependency
High commercial platform cost	Android tablets + open-source stack	Gap 4 - USD 50–150 hardware, zero licensing fees
Non-native accent degradation	Vosk Indian English model	Gap 1 - acoustically suited to South Asian English
STT noise degrading NLU accuracy	Noise-aware MobileBERT training	Gap 1 - core research contribution
Limited IT expertise	Android + FastAPI	Gap 4 - widely used platforms, locally serviceable
Operational inefficiency	WebSocket real-time routing	Gap 3 - department-specific request routing
Privacy and data exposure	On-device processing	Gap 3 - all voice data stays within hotel LAN

Table 2-2: Challenge-to-Tool Mapping

2.8 Summary

This chapter reviewed the existing literature and identified the research gaps this study addresses. Section 2.2 described the main barriers preventing Sri Lankan hotels from adopting mainstream voice assistant solutions; internet dependency, high cost, accent mismatch, limited IT expertise, and privacy risk. Section 2.3 showed how these barriers affect guest experience, operational efficiency, and market competitiveness.

Sections 2.4 and 2.5 reviewed existing solutions: commercial platforms like Alexa for Hospitality, offline STT research using Vosk, and compact NLU models like MobileBERT and DistilBERT. While each addresses part of the problem, none provides a complete integrated offline hospitality system.

Section 2.6 identified four research gaps. The core gap; Gap 1 is the absence of any measurement of how offline STT transcription noise degrades downstream NLU accuracy, and the absence of a noise-aware training strategy to recover it. Section 2.7 mapped each identified challenge to the selected technology tools.

The following chapter presents the research methodology used to design, build, and evaluate the complete system.

3 Methodology

3.1 Introduction

This chapter explains how the system was designed, built, and evaluated. It covers the research approach, development process, architecture, technology selections, dataset preparation, model training, and evaluation strategy. Every major decision connects back to the core constraint: building an affordable, offline voice assistant that actually works on budget hardware in a Sri Lankan hotel.

3.2 Research Approach

This project follows **Design Science Research (DSR)**, as described by Hevner et al. (2004). DSR is about building and evaluating IT artefacts that solve real, practical problems. It was the right approach here for three reasons:

1. **The research needs a working system.** Whether an offline voice assistant performs well on budget hardware can only be measured by actually building one. Surveys or case studies cannot produce sufficient evidence.
2. **The research gap is practical.** To the best of the author's knowledge, no existing system has combined offline on-device NLU with open-source edge STT for hospitality. Closing that gap means building a real implementation, not a theoretical proposal.
3. **Evaluation is part of the process.** DSR requires the artefact to be tested against defined criteria, here covering NLU accuracy under real pipeline conditions, speech recognition accuracy using Word Error Rate (WER), system latency, and deployment cost.

The DSR process for this project followed five phases:

1. **Problem Identification** - Identifying operational and technical gaps through the literature review (Chapter 2).
2. **Solution Design** - Defining system requirements, architecture and design of three-model NLU training experiment(Chapters 3-6).
3. **Artefact Development** - Building the Android application, backend server, staff dashboard, and three NLU model variants (Chapter 7).
4. **Evaluation** - Measuring NLU accuracy across training conditions, speech recognition WER, and deployment cost (Chapter 8).
5. **Communication** - Reporting findings, limitations, and contributions (Chapters 10 and 11).

3.3 Development Approach

The system was built using **iterative prototyping** rather than a sequential process. This approach was essential as there were uncertainties before any code was written. The system combines on-device speech recognition, neural intent classification, and real-time local network communication. None of these could be fully designed upfront because their real-world behavior on budget hardware was unknown.

- Speech recognition accuracy could only be measured by running Vosk on an actual Android device.
- Intent classification performance depends on how the model handles imperfect STT output, which is only visible after running the full pipeline end-to-end.
- End-to-end latency could only be measured with all components integrated and running together.

A sequential approach would have delayed these findings until it was too late to act on them. For example, the first prototype showed that the larger Vosk model caused load times exceeding 15 seconds on a budget tablet, directly affecting the final model selection.

The system was built across four iterations:

Iteration	Focus	Key Output	Key Decision Made
1	On-device speech recognition	Vosk integrated into Android app with live transcription	vosk-model-small-en-in-0.4 selected, as Indian English acoustic model matches Sri Lankan pronunciation better than US English variants
2	Intent classification pipeline	MobileBERT fine-tuned, converted to TFLite, deployed on-device	Hybrid classification implemented, as the neural model alone struggled on simple keyword requests
3	Backend and real-time communication	FastAPI server, SQLite database, WebSocket integration	WebSocket confirmed as viable for real-time guest-to-staff updates over a local network
4	System integration and evaluation	End-to-end system with staff dashboard and benchmarking	Full pipeline tested; NLU accuracy compared across three training conditions

Table 3-1: Development Iterations

Each iteration ended with a review against the research objectives, and what was learned fed into the next phase.

3.4 System Architecture

The system is built on a **three-tier architecture**:

- A guest-facing Android application that handles all on-device speech and language processing.
- A central hotel server that manages request routing and storage.
- A web-based staff dashboard for day-to-day operations.

All three tiers run entirely within the hotel's local area network (LAN). No data leaves the premises. Running both STT and NLU directly on the guest device was a deliberate design choice for two reasons:

- **Privacy**: raw audio never leaves the device.
- **Reduced server load**: the server only receives structured text requests.

The detailed system design is covered in Chapter 6.

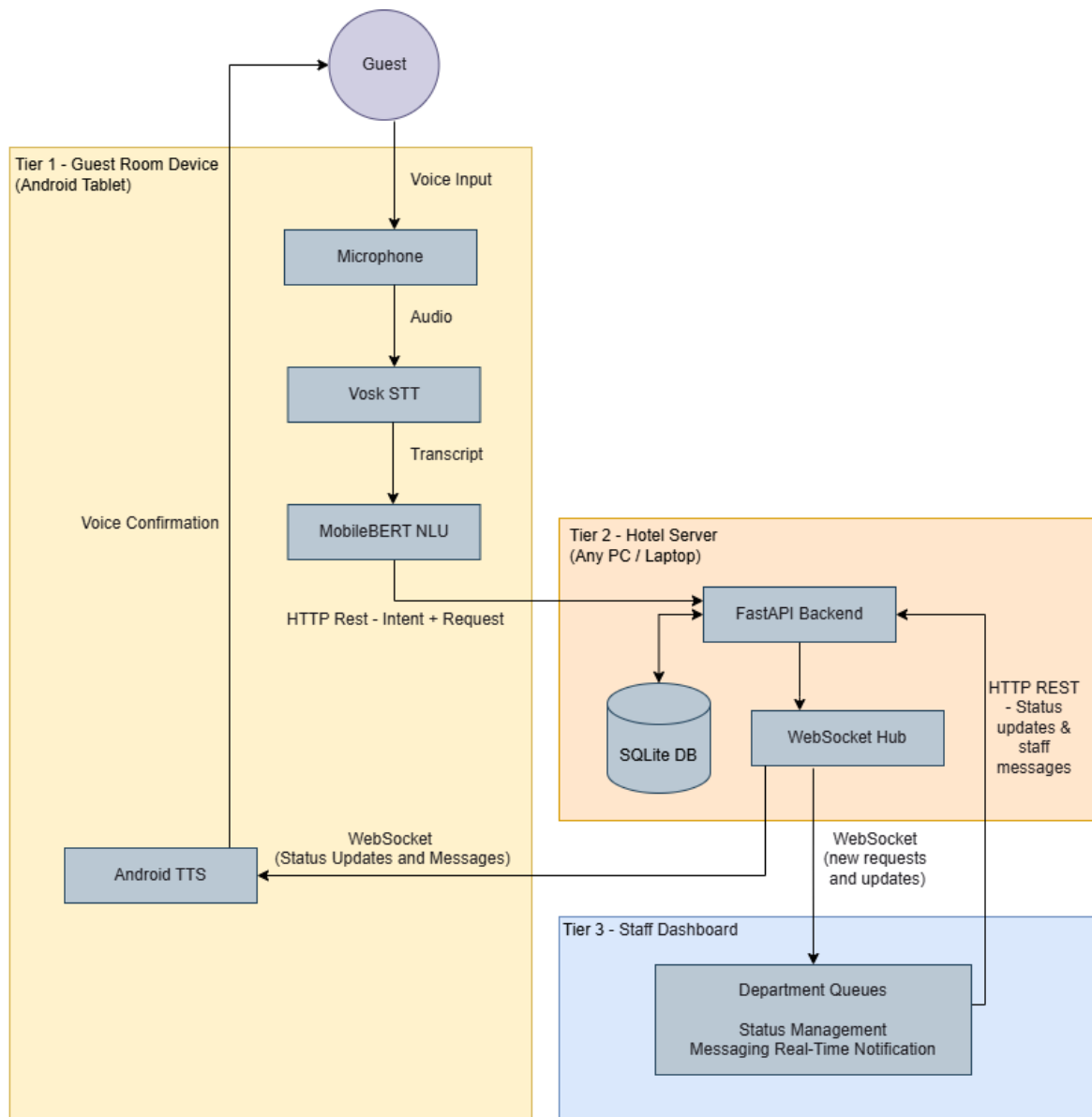


Figure 3-1: Three-tier Architecture

3.5 Technology Selection

The technology selection for each component; speech recognition, intent classification, backend framework, database, mobile platform, and text-to-speech are evaluated and justified in Chapter 5 (Section 5.6) against the project constraints formally defined in Chapter 4 (NFR-01 to NFR-04): offline capability, privacy, low latency and low-cost hardware.

3.6 Dataset Preparation

3.6.1 Intent Category Design

The dataset covers 18 intent categories derived from Sri Lankan hotel room service menus, the hospitality use cases in Buhalis & Moldavska (2021, 2022) and the service categories supported by Alexa for Hospitality. The full category list is defined in Chapter 4 (Table 4-2) as part of functional requirement FR-03.

3.6.2 Dataset Construction

The clean dataset (`new_hotel_dataset.csv`) contains 10,080 labelled phrases exactly 560 examples per intent across all 18 categories, giving a perfectly balanced distribution. The average sentence length is 7.16 words. All text was normalised to lowercase to match Vosk's output format.

The dataset was generated using the Claude Haiku API to produce natural language variations, then expanded with paraphrasing to cover formal, casual, indirect, and abbreviated phrasing styles. All phrases were written in Vosk output style lowercase, no punctuation, merging without apostrophes.

3.6.3 Vosk Transcription Pairing - The Core Experimental Dataset

This step is the most important part of the data preparation process. Each of the 10,080 clean phrases was converted to audio using Google Text-to-Speech (TTS) (gTTS with `tlid='co.in'` for Indian English pronunciation), converted to 16 kHz mono WAV using `ffmpeg`, and then transcribed by Vosk (`vosk-model-small-en-in-0.4`). The result is a paired dataset (`vosk_transcriptions.csv`) where every phrase has both a clean version and a Vosk-transcribed version side by side.

This pairing deliberately introduces the same STT noise which a real guest voice would produce when passed through the on-device Vosk engine. Without it, there would be no way to isolate the accuracy drop caused by the speech recognition step. The WER and Character Error Rate (CER) characteristics of these transcriptions are measured and reported in Chapter 8 (Section 8.3).

3.6.4 Three Training Datasets

Three training datasets were derived from the paired data, one for each model variant:

Dataset	Records	Contents	Model Trained
<code>new_hotel_dataset.csv</code>	10,080	Clean text only	Model A (baseline)
<code>vosk_only_dataset.csv</code>	10,080	Vosk-transcribed text only	Model B
<code>paired_dataset.csv</code>	14,863	Clean + Vosk mixed (duplicates removed)	Model C (proposed approach)

Table 3-2: Training Datasets

`paired_dataset.csv` has 14,863 records rather than 20,160. Because phrases where Vosk produced output identical to the clean text were removed to avoid redundancy.

3.6.5 Train / Validation / Test Split

Each model is trained using a stratified 85% / 15% train / validation split of its respective training dataset. The test set is 20% of `vosk_transcriptions.csv` (2,016 samples) that was set aside and never used for training. All three models were evaluated on the same test set to ensure a fair comparison.

3.7 Model Training

3.7.1 MobileBERT Fine-Tuning

All three model variants are fine-tuned from the same base model, google/mobilebert-uncased from HuggingFace, using an identical training configuration. The only variable across models is the training data. This ensures that any difference in results reflects the impact of training data composition on NLU performance, and nothing else. Table 3-3 summarises the training configuration applied consistently across all three model variants.

Hyperparameter	Value	Rationale
Base Model	google/mobilebert-uncased	Smallest BERT variant designed for on-device deployment
Task	Multi-class classification (18 intents)	-
Epochs	5 (with early stopping, patience = 2)	Training stops if F1 macro does not improve for 2 consecutive epochs
Learning Rate	3e-5	Standard for BERT fine-tuning (Devlin et al., 2019)
Batch Size	16	Suited to CPU training constraints
Optimiser	AdamW	Standard for transformer fine-tuning
Warmup Ratio	10% of training steps	Stabilises early training
Weight Decay	0.01	L2 regularisation
Max Gradient Norm	1.0	Gradient clipping
LR Scheduler	Linear decay	-
Max Sequence Length	32 tokens	Over 98% of hotel requests are under 10 words
Best Model Selection	Highest F1 macro on validation set	-
Seed	42	Reproducibility

Table 3-3: Training Configuration

3.8 Evaluation Methodology

The evaluation covers four dimensions; each tied directly to a research objective.

3.8.1 NLU Accuracy – Three Model Comparison

This is the central evaluation of the research. The three model variants (A, B, C) are each tested on the same 2,016-sample test set under two input conditions:

- **Clean text:** Evaluates performance without any transcription errors (upper-bound condition).
- **Vosk-transcribed text:** Evaluates performance under real pipeline conditions, where the accuracy gap becomes visible.

Metrics reported for each model and condition:

- **Accuracy:** Overall proportion of correct classifications.
- **Precision, Recall, F1-score:** Per-intent and macro-averaged.
- **Confusion matrix:** To identify systematic misclassification between semantically similar intents.

The key comparison is between Model A and Model C on Vosk output. This shows whether noise-aware training produces NLU accuracy sufficient for real pipeline conditions, one of the core technical dimensions of system viability.

3.8.2 Speech Recognition Accuracy

Vosk's transcription quality is measured using **WER**, the standard metric for speech recognition systems, calculated as:

$$\text{WER} = (\text{Substitutions} + \text{Insertions} + \text{Deletions}) / \text{Total Reference Words}$$

Vosk transcriptions from the pipeline are compared against the original clean text using the jiwer Python library. The WER is broken down by intent category to identify which service request types are most affected by transcription errors. This per-intent view is important because it directly shows which NLU categories are most likely to lose accuracy in a real deployment.

CER applies the same edit-distance calculation at the character level, dividing character-level substitutions, insertions, and deletions by the total number of reference characters. CER is typically lower than WER and gives a finer-grained view of transcription distortion; small misspellings that WER counts as full word errors appear proportionally smaller in CER.

$$\text{CER} = (\text{Character Substitutions} + \text{Insertions} + \text{Deletions}) / \text{Total Reference Characters}$$

3.8.3 System Latency

End-to-end response time was measured from voice input completion through to the staff dashboard update. Timestamps were logged at each pipeline stage: STT processing, NLU classification, HTTP submission, WebSocket delivery, and TTS confirmation. Results are reported as observed during integration testing.

3.8.4 Cost-Effectiveness

Per-room deployment cost was compared across this system and cloud-based alternatives (Google Cloud STT, Alexa for Hospitality), covering hardware, software, and ongoing fees projected over three years for a hypothetical 50-room hotel.

3.9 Summary

This chapter described the research methodology, dataset preparation, model training, and evaluation strategy. Technology selection is covered in Chapter 5 (Section 5.6) and the detailed system design in Chapter 6.

DSR guided the overall process, iterative prototyping shaped how the system was built, and the paired clean and Vosk-transcribed dataset was the core experimental contribution enabling a controlled three-model comparison under real deployment conditions. Key decisions, including vosk-model-small-en-in-0.4 for Indian English acoustic matching, the hybrid NLU pipeline, and WebSocket-based LAN communication, all driven by the constraints of budget Sri Lankan hotels. The next chapter documents the full functional and non-functional requirements that shaped these decisions.

4 Requirements

4.1 Introduction

This chapter covers how the system requirements were gathered and prioritised. Since this is a research prototype without a hotel industry client, the requirements came from three sources: published academic literature on hotel operations and voice assistant adoption (to understand what hotels actually need), an informal consultation with an experienced Sri Lankan hotel professional (to confirm that the operational problems identified in the literature are real and current), and a small survey of people with recent hotel stay experience (to understand what guests want). Together, these requirements define what "viable alternative to traditional room service communication" means in the Sri Lankan hotel context and they set the standard against which the prototype is later evaluated.

4.2 Requirements Elicitation

4.2.1 Hotel Operational Requirements - Literature Review

There is no hotel client for this project, so the operational requirements had to come from somewhere else. The approach used here, drawing requirements from published research when no direct client exists, is standard practice in Design Science Research (Hevner et al., 2004). The literature review in Chapter 2 covered several studies on voice assistant adoption and hotel operations (Buhalis & Moldavska, 2021, 2022; Hwang & Erdem, 2025; Yilmaz et al., 2025), and these were combined with Sri Lanka-specific sources (Wickramasinghe & Ratnayake, 2022; SLTDA, 2024) to understand what operational problems the system actually needs to solve.

Key findings from the literature:

1. **Telephone-based service creates bottlenecks.** All guest requests going through a single front desk line means that during busy periods, calls go unanswered, staff juggle multiple requests at once, and response times suffer (Buhalis & Moldavska, 2022). Hwang & Erdem (2025) confirm this from the staff side, showing that manual request handling is one of the main workload problems frontline hotel staff face.
2. **Miscommunication is a real problem, especially with international guests.** When requests are handled verbally over the phone, things get misunderstood, the wrong items get delivered, or the request goes to the wrong department. Buhalis & Moldavska (2021) point to this as one of the main reasons hotels have started looking at voice assistants: a structured, intent-based system removes the guesswork from freeform phone conversations. Yilmaz et al. (2025) also connect these kinds of service errors directly to negative online reviews.
3. **Most hotels have no way to track requests.** Most independent hotels have no proper system for logging what was requested, when it was fulfilled, or how long it took (Buhalis & Moldavska, 2022). When a guest asks about a pending request, staff have to physically check with colleagues. A simple digital log with real-time status updates would directly fix this.
4. **Subscription costs stop hotels from adopting commercial solutions.** Commercial platforms need dedicated hardware per room, ongoing subscription fees, and reliable internet, none of which fits the budget of an independent hotel (Buhalis &

Moldavska, 2022). This is especially relevant in Sri Lanka, where the accommodation sector is mostly small, owner-operated properties (SLTDA, 2024).

5. **Internet connectivity is unreliable outside Colombo.** Wickramasinghe & Ratnayake (2022) document that digital infrastructure quality varies a lot across Sri Lanka, with hotels outside the capital regularly losing connectivity for hours at a time. A system that needs constant internet access is simply not practical for this market. This finding directly drove the offline-first requirement.

In addition to the literature, an informal consultation was conducted with a Head Chef with over 15 years of experience across multiple Sri Lankan hotels. He described the current process: guest requests arrive at the front desk by telephone, the front desk forwards them to the relevant department, and the department manager records them in a physical logbook. Items are only entered into a computer system at final billing. No automated routing or real-time tracking is involved at any stage. This directly confirms Findings 1, 3, and 4 above, and provides local practitioner evidence that the problem this system addresses is real and current.

4.2.2 Guest Survey

A short survey was given to 20 individuals with recent hotel stay experience. Respondents were selected through convenience sampling; family members, relatives, and colleagues known to the researcher and all had stayed in at least one hotel within the past 12 months. The sample covered a range of ages: four teenagers (under 20), seven young adults (20-35), five middle-aged adults (36-59), and four older adults (60-70). This spread was intentional, since voice assistant comfort levels tend to vary across age groups. The full survey instrument and a demographic summary are in Appendix C.

The survey covered:

- How respondents currently make room service requests and whether they had experienced problems
- How comfortable they would be using a voice assistant for hotel services
- Privacy concerns about voice-enabled devices in hotel rooms
- Which services they would use most through a voice interface
- How they would want the system to confirm a request before submitting it

Key findings:

1. **75% (15/20)** said they had experienced delays or miscommunication when making service requests by phone during a hotel stay. This directly supports what the literature says about telephone-based service being a bottleneck.
2. **65% (13/20)** said they would use a voice assistant for room service if they knew the audio was not recorded or sent anywhere outside the room. Privacy assurance was the key factor and several respondents who initially said they would not use one changed their answer once the on-device processing was explained.
3. The most commonly requested services were towels and bedding (90%), housekeeping (85%), food and beverage orders (80%), and toiletries (65%) which directly shaped the 18 intent categories built into the system.
4. **80% (16/20)** preferred voice input with an on-screen text confirmation before the request was submitted, rather than voice-only interaction. This was especially strong for food

orders, where mistakes are most troublesome, and directly forced the design of the confirmation step in the guest application.

5. Privacy was a important concern. **25% (5/20)** said they would be uncomfortable with a cloud-connected voice device in their hotel room. This reinforced the decision to keep all audio processing on the device and not transmit any voice data externally.

4.3 Functional Requirements

The requirements below came from combining the literature findings (Section 4.2.1) with the guest survey results (Section 4.2.2). Where a requirement came from only one source, that is noted. Requirements supported by both sources were treated as higher priority.

ID	Requirement	Priority	Source
FR-01	The system shall accept guest service requests through voice input	Must	Guest survey; Buhalis & Moldavska (2022)
FR-02	The system shall convert speech to text on the device without internet connectivity	Must	Wickramasinghe & Ratnayake (2022) connectivity constraints
FR-03	The system shall classify guest requests into one of 18 predefined hotel service intent categories	Must	Buhalis & Moldavska (2021, 2022) routing needs
FR-04	The system shall display a text confirmation of the recognized request before submission	Must	Guest survey preference for confirmation step
FR-05	The system shall provide voice feedback confirming that a request has been submitted	Must	Guest survey
FR-06	The system shall deliver requests to the staff dashboard in real time over the hotel's local network	Must	Hwang & Erdem (2025) response time concerns
FR-07	The system shall allow staff to update request status (pending, in progress, completed, cancelled)	Must	Buhalis & Moldavska (2022) no existing tracking method
FR-08	The system shall route requests to the appropriate hotel department automatically based on intent	Must	Buhalis & Moldavska (2022) coordination challenges
FR-09	The system shall enable bidirectional messaging between staff and the guest's room device	Should	Hwang & Erdem (2025)
FR-10	The system shall allow guests to cancel a pending request by voice command	Should	Guest survey
FR-11	The system shall allow guests to rate a completed service on a 1–5 scale	Should	Yilmaz et al. (2025) service quality measurement
FR-12	The system shall allow staff to transfer a request to a different department via the dashboard	Should	Buhalis & Moldavska (2022)
FR-13	The system shall maintain a record of all requests for operational review	Could	Buhalis & Moldavska (2022)

Table 4-1: Functional Requirements

The 18 intent categories in FR-03 were defined by looking at room service menus from Sri Lankan hotels, the use cases in Buhalis & Moldavska (2021, 2022), and the service categories supported by Alexa for Hospitality.

Intent Category	Description
towel_request	Requests for towels
room_cleaning	Housekeeping and cleaning requests
food_order	Food, beverage, and room service orders
toiletries_request	Bathroom amenity requests
pillow_request	Pillow and bedding requests
temperature_control	Heating and cooling requests
blanket_request	Blanket and comforter requests
maintenance	Technical and repair requests
laundry_service	Laundry and dry cleaning requests
concierge_general	General information and service queries
wake_up_call	Alarm and wake-up requests
concierge_taxi	Transportation and taxi requests
do_not_disturb	Privacy and do-not-disturb requests
lighting_control	Light adjustment requests
noise_complaint	Noise and disturbance complaints
emergency	Emergency and urgent assistance
checkout_billing	Checkout and billing inquiries
misc_request	Uncategorised or general requests

Table 4-2: Predefined Intent Categories (FR-03)

4.4 Non-Functional Requirements

The non-functional requirements define the constraints the system has to work within. Each one follows directly from the findings in Section 4.2, with sources noted in the table.

ID	Requirement	Target	Rationale
NFR-01	The system shall operate fully without internet connectivity	Full offline capability	Unreliable connectivity at target hotel locations
NFR-02	All voice processing shall occur on-device; no audio or transcript shall be transmitted externally	Zero external voice data	Privacy concerns from guests and literature
NFR-03	End-to-end response time from voice input to voice confirmation shall not exceed 5 seconds	< 5 seconds	Guest experience - comparable to a telephone response
NFR-04	The system shall run on commodity Android tablets costing under \$150 USD	< \$150 per room	Budget constraints of target small hotels
NFR-05	The system shall achieve a minimum intent classification accuracy of 90% on real speech input	≥ 90%	Reliability threshold for service routing
NFR-06	The system shall support concurrent operation across multiple room devices on a single hotel server	Multi-room	Practical deployment on a single on-site machine

NFR-07	The staff dashboard shall be accessible from any device with a web browser without installation	Browser-only access	Minimize additional hardware requirements for staff
--------	---	---------------------	---

Table 4-3: Non-Functional Requirements

4.5 Requirements Prioritisation

Once the requirements were defined, they were prioritised using the *Must have, Should have, Could have, Won't have* (MoSCoW) method. The split between Must Have and Should Have was based on what the literature points to as essential for adoption in small hotel settings, versus what would be useful but is not critical (Buhalis & Moldavska, 2022; Hwang & Erdem, 2025).

Priority	Requirements
Must Have	FR-01, FR-02, FR-03, FR-04, FR-05, FR-06, FR-07, FR-08, NFR-01, NFR-02, NFR-04
Should Have	FR-09, FR-10, FR-11, FR-12, NFR-03, NFR-05, NFR-06
Could Have	FR-13, NFR-07
Won't Have (this release)	Multilingual support, voice-based guest feedback collection, integration with existing hotel Property Management System (PMS) software

Table 4-4: MoSCoW Prioritisation

The "Won't Have" items are deliberate scope decisions. Multilingual support would be highly valuable, but it is out of scope for this prototype. These are discussed as future work in Chapter 11.

4.6 Summary

Requirements were gathered from three sources: published academic literature on hospitality operations, an informal consultation with an experienced Sri Lankan hotel professional, and a convenience survey of 20 people with recent hotel stay experience (Section 4.2). Using all three gave a clearer picture than any single source alone would have.

The picture that emerged is consistent: hotels need something that removes the telephone bottleneck, gives staff a way to track requests, and can be deployed without subscription fees or IT support. Guests want it to be fast, straightforward, and above everything else, not sending their voice data anywhere outside their room.

These requirements directly shaped the design: offline operation, on-device audio processing, commodity Android hardware, and no subscription fees. A prototype that fails on any one of these would not be a credible alternative to traditional room service communication in a Sri Lankan hotel. Multilingual support is picked up as future work in Chapter 11. The next chapter covers the technology choices and analysis that turned these requirements into a working design.

5 Analysis

5.1 Introduction

This chapter takes the requirements from Chapter 4 and translates them into system design decisions. It covers the overall architecture, use cases, data model, and technology selection, focusing on why many common options did not fit the project constraints.

5.2 High-Level System Conceptual View

The system is split into three layers, all running over the hotel's local Wi-Fi. There is no internet connection needed for any day-to-day system operation.

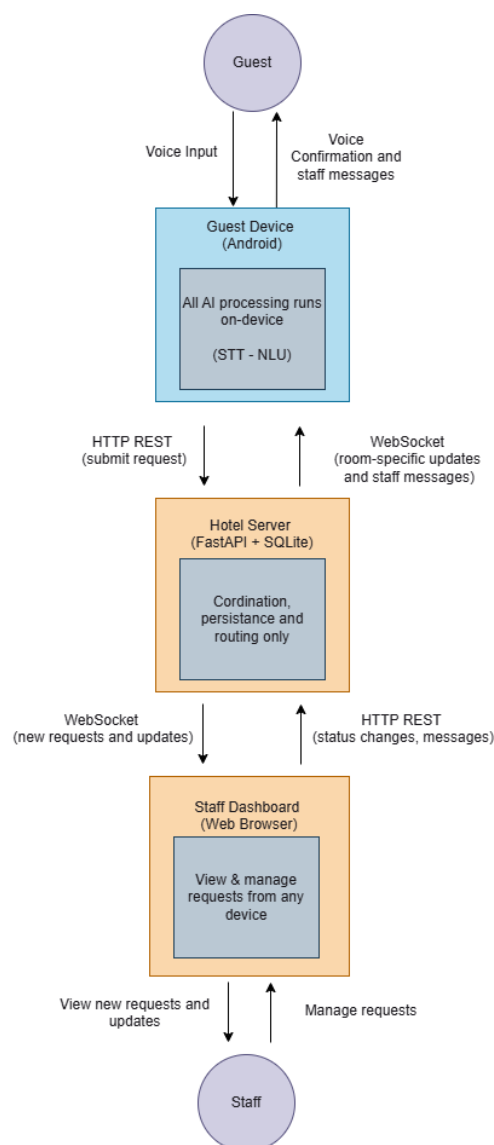


Figure 5-1: High-Level System Conceptual View

All AI processing, speech recognition and intent classification, happens on the guest device. The server is deliberately lightweight: it stores requests, routes them to departments, and pushes real-time updates to connected clients. One known limitation: if the server is temporarily unreachable, STT and NLU still work on-device, but the guest cannot submit the request. The app prompts them to use the room phone instead. There is no local queue, if submission fails, the request is lost. Raw audio never leaves the tablet at any point, satisfying NFR-02.

5.3 Use Case Model

There are two actors: the **Guest**, who interacts through voice and touch on the Android tablet, and the **Staff Member**, who uses the web dashboard to manage requests.

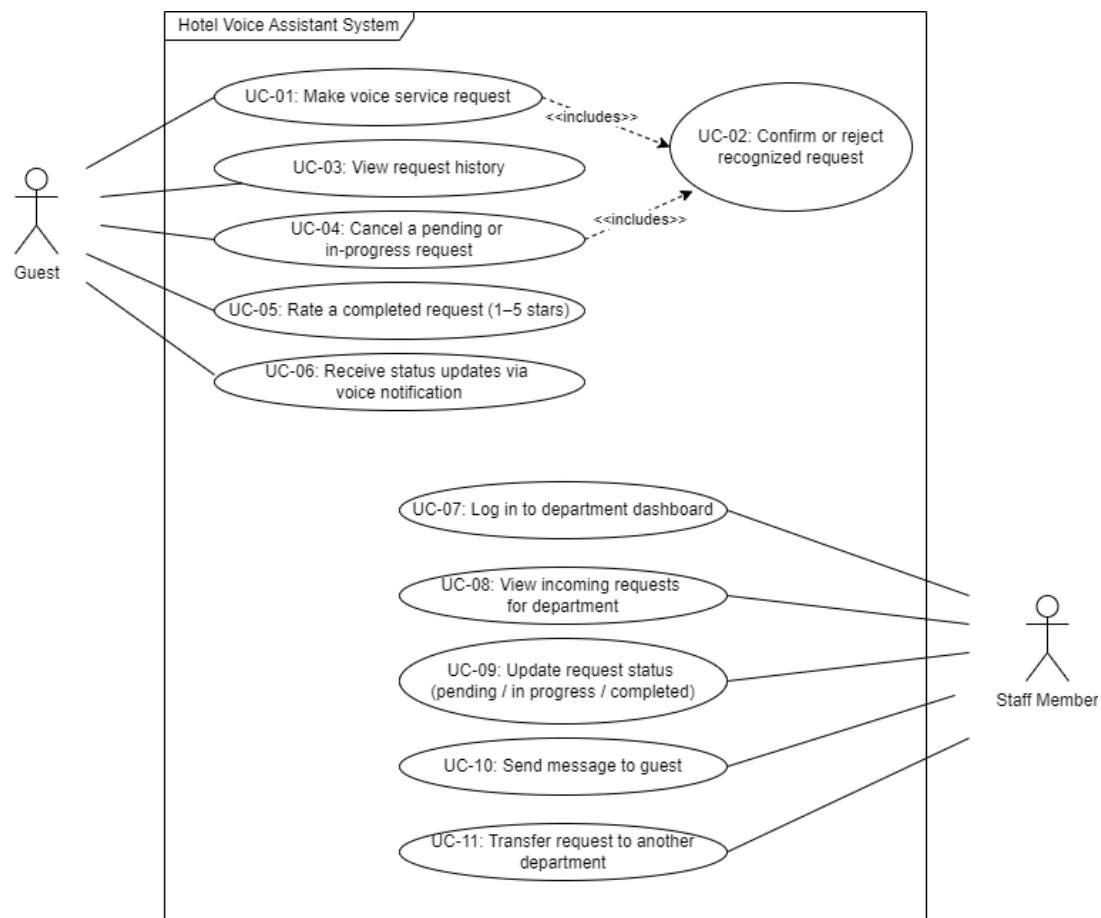


Figure 5-2: Unified Modelling Language (UML) Use Case Diagram

5.3.1 Use Case Descriptions

ID	Use Case	Actor	Description
UC-01	Make voice service request	Guest	Guest presses the microphone button and speaks a service request. The system records audio, transcribes it using Vosk, classifies the intent using the hybrid NLU pipeline, and presents a confirmation screen.
UC-02	Confirm or reject request	Guest	The system reads back the recognized request via Android TTS. Guest says "yes" to confirm or "no" to reject. On

			confirmation, the request is submitted to the server via HTTP POST.
UC-03	View request history	Guest	Guest views a scrollable list of current and past requests with live status indicators, sorted by priority: in progress → pending → completed → cancelled
UC-04	Cancel request	Guest	Guest says "cancel order [number]". The system extracts the request ID using regex, confirms via voice, and sends a cancel request to the server.
UC-05	Rate completed service	Guest	After a request is marked completed, the guest can submit a 1-5 star rating. The rating is sent to the server and becomes visible on the staff dashboard.
UC-06	Receive real-time updates	Guest	When staff update a request status or send a message, the guest device receives a WebSocket notification and announces it via TTS.
UC-07	Log in to dashboard	Staff	Staff member selects their department and enters their name to access their department-filtered request queue.
UC-08	View department requests	Staff	Dashboard shows all requests routed to the staff member's department, with status badges, timestamps, and room numbers.
UC-09	Update request status	Staff	Staff clicks "In Progress" or "Complete". The change is immediately broadcast to the relevant guest device via WebSocket.
UC-10	Send message to guest	Staff	Staff types a message (e.g., "We'll be there in 5 minutes") which is delivered to the guest device and read aloud via TTS.
UC-11	Transfer request	Staff	Staff reassigns a request to a different department using a dropdown selector. The request immediately appears in the new department's queue.

Table 5-1: Use Case Descriptions

5.4 Process Flow Analysis

5.4.1 Voice Request Pipeline

The voice pipeline covers three stages from microphone press to staff dashboard: audio capture and STT on-device (Figure 5.3), hybrid intent classification, confirmation and submission (Figure 5.4). The specific parameter values chosen for each stage are covered in the design chapter.

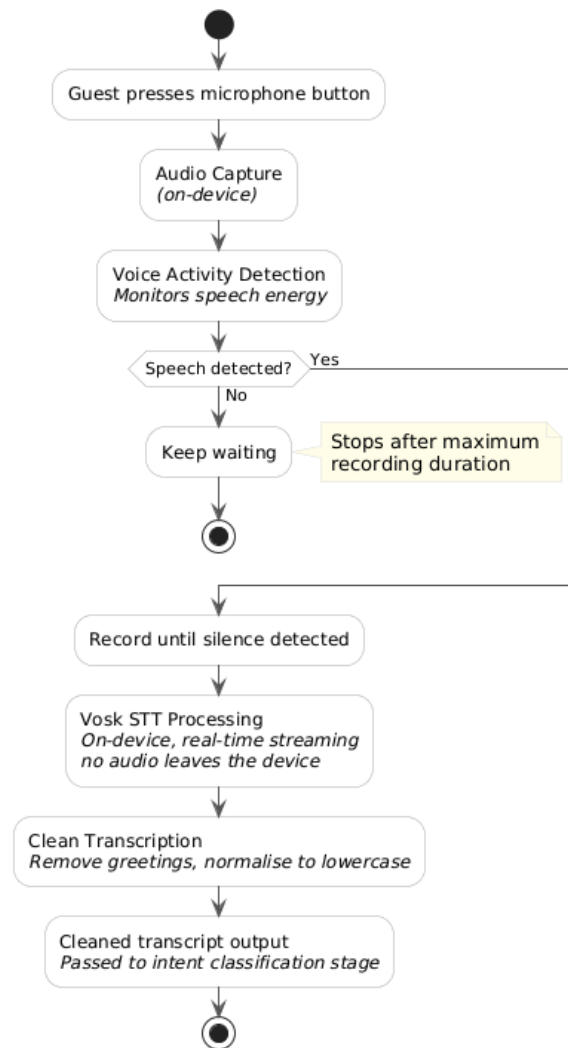


Figure 5-3: Audio Capture and Speech Recognition

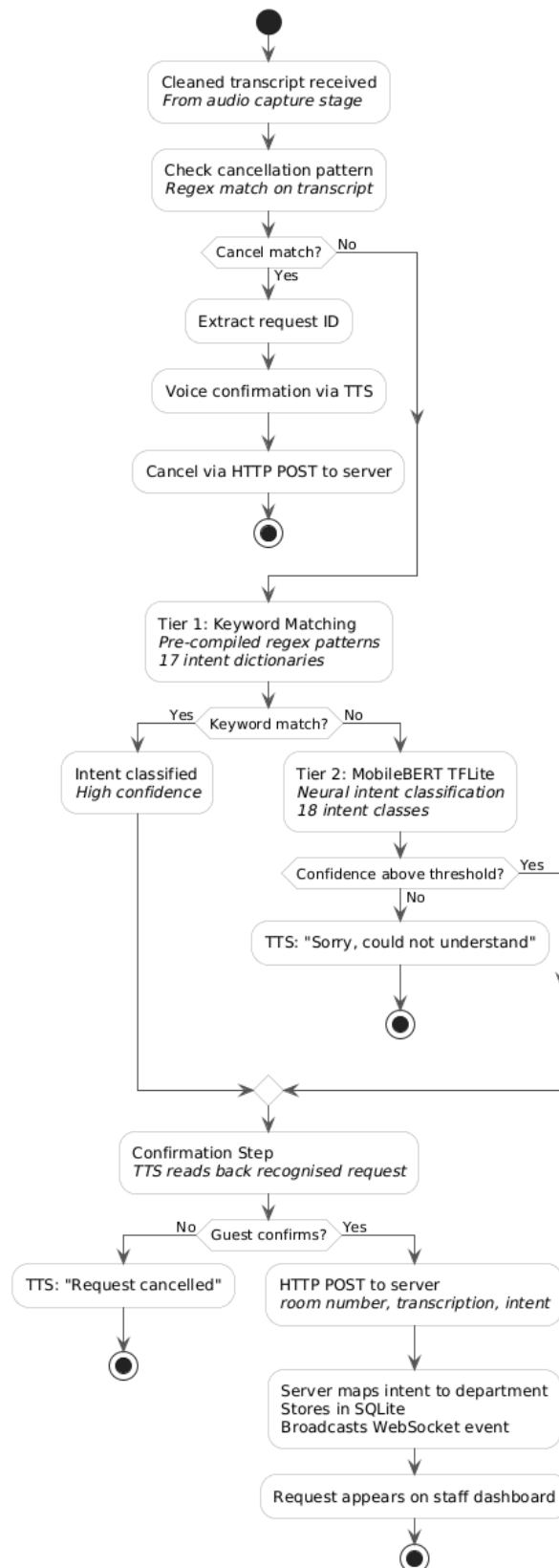


Figure 5-4: Intent Classification and Request Submission

5.4.2 Staff Request Handling Flow

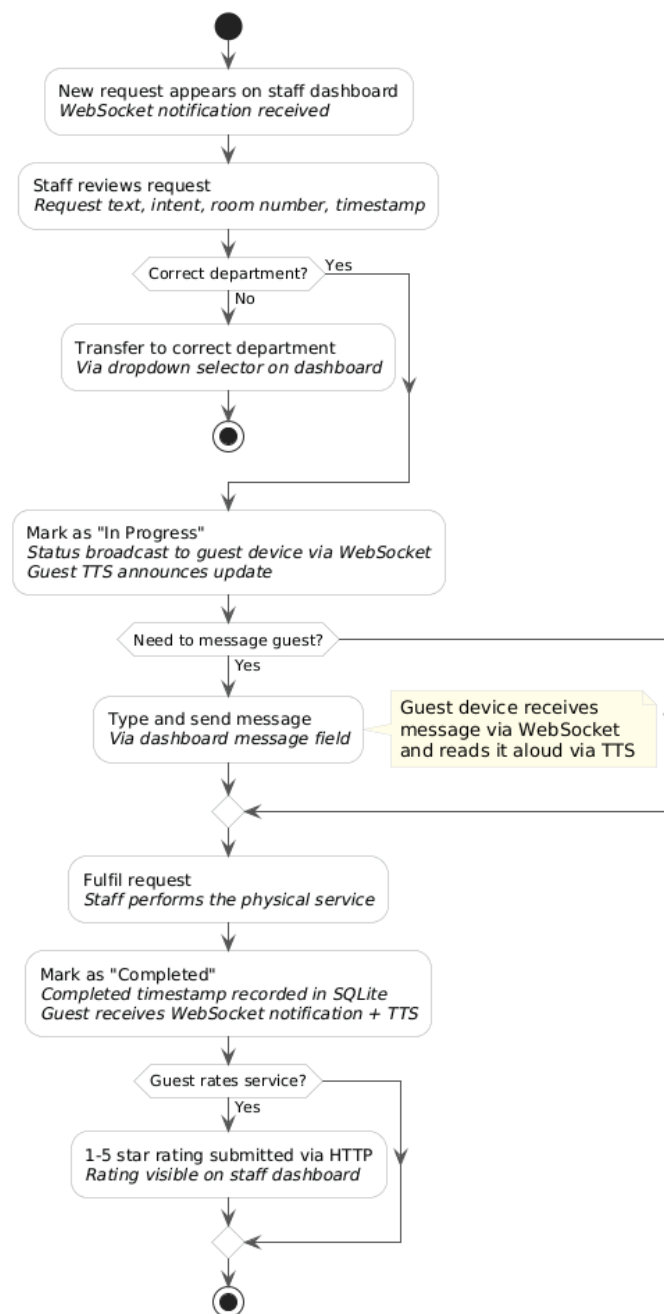


Figure 5-5: Staff Request Handling Flow

5.5 Entity-Relationship Model

The data model has five tables:

requests, staff_messages, rooms, departments and intent_department_mapping.

Full table descriptions are in Section 6.4.

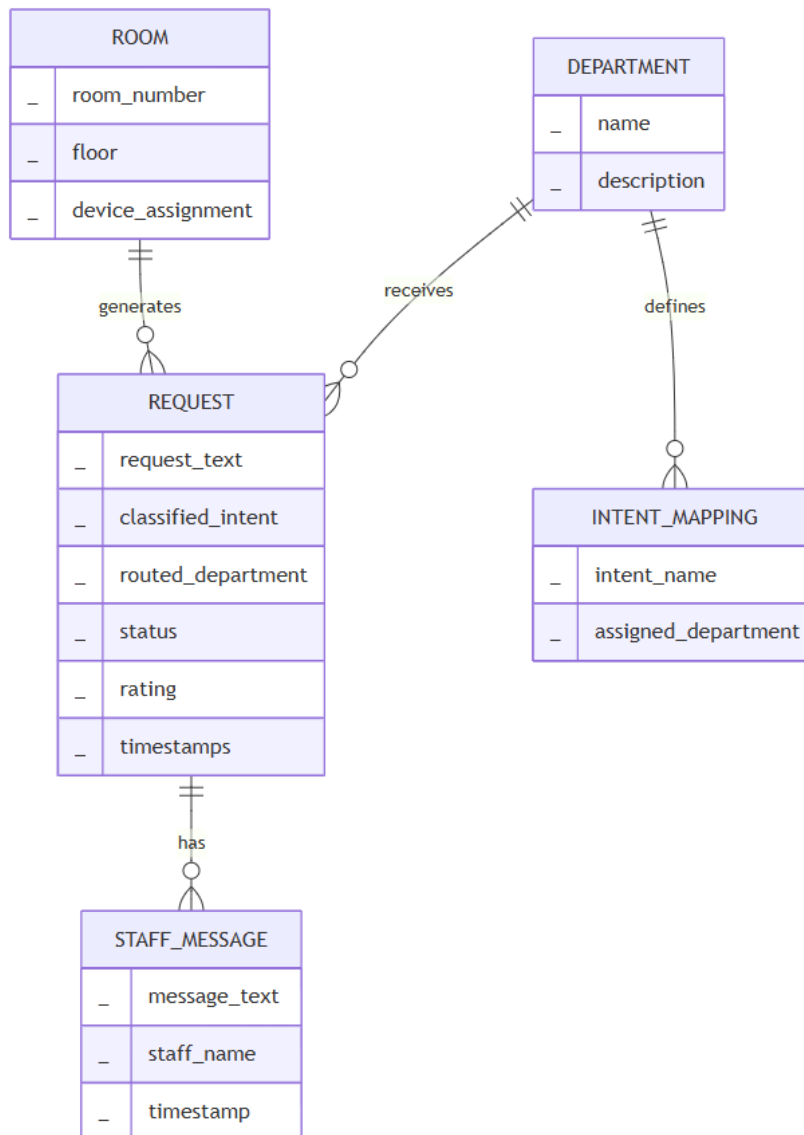


Figure 5-6: Entity-Relationship Diagram

5.6 Technology Evaluation and Selection

Every component was evaluated against four constraints from Chapter 4:

- Offline capability (NFR-01)
- Privacy (NFR-02)
- Low response latency (NFR-03)
- Low hardware cost (NFR-04)

These four constraints are not just engineering preferences; they are what determine whether the system is viable as an alternative to traditional room service communication.

The prototype-vs-production distinction matters here. Some choices, like SQLite over PostgreSQL, are practical for a research prototype but would need to change at production level. Where that is the case, the production alternative is identified explicitly. Table 5.2 gives the full picture of what was selected and why.

Component	Prototype Selection	Production Recommendation	Primary Reason for Prototype Choice
Speech Recognition	Vosk (vosk-model-small-en-in-0.4, ~36MB)	Custom Vosk language model fine-tuned on hospitality vocabulary	Real-time streaming; native Android SDK; fully offline; Indian English accent support
Intent Classification	MobileBERT TFLite (hotel_mobilebert.tflite, 26MB) + Rule-based hybrid	Expanded dataset; multi-language models	Purpose-built for mobile; ~62ms latency; no server required
Backend Framework	FastAPI (Python)	FastAPI with load balancing	Native async + WebSocket; Python ML ecosystem
Database	SQLite	PostgreSQL	Zero-configuration; no DBA required; single-file
Real-Time Communication	WebSocket (OkHttp / Starlette)	WebSocket with message queue (Redis)	Full-duplex; no broker dependency
Mobile Platform	Android Native (Kotlin/Jetpack Compose)	Android with Mobile Device Management (MDM) provisioning	Low-cost hardware; native ML SDK; local availability
Text-to-Speech	Android Native TTS	Piper TTS (custom hotel voice model)	Pre-installed; zero cost; no additional model required
Staff Dashboard	Web-based (HTML/CSS/JS)	React/Vue with role-based auth	Browser-accessible; zero installation; rapid iteration
Server Configuration	Single IP/port (SharedPreferences)	MDM-managed certificates	Sufficient for controlled prototype evaluation

Table 5-2: Technology Stack Summary

5.6.1 Speech Recognition: Vosk vs Whisper vs Cloud APIs vs CMU Sphinx

Speech-to-text is the entry point of the whole pipeline, so this was the most important component decision. Four options were evaluated.

Vosk is an open-source offline speech recognition toolkit from Alpha Cephei. It supports over 20 languages with model sizes ranging from 50MB to 1.8GB (Alpha Cephei, 2023) and runs entirely on the device with real-time streaming, no network needed.

Whisper from OpenAI (Radford et al., 2023) was trained on 680,000 hours of multilingual audio and achieves very strong accuracy. Models range from ~150MB (tiny) to ~6GB (large). The tiny

variant can technically run on Android, but it works in batch mode, it waits until the recording is done before processing, rather than transcribing in real time.

Google Cloud Speech-to-Text and Amazon Transcribe are commercial cloud services with excellent accuracy, but both require a constant internet connection and charge per minute of audio. Google Cloud STT costs \$0.006 per 15 seconds (Google Cloud, 2024), an ongoing cost that does not work for the target hotels.

CMU Sphinx / PocketSphinx (CMU Sphinx, 2023) is an older offline engine (~30MB) built on pre-deep-learning acoustic models (Huggins-Daines et al., 2006). It has seen very little development since 2019 and its accuracy is noticeably below modern systems.

Criteria	Vosk (~36MB)	Whisper (tiny)	Google Cloud STT	CMU Sphinx
Offline Capable	Yes	Yes	No	Yes
Model Size	~36MB	150MB	N/A (cloud)	~30MB
Real-Time Streaming	Yes	No (batch)	Yes	Yes
Accuracy (General English)	Good	Good	Excellent	Fair
South Asian Accent Robustness	Good (Indian EN model)	Good	Excellent	Poor
Native Android SDK	Yes	No (wrapper required)	Yes	Yes
Computational Requirements	Low	Moderate	N/A	Very Low
Cost	Free	Free	\$0.006/15 sec	Free
Active Development	Yes	Yes	Yes	Minimal
Satisfies NFR-01 (Offline)	✓	✓	X	✓
Satisfies NFR-02 (Privacy)	✓	✓	X	✓

Table 5-3: Speech Recognition Technology Comparison

Selection: Vosk (vosk-model-small-en-in-0.4)

Vosk was chosen for three reasons.

- First, real-time streaming: Vosk processes audio as the guest speaks. Whisper waits for the full recording before processing, adding latency that makes the interaction feel unnatural and that risks missing the 5-second target (NFR-03).
- Second, Vosk has a native Android SDK (Alpha Cephei, 2023). Running Whisper on Android would require ONNX conversion and a custom inference wrapper, extra complexity for a research prototype. vosk-model-small-en-in-0.4 at ~36MB fits within budget tablet memory constraints (Chapter 7), and was trained on Indian English data, a closer acoustic match to Sri Lankan English vowel quality, intonation, and consonant patterns than any US English model. India is also Sri Lanka's largest source of international visitors (SLTDA, 2024), making this the most relevant accent profile for the system.
- Third, privacy: audio never leaves the tablet, satisfying NFR-02 directly. Cloud options were ruled out for violating both NFR-01 and NFR-02. CMU Sphinx was ruled out for poor accuracy and lack of active maintenance.

Production pathway: A custom Vosk language model fine-tuned on hospitality vocabulary, room service items, department names would reduce WER for domain-specific terms while keeping STT fully on-device.

5.6.2 Intent Classification: MobileBERT vs DistilBERT vs Rasa DIET vs Rule-Based

Once the guest's speech is transcribed, the system needs to figure out what they actually want, and route it to the right department. Four approaches were evaluated.

Criteria	MobileBERT (TFLite)	DistilBERT (TFLite)	Rasa DIET	Rule-Based Only
Model Size	~26MB	~67MB	~100MB+ server	<1MB
On-Device Inference	Yes	Yes	No (server-side)	Yes
Inference Latency (mobile)	~62ms	~150ms	N/A	<5ms
Classification Accuracy	High (99.06% on Vosk test set)	High	High	Moderate
Handles Natural Phrasing Variation	Yes	Yes	Yes	No
Requires Running Server	No	No	Yes	No
Native TFLite Conversion	Yes	Requires extra steps	Not supported	N/A
Purpose-Built for Mobile	Yes	No	No	N/A
Satisfies NFR-03 (Latency)	✓	Marginal	Dependent on server	✓

Table 5-4: Intent Classification Technology Comparison

Selection: Hybrid MobileBERT TFLite + Rule-Based Pipeline

MobileBERT is the only transformer specifically designed for mobile hardware. Trained via progressive knowledge distillation from BERT-LARGE, it achieves 4.3× compression and 5.5× speedup over BERT-BASE while scoring 77.7 on GLUE (Sun et al., 2020). At 26MB in TFLite format, less than half the size of DistilBERT (~67MB) (Sanh et al., 2019), and ~62ms inference time, it makes real-time classification on a budget tablet practical.

DistilBERT at ~67MB and ~150ms is more than twice the size of MobileBERT and significantly slower, a poor fit for budget tablets (NFR-04). Rasa DIET (Bunk et al., 2020) requires a persistent server process, which conflicts with on-device NLU. A rule-based system alone cannot handle the natural variation in guest phrasing.

A neural-only approach has a known limitation in this pipeline: Vosk does not always produce identical transcriptions for the same spoken phrase, so the same intent can arrive at the classifier with slightly different surface forms. A model trained on clean text may assign lower confidence to a valid request containing a minor transcription variant, creating a misclassification risk (Zhang, L. et al., 2022). To address this, a **hybrid two-tier pipeline** was designed: a fast rule-based keyword matching layer handles high-frequency requests at a fixed confidence of 0.99, with MobileBERT invoked only for indirect or ambiguous cases. If confidence falls below 0.60 after up

to two retries, the request is rejected rather than misrouted. The pipeline's behavior in practice is discussed in Chapter 7.

As a side benefit, the rule-based layer is very fast, common requests like "extra towels" or "room service" are resolved in under 5ms without invoking the neural model.

Production pathway: The keyword dictionary needs manual maintenance as new phrasings are met. In a production system, logging real interactions and periodically retraining the model would reduce the need for manual rule additions over time.

5.6.3 Backend Framework: FastAPI vs Flask vs Django vs Express.js

Criteria	FastAPI	Flask	Django	Express.js
Native Async Support	Yes	No	Partial (v3.1+)	Yes
Native WebSocket Support	Yes (Starlette)	Requires Flask-SocketIO	Requires Django Channels	Requires Socket.IO
Automatic API Documentation	Yes (Swagger/OpenAPI)	No	No	No
Python ML Ecosystem	Yes	Yes	Yes	No
Dependency Footprint	Light	Light	Heavy	Light
Request Validation	Built-in (Pydantic)	Manual	Built-in (Forms)	Manual
Performance (requests/sec)	High	Moderate	Moderate	High

Table 5-5: Backend Framework Comparison

Selection: FastAPI

FastAPI (Ramírez, 2024) was chosen for native async support. The server must hold WebSocket connections open with multiple tablets and the staff dashboard simultaneously while handling HTTP requests, without blocking. Flask's synchronous design would require Flask-SocketIO, adding a dependency and complicating concurrency. Django was overkill, the system needs a focused API server and WebSocket hub, not a full framework with an Object-Relational Mapping (ORM) layer, templating engine, and admin panel.

Express.js was not suitable because the entire ML stack, training, evaluation, and the NLU pipeline, is Python-based. Adding a Node.js backend would mean maintaining two language environments and duplicating business logic.

Production pathway: A single FastAPI process is fine for a controlled research environment. But scaling to multiple hotel properties would need nginx as a load balancer, multiple worker processes, and a Redis-backed message queue to maintain reliable WebSocket state.

5.6.4 Database: SQLite vs PostgreSQL vs MySQL vs MongoDB

Criteria	SQLite	PostgreSQL	MySQL	MongoDB
Server Process Required	No	Yes	Yes	Yes

Configuration Required	None	Moderate	Moderate	Moderate
Concurrent Write Performance	Limited	Excellent	Good	Good
Storage Footprint	Minimal (<1MB per hotel)	~100MB+	~200MB+	~300MB+
Data Model	Relational	Relational	Relational	Document
DBA Expertise Required	None	Yes	Yes	Yes
Backup Method	Copy single file	pg_dump	mysqldump	mongodump

Table 5-6: Database Technology Comparison

Selection: SQLite

A small hotel generating 100 - 200 service requests per day does not approach SQLite's limits (Hipp, 2024), and write concurrency at that volume is negligible. SQLite needs no server process, no installation, and no ongoing admin, ideal for hotels with no dedicated IT staff.

The data is naturally relational: requests belong to rooms, messages belong to requests. MongoDB's document model is a poor fit. PostgreSQL and MySQL are capable but require user accounts, network configuration, and ongoing maintenance, not viable for hotels with no IT staff.

Production pathway: SQLite serializes writes, which is sufficient for small hotel deployments but would become a bottleneck under constant concurrent load. The schema and FastAPI Create, Read, Update, Delete (CRUD) layer are already written to be PostgreSQL-compatible, so switching would only require a driver and connection string change

5.6.5 Real-Time Communication: WebSocket vs HTTP Polling vs Server-Sent Events vs MQTT

Criteria	WebSocket	HTTP Polling	Server-Sent Events	MQTT
Bidirectional	Yes	Simulated	No (server → client only)	Yes
Latency	Low	High (polling interval)	Low	Low
Connection Overhead	Single handshake	Repeated connections	Single handshake	Single handshake
Requires External Broker	No	No	No	Yes (e.g., Mosquitto)
Native Android Support	Yes (OkHttp)	Yes	Limited	Requires library
Battery Impact on Tablets	Low	High	Low	Low

Table 5-7: Real-Time Communication Technology Comparison

Selection: WebSocket (OkHttp on Android / Starlette on server)

WebSocket (Fette and Melnikov, 2011) keeps a single persistent TCP connection open and allows both sides to send messages at any time. The system needs this in three places: guest tablets sending requests and receiving status updates, the staff dashboard receiving requests

and sending messages back, and the server pushing changes to all connected clients. This bidirectional requirement rules out Server-Sent Events, they only go one way (server to client).

HTTP polling includes latency, a guest would not learn their request is "in progress" until the next polling cycle, and repeated connections drain battery faster than a persistent WebSocket. Message Queuing Telemetry Transport (MQTT) (OASIS, 2019) needs a separate broker process like Mosquitto, which is unnecessary infrastructure for a point-to-point LAN scenario.

The server maintains separate WebSocket endpoints for guest devices and the staff dashboard, with connection state held in memory. If the connection drops, the Android client retries automatically, waiting a little longer between each attempt but never exceeding a set maximum delay.

Production pathway: Because connection state is in-memory, it is lost if the server restarts. A Redis Pub/Sub layer would persist WebSocket state and allow the server to restart without dropping all connected clients.

5.6.6 Mobile Platform: Android Native vs iOS vs Flutter vs React Native

Criteria	Android (Kotlin)	iOS (Swift)	Flutter	React Native
Device Cost Range (Sri Lanka)	\$50–\$150	\$300+ (iPad)	Depends on target	Depends on target
Local Hardware Availability	High	Limited	Target-dependent	Target-dependent
Vosk SDK Support	Native	Native	Community plugin	Community plugin
TFLite SDK Support	Native	Core ML (conversion needed)	Community plugin	Community plugin
On-Device TTS	Built-in	Built-in	Plugin required	Plugin required
ML Inference Performance	Java Native Interface (JNI)	Native	Near-native	JS bridge overhead
Jetpack Compose / Material 3	Yes	N/A	Via Flutter widgets	N/A

Table 5-8: Mobile Platform Comparison

Selection: Android Native (Kotlin 2.0.21 with Jetpack Compose)

Android was chosen for three reasons:

First, commodity Android tablets cost \$50–\$150 at local Sri Lankan retailers. Entry-level iPads start at \$300+ more than twice the per-room cost, making iOS unviable (NFR-04). Android hardware is also easier to repair locally.

Second, both Vosk and TensorFlow Lite (Google, 2024a) have first-party native Android SDKs. Flutter and React Native do have community plugins for these libraries, but community plugins for ML inference bring version compatibility risks that could cause hard-to-diagnose failures during evaluation. Keeping to the native SDKs avoids that uncertainty.

Third, Jetpack Compose (Google, 2024b) with Material Design 3 is a good fit for a UI driven by changing state, recording state, processing state, live request list updates, and WebSocket-driven

status changes all map cleanly onto Compose's observable state model. This design alignment carries through to the implementation, where Compose's recomposition behaviour becomes directly relevant, as discussed in Chapter 6.

Production pathway: The prototype uses a fixed room number and a server IP stored manually in SharedPreferences. A real deployment would use a Mobile Device Management (MDM) platform like VMware Workspace ONE to remotely configure room numbers and server credentials across all tablets without touching each device physically.

5.6.7 Text-to-Speech: Android Native TTS vs Piper TTS vs Amazon Polly

Criteria	Android Native TTS	Piper TTS	Amazon Polly
Additional Storage	None (pre-installed)	~50–100MB per voice model	N/A (cloud)
Voice Quality	Acceptable	High (natural)	High (natural)
Custom Voice Support	No	Yes	Yes
Offline Operation	Yes	Yes	No
Configuration Required	None	Model download + setup	API key + internet
Cost	Free	Free	\$4 per 1M characters

Table 5-9: Text-to-Speech Technology Comparison

Selection: Android Native TTS

Android's built-in TTS engine is already on every Android device, no extra storage, no setup, no cost. For this prototype, where the research question is about NLU accuracy and pipeline latency rather than voice quality, the native engine is sufficient for reading back confirmations and status announcements.

Production pathway: Voice quality matters in a real hotel, since it shapes how guests perceive the assistant. Piper TTS (Hansen, 2023) is an open-source neural TTS engine for edge deployment, it produces natural-sounding voices from compact models (~50-100MB) and runs fully offline on low-power hardware. A custom English voice model fine-tuned for hotel-specific phrasing could be deployed directly on the Android tablet, keeping TTS fully on-device. Amazon Polly was not suitable because it requires cloud connectivity (violates NFR-01) and costs \$4 per million characters synthesized (Amazon Web Services, 2024).

5.7 Summary

The three-layer architecture, AI on the guest device, coordination on the server, management on the staff dashboard, follows directly from the offline and privacy constraints. Cloud options were eliminated for violating NFR-01 and NFR-02; heavier frameworks would be too complex to manage without dedicated IT support. For the prototype, the final choices are vosk-model-small-en-in-0.4, MobileBERT TFLite, FastAPI, SQLite, and Android Native, each chosen to fit the specific context of Sri Lankan hotels.

The following chapter covers the actual implementation of this architecture and the key engineering decisions that came up along the way.

6 Design

6.1 Introduction

This chapter translates the technology choices from Chapter 5 into a working architecture. It covers the three-tier system structure, voice processing pipeline, database schema, API design, and both user interfaces. Every design choice was made to satisfy the four viability constraints, offline operation, privacy, latency, and low hardware cost, with reasoning given where the choice needs explanation.

6.2 System Architecture

All AI work, speech recognition and intent classification, happens on the guest's tablet. The server only coordinates, stores requests, and routes them to the right department, so it needs no GPU and can run on any basic hotel hardware.

6.2.1 Three-Tier Architecture

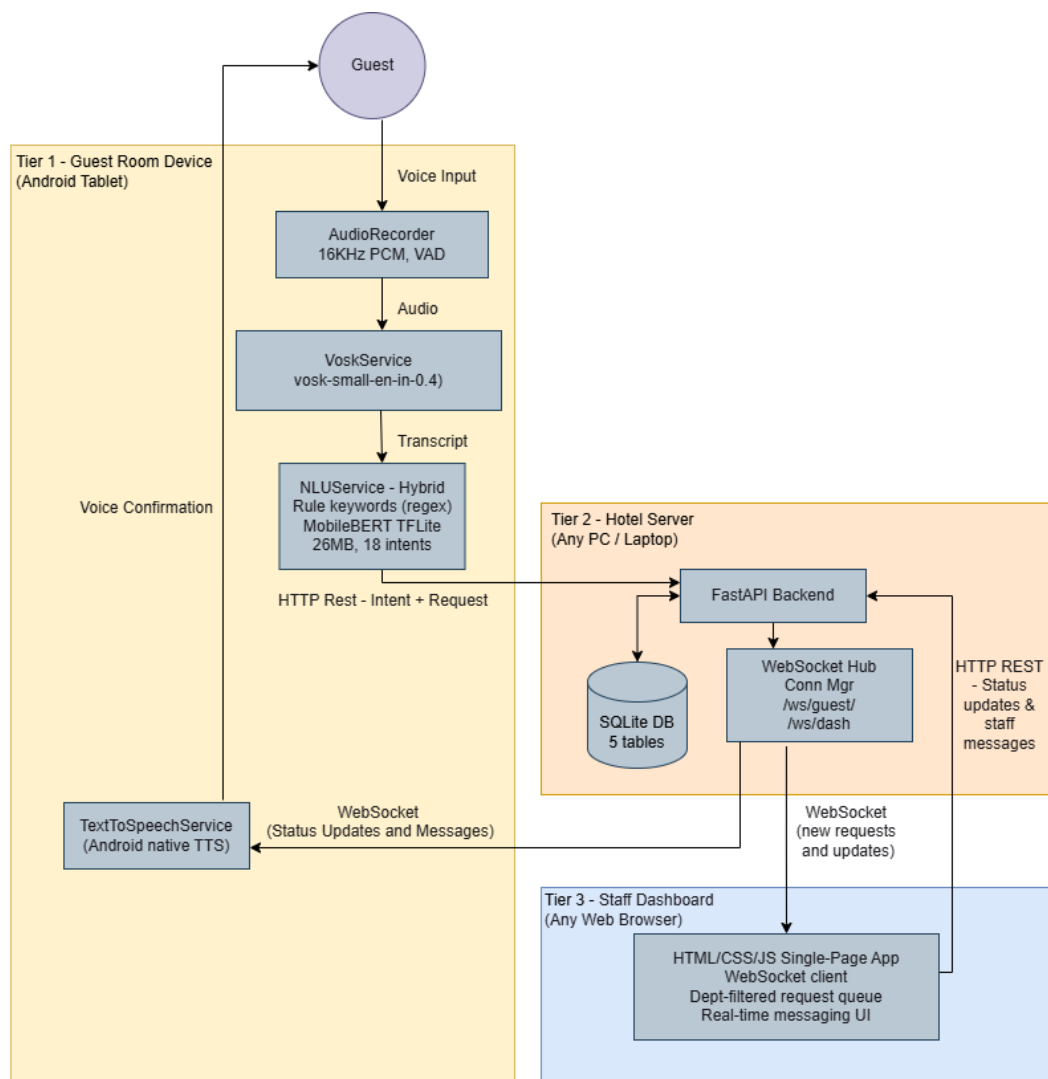


Figure 6-1: System Architecture Diagram

Tier 1 - Guest Device (Android Tablet)

Each guest room has one Android tablet running the full AI pipeline, audio capture, speech recognition via Vosk, NLU classification, and TTS confirmation. It submits requests via HTTP and receives real-time updates via WebSocket.

The key components on the device are:

- **AudioRecorder** - captures 16kHz, 16-bit PCM mono audio in 4,096-byte chunks, with voice activity detection based on Root Mean Square (RMS) energy threshold (0.02)
- **VoskService** - on-device speech-to-text using vosk-model-small-en-in-0.4 (~36MB)
- **NLUService** - hybrid two-tier classification pipeline (rule-based keywords → MobileBERT TFLite)
- **TextToSpeechService** - Android native TTS for voice confirmations and status announcements
- **ApiService** - HTTP Representational State Transfer (REST) client (OkHttp 4.11.0) for submitting requests and retrieving history
- **WebSocketService** - persistent WebSocket connection for real-time updates from the server

Tier 2 - Hotel Server

The server runs on any PC on the hotel's local network. It exposes 17 endpoints, manages the SQLite database, and pushes events to connected clients via WebSocket. Routing uses a three-level hierarchy: intent lookup in intent_department_mapping, keyword fallback if the intent is missing or unrecognised, and Front Desk as the default.

Tier 3 - Staff Dashboard

The staff dashboard is a single-page app served at /dashboard, accessible in any browser without installation. It connects to /ws/dashboard and shows each department's filtered view of the live request queue.

6.3 Voice Processing Pipeline Design

The voice processing pipeline covers the full guest interaction from the moment the microphone button is pressed to the request appearing on the staff dashboard.

6.3.1 Pipeline Overview

The pipeline has three major phases:

Phase 1 - Audio Capture and Transcription

When the guest presses the microphone button, AudioRecorder captures 16kHz PCM audio in 4,096-byte chunks and streams them to Vosk. Voice activity detection monitors RMS energy against a threshold of 0.02, stopping after 1,500ms of silence or the 10-second maximum. The Vosk transcription is then cleaned, filler words, greetings removed and text lowercased.

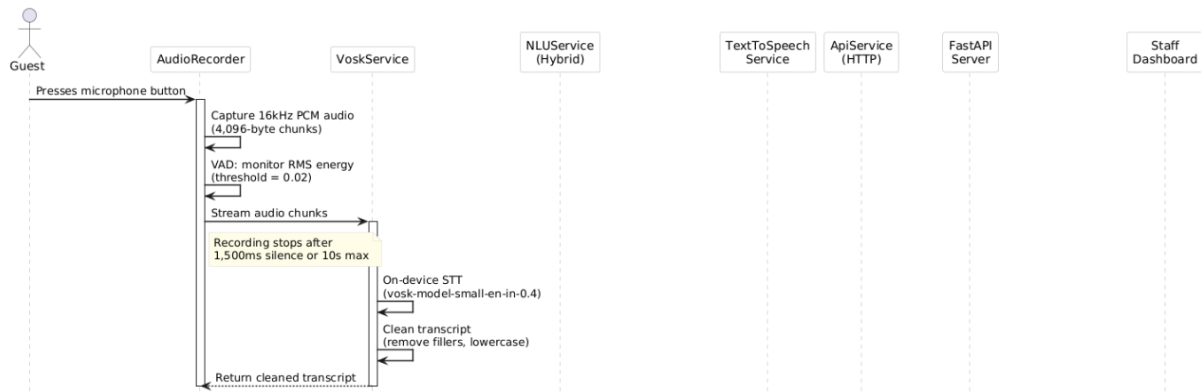


Figure 6-2: Audio Capture and Transcription

Phase 2 - Intent Classification

The cleaned text first goes through a cancel detection check using a regex pattern for "cancel order [number]". If it matches, the request ID is extracted, a voice confirmation plays, and the cancellation is sent to the server. If not, the text moves into the hybrid NLU pipeline.

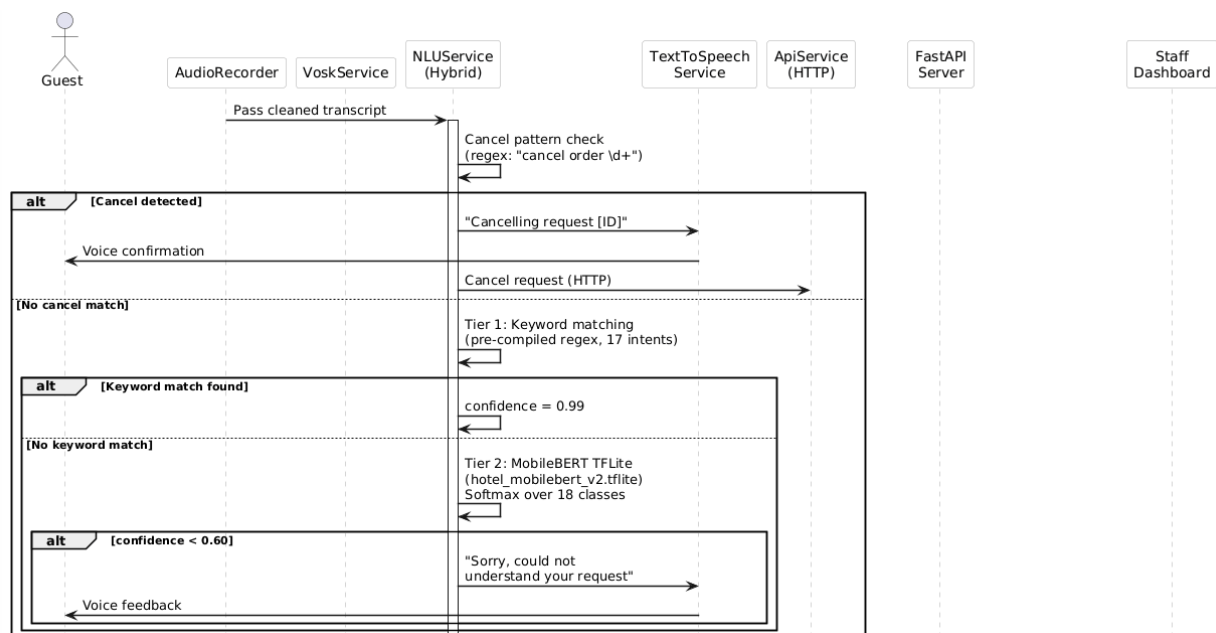


Figure 6-3: Intent Classification

Phase 3 - Confirmation and Submission

Once intent is classified above threshold (0.60 neural; 0.99 keyword), the app reads the request back via TTS: "You'd like [request description]. Shall I submit this?" The guest says yes or no. On confirmation, an HTTP POST goes to the server, which stores, routes, and broadcasts a new_request WebSocket event to all connected dashboards.

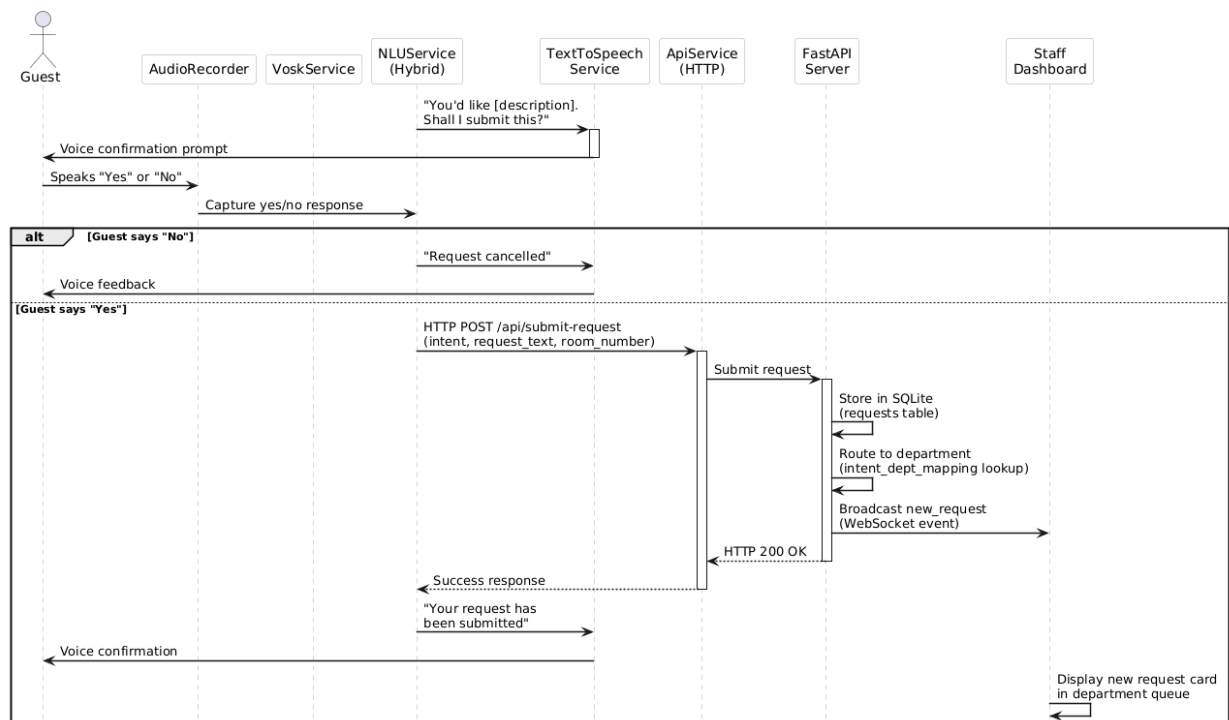


Figure 6-4: Confirmation and Submission

6.3.2 Hybrid NLU Pipeline – Core Design Feature

The two-stage hybrid NLU pipeline was not planned from the start. It emerged from a practical problem during development: purely neural classification was giving unexpectedly low confidence on simple requests. For example "I need towels" was being classified as pillow_request with only 0.72 confidence after small Vosk transcription variations.

Tier 1 - Rule-Based Keyword Matching

The keyword dictionary covers 17 intent categories (all except misc_request). Each intent maps to multi-word contextual phrases, not single words, to avoid false positives. Food_order uses "bottled water" and "room service", not just "water", which could appear in a maintenance request. Patterns are pre-compiled at startup. A keyword match returns confidence 0.99 and skips the neural model entirely.

Tier 2 - MobileBERT Neural Inference

When there is no keyword match, the text is tokenized using the BERT uncased vocabulary (30,522 tokens), padded or truncated to 32 tokens, and passed through hotel_mobilebert.tflite (26MB). Output logits are converted to probabilities via softmax across 18 intent classes and the highest-probability class is taken.

The 0.60 threshold was set through testing, high enough to filter genuinely ambiguous input, low enough to accept natural speech variation

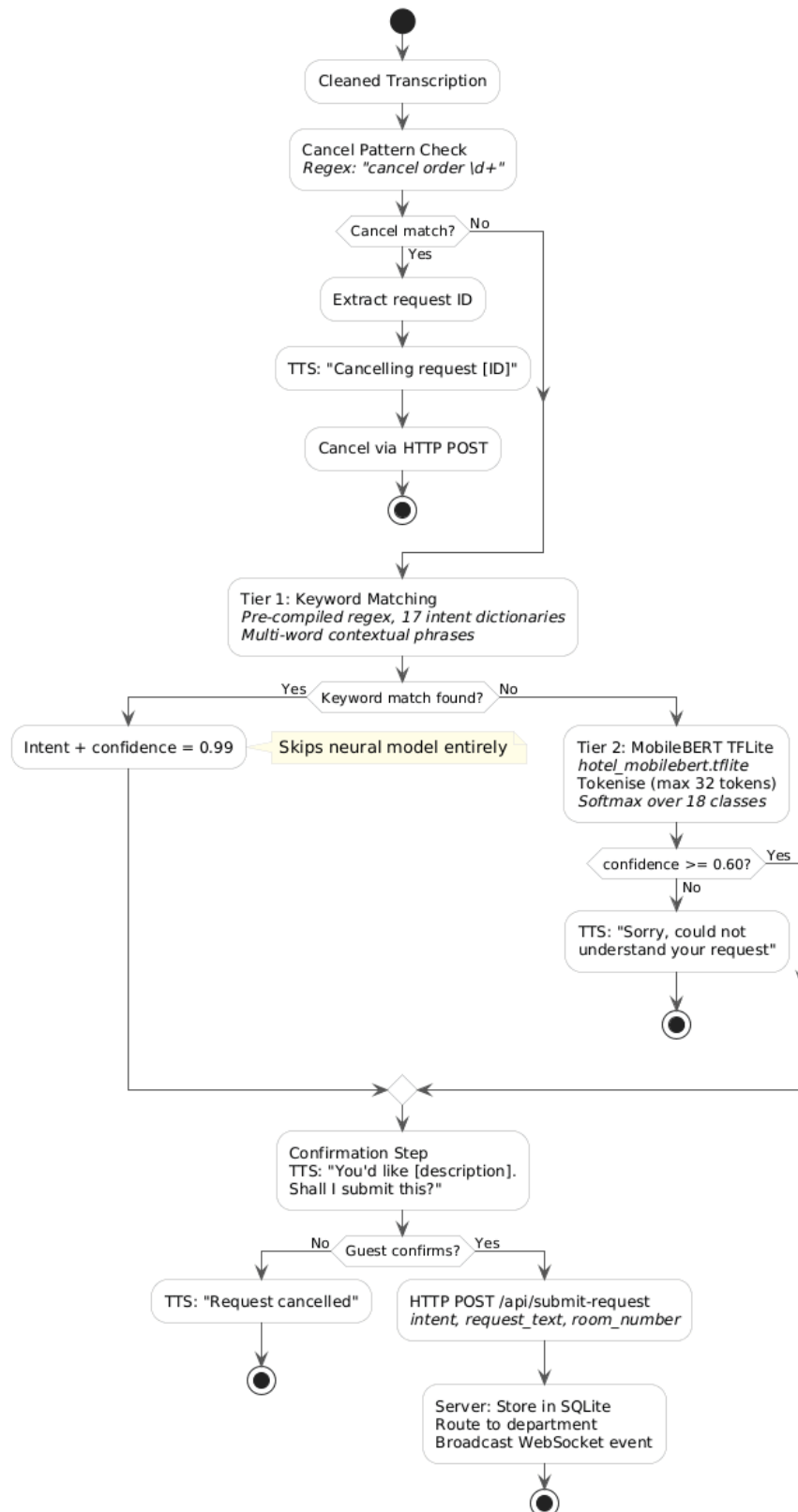


Figure 6-5: Hybrid NLU Pipeline Flow

6.4 Database Design

The database uses SQLite with five tables, designed so that migrating to PostgreSQL for a larger deployment would require no schema changes.

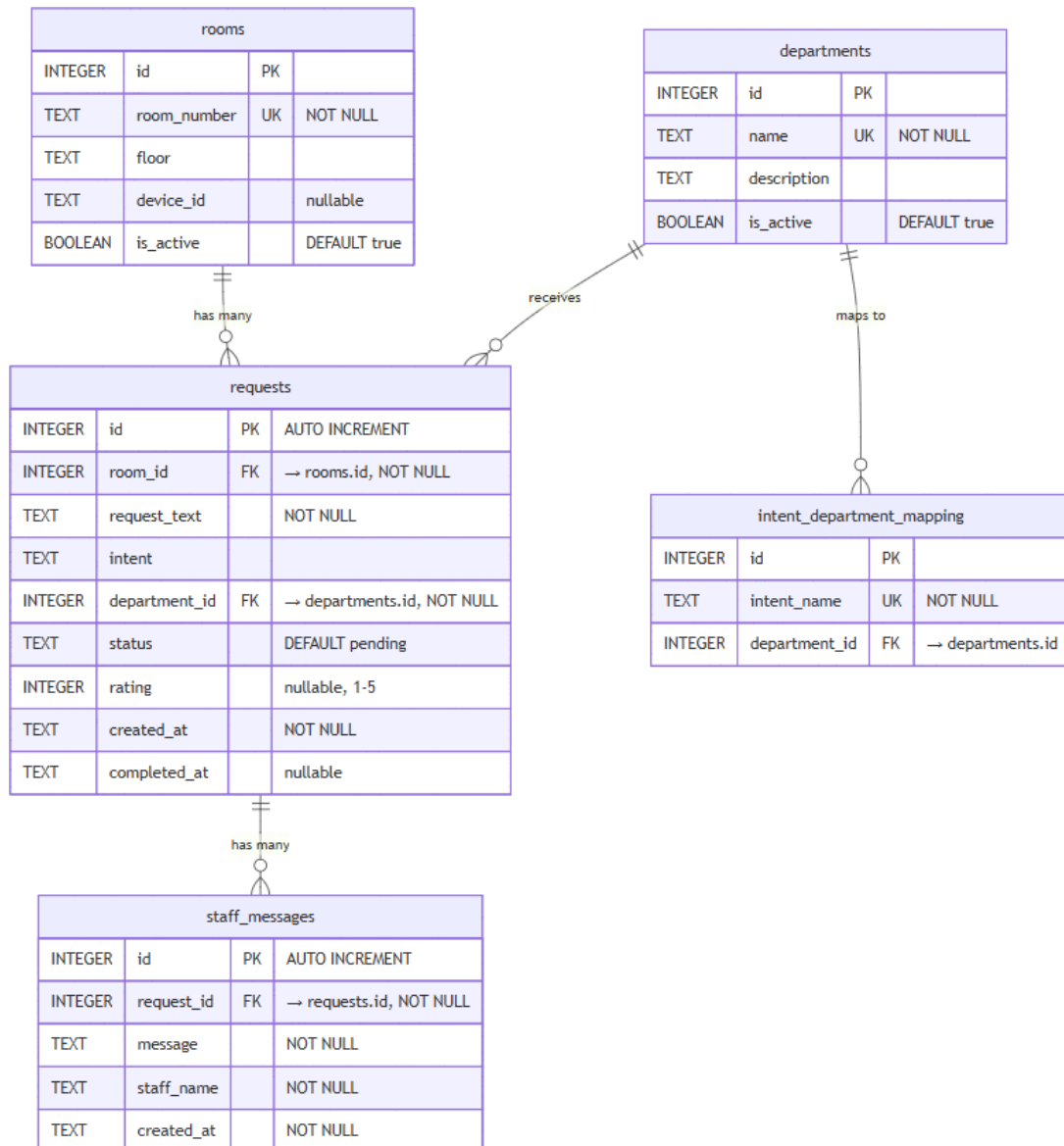


Figure 6-6: Entity-Relationship Diagram

Table descriptions:

rooms - Pre-seeded with 15 rooms (Standard, Deluxe, and Suite types). The room_number field is the identifier used everywhere, request cards, WebSocket channel names, and the history endpoint.

departments - Pre-seeded with five departments: Housekeeping, Room Service, Maintenance, Front Desk, and Concierge. Keeping departments in a table rather than code means a hotel can rename or restructure without touching application logic.

intent_department_mapping - Maps each of the 18 intent categories to a department. This is the first place the server looks when routing a request. Moving routing to a table means the hotel can adjust department assignments by updating a row, without redeploying the server.

requests - The central table. Every service request is stored here from submission to completion. The status field tracks the full lifecycle. Rating is nullable and filled only if the guest rates the service. `completed_at` is only set when status reaches completed.

staff_messages - Stores messages sent from staff to guest rooms, linked to the relevant request

6.5 API and Communication Design

6.5.1 REST API Endpoints

The backend exposes 15 HTTP endpoints following RESTful conventions, plus 2 WebSocket endpoints and 1 endpoint to serve the dashboard HTML. 17 in total.

Method	Endpoint	Purpose	Called By
GET	/	API status, intent count, department list	Monitoring
GET	/api/departments	List all department names	Dashboard
GET	/api/departments/detail	Departments with descriptions	Dashboard
GET	/api/rooms	List all registered room numbers	Dashboard / Admin
GET	/api/intent-mapping	Intent-to-department mapping	Debug / Admin
GET	/api/intent-mapping/detail	Full intent mapping with descriptions	Debug / Admin
GET	/api/requests	All requests in database	Dashboard
GET	/api/request-history	Request history for a specific room (?room_number=)	Guest device
GET	/dashboard	Serve the staff dashboard HTML	Browser
POST	/api/submit-request	Submit a new service request	Guest device
POST	/api/update-status	Update a request's status	Dashboard
POST	/api/update-department	Transfer request to a different department	Dashboard
POST	/api/cancel-request	Cancel a pending or in-progress request	Guest device
POST	/api/send-message	Send a staff message to a guest room	Dashboard
POST	/api/rate-request	Submit a guest rating (1-5) for a completed request	Guest device

Table 6-1: List of REST API Endpoints

All endpoints use JSON. Pydantic models handle input validation and return clear error messages on malformed data.

6.5.2 WebSocket Communication Design

Two WebSocket endpoints handle real-time events. Guest and dashboard channels are kept separate, each client receives only the events relevant to it.

Guest channel: /ws/guest/{room_number}

Each guest device keeps a persistent WebSocket connection identified by its room number. The server pushes three types of events:

Event Type	Fields	Trigger
status_update	request_id, status, message	Staff updates request status
department_update	request_id, message	Request transferred to different department
staff_message	request_id, message, staff_name	Staff sends a message to the room

Table 6-2: WebSocket Communication - Guest Channel

On the Android app, all incoming WebSocket events are read aloud via TTS except department transfer update.

Dashboard channel: /ws/dashboard

The staff dashboard connects to a single shared WebSocket endpoint. Every connected dashboard instance gets the same event stream, when one staff member updates a request, all other open dashboards reflect it instantly.

Event Type	Fields	Trigger
initial	requests[] (full list)	On first connection gives the dashboard its full current state
new_request	id, room_number, request_text, intent, department, status, created_at, rating	Guest submits a new request
status_update	request_id, status	Request status changes
department_update	request_id, department	Request transferred
staff_message	request_id, message, staff_name	Message added
rating_update	request_id, rating	Guest submits a rating

Table 6-3: WebSocket Communication - Dashboard Channel

The initial event is sent as soon as a dashboard client connects, giving it the full current state of all requests without needing a separate HTTP call.

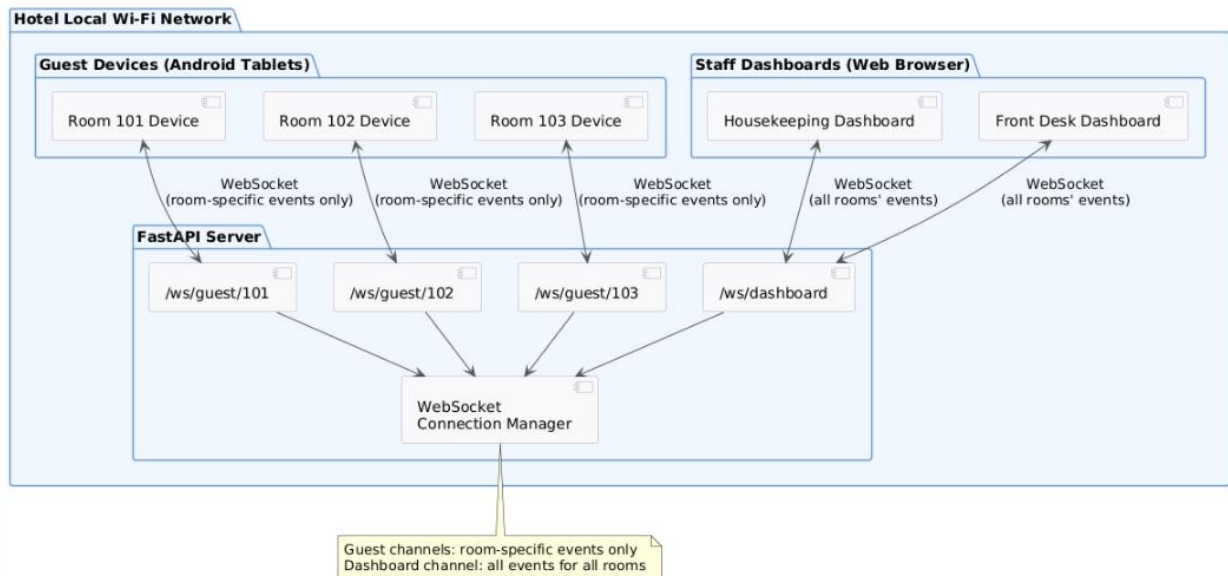


Figure 6-7: WebSocket Communication Topology

6.5.3 Request Lifecycle State Management

A request moves through a defined set of states from submission to completion. Invalid transitions are blocked in the application logic, for example, a completed request cannot be moved back to `in_progress`.

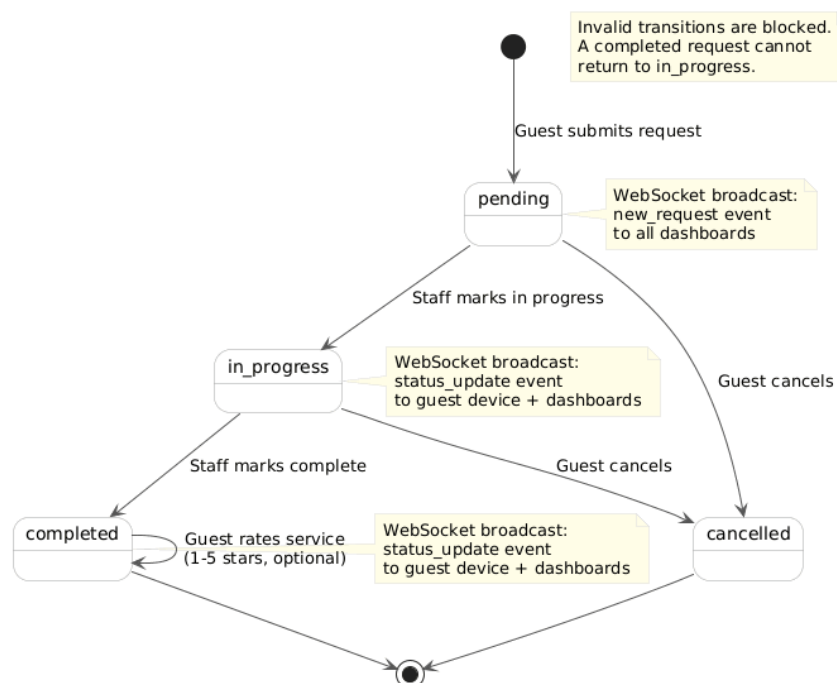


Figure 6-8: Request State Diagram

Every state transition triggers a WebSocket broadcast to the guest device channel and all connected dashboards.

6.6 Department Routing Design

When a request arrives via POST /api/submit-request, the server decides which department should handle it using a three-level hierarchy:

1. **Intent-based routing (primary)** - The request includes an intent field classified on-device. The server looks this up in the intent_department_mapping table and routes accordingly.
2. **Keyword fallback (secondary)** - If the intent is missing or not found in the table, the server analyses the request_text using its own keyword lists per department (separate from the on-device NLU).
3. **Default routing** - If neither method finds a match, the request goes to Front Desk.

Department	Intents	Colour
Housekeeping	room_cleaning, towel_request, toiletries_request, blanket_request, pillow_request, laundry_service, do_not_disturb	#10b981 (green)
Room Service	food_order	#f59e0b (amber)
Maintenance	maintenance, temperature_control, lighting_control	#ef4444 (red)
Front Desk	wake_up_call, checkout_billing, noise_complaint, emergency, misc_request	#3b82f6 (blue)
Concierge	concierge_general, concierge_taxi	#8b5cf6 (purple)

Table 6-4: Department Routing Design

Staff can also manually transfer a request using a dashboard dropdown; it disappears from the original queue and appears instantly in the new one via WebSocket.

6.7 User Interface Design

The guest app needs to work in a dark hotel room, from bed if needed, with no training. The staff dashboard needs to perform during a busy shift with multiple departments using it at the same time.

6.7.1 Guest Application Interface

The app is built with Jetpack Compose and Material Design 3. The entire interface is a single screen divided into three functional sections.

Section 1 - Room Information Bar (top-left) Shows the room number, Wi-Fi connection status, and the configured server network profile. Both guests and maintenance staff can see the key context at a glance without navigating any menus.

Section 2 - Voice Interaction Area (left panel, centre-left) The left panel contains a large circular microphone button. While recording, it pulses with an animation and a 20-bar audio visualiser shows the captured signal level. A status line below it ("Tap to speak" → "Listening..." → "Processing...") gives the guest continuous feedback. After each interaction, the transcription, recognised intent, confidence score, and department colour are displayed.

Section 3 - Request History (right panel, scrollable) A scrollable list of the room's requests, sorted by priority: in-progress at the top, then pending, completed, and cancelled at the bottom.

Each card shows the request ID, a colour-coded status badge, the request text (truncated to two lines), department colour, and a relative timestamp ("2 mins ago"). Pending and in-progress requests have a cancel button; completed ones have a 5-star rating selector.

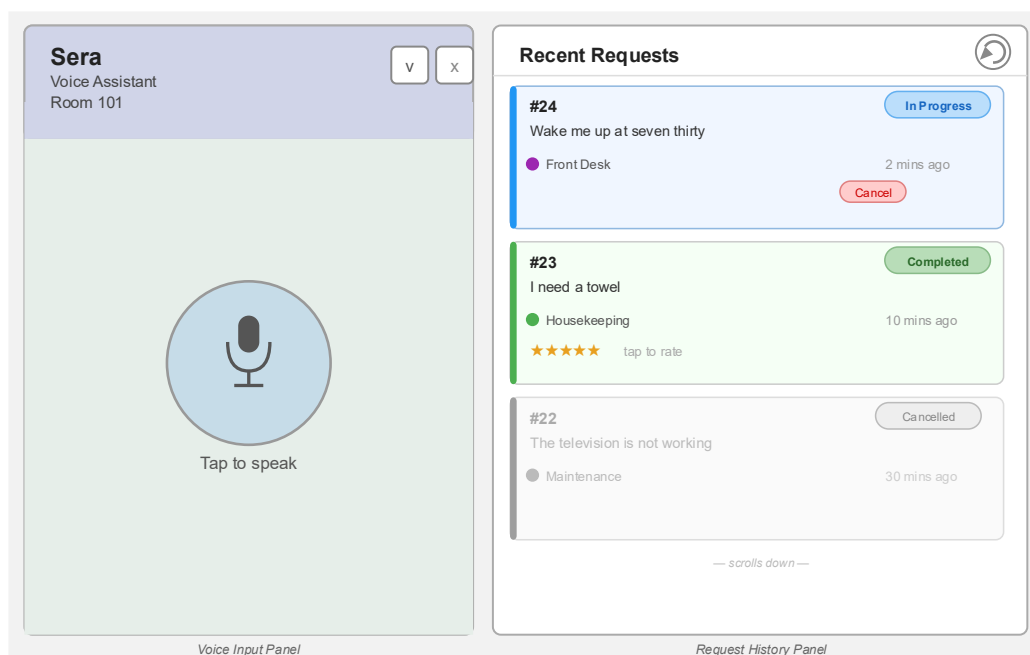


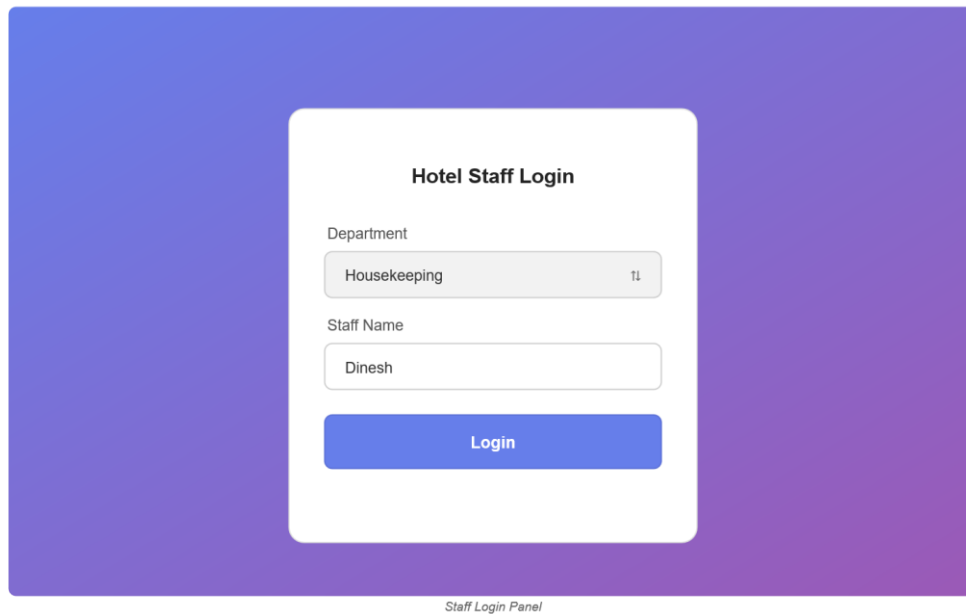
Figure 6-9: Guest Application Interface

Decision	Rationale
Large central microphone button	Dimly lit rooms and in-bed use require a large, easy tap target.
Voice confirmation before submission	Guest survey (Chapter 4) showed strong preference for a review step, especially for food orders where mistakes are most troublesome.
Status-sorted request list	Guests care most about active requests; showing in-progress first reduces scrolling through completed history.
Department colour coding	Consistent colours (green for Housekeeping, amber for Room Service, red for Maintenance, blue for Front Desk, purple for Concierge) allow instant visual recognition without reading labels.
Relative timestamps ("2 mins ago")	More natural than absolute timestamps for guests checking whether a request has been acknowledged.
No keyboard input anywhere	Removing the keyboard lowers the barrier for guests less comfortable with on-screen typing.

Table 6-5: Guest UI Design Decisions

6.7.2 Staff Dashboard Interface

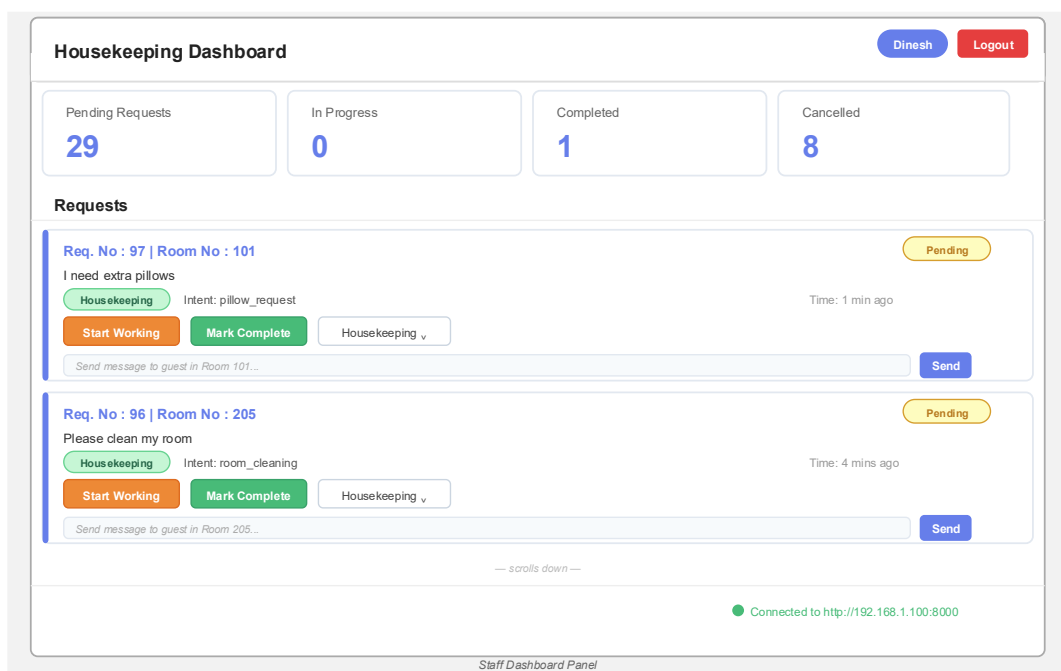
The dashboard is a web-based single-page app served at /dashboard. It works on any modern browser, a desktop at the front desk, a tablet used by housekeeping, or a phone a maintenance engineer carries around.



The login screen features a white card on a purple gradient background. The card is titled 'Hotel Staff Login'. It contains a 'Department' dropdown menu with 'Housekeeping' selected, a 'Staff Name' text input field with 'Dinesh' entered, and a blue 'Login' button at the bottom.

Staff Login Panel

Figure 6-10: Login Screen



The dashboard is titled 'Housekeeping Dashboard' and includes a header with 'Dinesh' and a 'Logout' button. It features four count cards: 'Pending Requests' (29), 'In Progress' (0), 'Completed' (1), and 'Cancelled' (8). Below these are two request cards. The first card shows 'Req. No : 97 | Room No : 101' with the text 'I need extra pillows', a 'Housekeeping' intent label, a 'Pending' status badge, and a timestamp of 'Time: 1 min ago'. It includes 'Start Working' and 'Mark Complete' buttons, a department dropdown, and a message input field with a 'Send' button. The second card shows 'Req. No : 96 | Room No : 205' with the text 'Please clean my room', a 'Housekeeping' intent label, a 'Pending' status badge, and a timestamp of 'Time: 4 mins ago'. It also includes 'Start Working' and 'Mark Complete' buttons, a department dropdown, and a message input field with a 'Send' button. A 'Send message to guest in Room 101...' link is present below the first card. A 'Send message to guest in Room 205...' link is present below the second card. A 'scrolls down' indicator is at the bottom. A status bar at the bottom right indicates 'Connected to http://192.168.1.100:8000'.

Staff Dashboard Panel

Figure 6-11: Dashboard Layout

Login Screen Staff select their department from a dropdown, enter their name, and log in. No accounts or passwords, reasonable for a prototype focused on service routing.

Dashboard Layout The main view shows a header with the department and staff name, a statistics bar with four count cards (Pending, In Progress, Completed, Cancelled), and the request queue filtered to the logged-in department's requests only.

Each request card shows: room number (highlighted), request text, intent label, status badge, timestamp, action buttons (In Progress, Complete), a department transfer dropdown, a message input with a send button, message history, and the guest rating if one has been submitted.

A connection status indicator shows a green "Connected" badge when the WebSocket is live and red "Disconnected" if it drops, useful given the unreliable Wi-Fi connection common in hotel settings

Decision	Rationale
Department-filtered view	Staff only see their department's requests, reducing noise during busy shifts.
Web-based, no installation	No software needed on staff devices; any browser works, including personal phones (NFR-07).
Real-time updates without page refresh	New requests appear instantly via WebSocket; no manual refresh needed, reducing missed requests.
Inline messaging	Staff communicate with guests directly from the request card, keeping all context in one place.
Room number prominently displayed	Room number is the primary identifier; visually emphasised on every card.
Connection status indicator	A visible indicator lets staff know immediately if they have stopped receiving live updates.

Table 6-6: Staff Dashboard Design Decisions

6.8 Summary

The three-tier structure keeps all AI processing on the guest tablet and server coordination minimal, which is why the system can run on low-cost hardware with no GPU.

The hybrid NLU pipeline was not planned from the start. It emerged after purely neural classification returned low confidence on simple requests. Combining keyword matching for clear-cut intents with MobileBERT for the rest, outperformed either approach on its own.

Routing logic lives in the `intent_department_mapping` table rather than in code, so hotels can adjust department assignments without redeploying. Both interfaces were shaped by real deployment scenarios, a dimly lit hotel room and a busy service shift, rather than generic guidelines. Chapter 7 covers the implementation.

7 Implementation

7.1 Introduction

This chapter explains how the system designed in Chapter 6 was actually built. It walks through the development environment, the four iterative build cycles, and the specific implementation decisions for each component, the Android guest app, the NLU training pipeline, the backend server, and the staff dashboard. Deviations from the original plan, those problems and how they were fixed are also described. They often shaped the final design more than the features that worked first time.

7.2 Development Environment

The system spans three technology stacks, one for each layer of the architecture. Table 7-1 gives a full breakdown of everything used during development.

Component	Technology	Version
Guest Application		
Language	Kotlin	2.0.21
UI Framework	Jetpack Compose	Latest stable (BOM 2024.09.00)
Integrated Development Environment (IDE)	Android Studio	Hedgehog / Iguana
Min SDK	Android API 26 (Android 8.0)	
Compile/Target SDK	Android API 34 (Android 14)	
Build System	Gradle (Kotlin DSL)	
TFLite Runtime	TensorFlow Lite	2.17.0
WebSocket Client	OkHttp	4.11.0
STT Model	vosk-model-small-en-in-0.4	~36 MB
NLU Model	hotel_mobilebert.tflite	26 MB
Audio Format	16 kHz, 16-bit PCM, mono	-
Audio Chunk Size	4,096 bytes	-
TTS Engine	Android native TextToSpeech	Pre-installed
NLU Training Pipeline		
Language	Python	3.10+
ML Framework	PyTorch + HuggingFace Transformers	4.57.3
Model Conversion	TensorFlow / tf-keras	2.13+
Data Processing	pandas, scikit-learn	
Dataset Generation	Claude Haiku API (anthropic)	
TTS for Vosk Pairing	gTTS (Google Text-to-Speech)	2.3+
Audio Conversion	ffmpeg / pydub	
STT for Pairing	Vosk Python SDK	0.3.45+
Backend Server		
Language	Python	3.10+
Web Framework	FastAPI	0.104.1
Asynchronous Server Gateway Interface (ASGI) Server	Uvicorn	0.24.0
Database	SQLite 3	
WebSocket	Starlette (via FastAPI)	

Request Validation	Pydantic	2.5.0
Staff Dashboard		
Languages	HTML5, CSS3, Vanilla JavaScript	
WebSocket Client	Native browser WebSocket API	

Table 7-1: Summary of the complete development environment

There was a one deliberate choice to keep dependencies minimal. The dashboard uses plain JavaScript, no frontend framework. The backend has no ORM, all queries are plain SQL. The Android app only adds Vosk and TFLite on top of standard Android libraries. A minimal stack makes the prototype easier to understand, evaluate, and reproduce.

7.3 Iterative Implementation Process

As described in Chapter 3, the system was built across four iterations. Each cycle produced a working prototype that was tested before moving on to the next.

7.3.1 Iteration 1: On-Device Speech Recognition

What was built:

- Android application skeleton with microphone permission handling
- Vosk model loaded from the app's assets directory
- Audio capture using Android's AudioRecord API at 16kHz, 16-bit mono PCM
- Energy-based voice activity detection (VAD)
- Real-time transcription display on screen

Issues encountered:

The first model tried was the larger Vosk English model (1.8GB), which took over 15 seconds to start and crashed with out-of-memory errors on a budget tablet. Switching to `vosk-model-small-en-in-0.4` (~36MB) resolved this, 3 seconds to load, good accuracy, and a better acoustic match for Sri Lankan English than US English models.

Getting the VAD right took some trial. The initial silence timeout was too short and cut off recordings at middle of the sentence. A 1,500ms timeout and 10,000ms maximum worked well, long enough for natural speech, short enough to not hang in a noisy room.

7.3.2 Iteration 2: Intent Classification Pipeline

What was built:

- Three MobileBERT model variants (A, B, C) trained using the script `step3_train_three_models.py`
- TFLite conversion pipeline (`step5_convert_best_model.py`)
- TFLite model integrated into the Android app via `NLUService.kt`
- Custom BERT word-piece tokenizer written in Kotlin
- Rule-based keyword dictionary with pre-compiled regex patterns
- Hybrid classification logic with a 0.60 confidence threshold
- Label map and vocabulary loaded from app assets

Issues encountered:

The standard HuggingFace tokenizer is Python-only and cannot run on Android, so a custom Kotlin tokenizer was built to replicate the same logic: loads the BERT vocabulary (30,522 tokens) from vocab.json, splits words into subword pieces, adds the required special tokens ([CLS], [SEP], [UNK]), pads to 32 tokens, and truncates anything longer.

During early testing, the neural model gave low confidence on simple requests when Vosk introduced small transcription errors, "I need towels" could come through as "i need tawels". This led directly to the keyword matching layer, catching clear requests before the neural model.

Noise-aware model training was the main goal. In a real pipeline, the model receives Vosk-transcribed input, not clean text, so three training strategies were compared using identical hyperparameters but different data:

- **Model A** - trained on clean text only (new_hotel_dataset.csv)
- **Model B** - trained on Vosk-transcribed text only (vosk_only_dataset.csv)
- **Model C** - trained on a mix of clean and Vosk-paired data (paired_dataset.csv)

Model C came out on top on the Vosk test set and was deployed as the model for the prototype.

7.3.3 Iteration 3: Backend and Real-Time Communication**What was built:**

- FastAPI application with all REST endpoints
- SQLite database with a five-table schema
- WebSocket connection manager for guest and dashboard channels
- Department routing engine (database-driven with a keyword fallback)
- Staff dashboard as a single self-contained HTML file
- Full request flow from the Android app through the server to the dashboard

Issues encountered:

WebSocket connections silently dropped when the screen turned off or Wi-Fi briefly dropped. A reconnection mechanism was added in WebSocketService.kt that waits longer between each retry attempt if the connection keeps failing:

```
private fun scheduleReconnect() {
    val delay = Math.min(2000L * (1 shl reconnectAttempts), MAX_RECONNECT_DELAY) // 30s cap
    reconnectHandler.postDelayed({
        reconnectAttempts++
        internalConnect()
    }, delay)
}
```

Retry delays go: 2s → 4s → 8s → 16s → 30s (capped), recovering quickly from brief drops without flooding the server during longer outages.

The initial dashboard showed all departments at once, which quickly became visually cluttered. Department filtering was added so staff only see their own queue after logging in.

7.3.4 Iteration 4: Integration and Refinement

What was built:

- Voice confirmation flow (TTS readback → listen for yes/no → submit or cancel)
- Request cancellation by voice ("cancel order [number]")
- Guest rating system (1-5 stars)
- Staff-to-guest messaging with TTS announcement on the guest device
- Request department transfer from the dashboard
- Request history sorted by status priority
- Transcription cleaning (stripping greetings like "Hi Sera")

Issues encountered:

The voice confirmation flow created a feedback loop, TTS output was picked up by the microphone. This was fixed by adding a delay between TTS finishing and the microphone activating, and by keeping the confirmation window shorter than a standard request.

Parsing spoken request IDs needed to handle both digit strings and spoken numbers. A regex extracts the numeric ID from cancellation phrases, so both "cancel order 146" and "cancel order one four six" work.

7.4 Component Implementation Details

7.4.1 Android Guest Application

The guest app is organised around a MainActivity that sets up and coordinates six core classes:

Class	File	Responsibility
AudioRecorder	AudioRecorder.kt	Captures 16kHz, 16-bit PCM mono audio in 4,096-byte chunks; energy-based VAD
VoskService	VoskService.kt	On-device speech-to-text using vosk-model-small-en-in-0.4
NLUService	NLUService.kt	Hybrid intent classification (keyword matching → MobileBERT TFLite)
ApiService	ApiService.kt	HTTP REST client (OkHttp 4.11.0) for submitting and retrieving requests
WebSocketService	WebSocketService.kt	Persistent WebSocket connection with retry reconnection mechanism
TextToSpeechService	TextToSpeechService.kt	Android native TTS (speech rate 0.9x, pitch 1.0, Locale.US)

Table 7-2: Android Application Components

The UI is built entirely in Jetpack Compose in VoiceAssistantScreen, updating reactively as request states change. The microphone button pulses while recording, and a 20-bar audio visualizer shows captured signal strength.

The on-device ML files are bundled in the Android Package Kit (APK) under assets/models/:

Asset	Size	Purpose
models/vosk-model-small-en-in-0.4/	~36MB	Offline speech recognition model (Indian English)

models/nlu/hotel_mobilebert.tflite	26MB	Intent classification model (Model C - noise-aware)
models/nlu/vocab.json	~623KB	BERT word-piece vocabulary (30,522 tokens)
models/nlu/label_map.json	<1KB	Intent index-to-name mapping (18 intents)

Table 7-3: On-device ML assets

The TFLite model takes three inputs (`input_ids`, `attention_mask`, `token_type_ids`) each of shape `[1, 32]` (int32) and outputs logits of shape `[1, 18]` (float32). `token_type_ids` is all zeros (single-sentence classification) but must be present.

7.4.2 NLU Training Pipeline

The training pipeline is six sequential Python scripts in `nlu-model/research/`:

Script	Purpose	Output
<code>step1_create_dataset.py</code>	Generates 10,080 clean hotel phrases using Claude Haiku API	<code>new_hotel_dataset.csv</code>
<code>step2_generate_vosk_noise.py</code>	Runs each phrase through gTTS → ffmpeg → Vosk to produce transcribed pairs	<code>vosk_transcriptions.csv</code> , <code>vosk_only_dataset.csv</code> , <code>paired_dataset.csv</code>
<code>step3_train_three_models.py</code>	Fine-tunes three MobileBERT variants (A, B, C) with identical hyperparameters	Model A, B, C checkpoints
<code>step4_evaluate.py</code>	Evaluates all three models on clean and Vosk test sets	<code>evaluation_results.json</code> , confusion matrix PNGs
<code>step5_convert_best_model.py</code>	Converts Model C (best checkpoint) to TFLite	<code>hotel_mobilebert_v2.tflite</code> (copied to Android assets as <code>hotel_mobilebert.tflite</code>)
<code>step6_wer_analysis.py</code>	Computes WER and CER statistics on Vosk transcriptions	Per-intent and overall error rate analysis

Table 7-4: NLU Pipeline Scripts

The dataset follows standard intent classification conventions, balanced examples per class and clear intent boundaries (Larson et al., 2022). Fine-tuning BERT-family models on domain-specific data this way produces strong results even with modest training sizes (Y. Zhang et al., 2022).

Training configuration (identical across all three models):

All three models used identical hyperparameters (detailed in Chapter 3, Table 3-3). The only difference is training data:

- **Train/ val split:** 85%/15% stratified split of each model's training dataset
- **Test set:** 20% set aside from `vosk_transcriptions.csv` (2,016 samples), shared across all three models for fair comparison

Model	Training Data	Records	Test Accuracy (Vosk)	F1 Macro (Vosk)
Model A	<code>new_hotel_dataset.csv</code> (clean text)	10,080	89.34%	0.8908
Model B	<code>vosk_only_dataset.csv</code> (Vosk-transcribed)	10,080	96.38%	0.9636
Model C	<code>paired_dataset.csv</code> (clean + Vosk mixed)	14,863	99.06%	0.9905

Table 7-5: Training Dataset Summary

Model C was selected for deployment. It closes the 8.73-point gap between Model A on clean input (98.07%) and Vosk output (89.34%), reaching 99.06% on the Vosk test set.

Model conversion (`step5_convert_best_model.py`)

The best Model C checkpoint goes through three stages to become a TFLite file the Android app can use:

1. **PyTorch → TensorFlow** - PyTorch weights are loaded into a TensorFlow MobileBERT using `from_pt=True` in `TFMobileBertForSequenceClassification`, avoiding retraining from scratch.
2. **TensorFlow → SavedModel** - The model is wrapped in a `tf.Module` with a concrete serving function fixing input shapes to `[1, 32]`, needed for the converter to trace the graph.
3. **SavedModel → TFLite** -
The TFLite converter applies `tf.lite.Optimize.DEFAULT` (dynamic range quantisation, weights to INT8, activations stay float32) with `SELECT_TF_OPS` for BERT operations standard builtins cannot handle.

This reduces the model from ~94MB (PyTorch) to 26MB (TFLite) 72% smaller. Full INT8 quantization was skipped because BERT models are sensitive to it and can lose accuracy.

The final file `hotel_mobilebert_v2.tflite` is copied into the Android assets directory as `hotel_mobilebert.tflite`.

7.4.3 Backend Server

The backend is kept deliberately simple a single FastAPI application in `backend/app/` with just three files:

File	Purpose
main.py	FastAPI application, all 17 route handlers, WebSocket endpoints, department routing engine
database.py	SQLite schema definition, table creation, and all query functions
models.py	Pydantic models for request/response validation

Table 7-6: Backend File Structure

To start the server:

```
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

Binding to 0.0.0.0 rather than localhost makes the server reachable from any device on the hotel's Wi-Fi.

The WebSocket connection manager keeps two registries, a room-number-to-connection dictionary for guest devices, and a list of active dashboard connections. When a request comes in via POST /api/submit-request, the server:

1. Saves the request to SQLite
2. Looks up the department from the intent_department_mapping table
3. Falls back to keyword analysis if the intent is not in the mapping
4. Falls back to Front Desk if neither method gives a match
5. Broadcasts a new_request WebSocket event to all connected dashboards
6. Returns the assigned department and request ID to the guest device

Status updates push a status_update event to the room's WebSocket channel, which the guest device announces via TTS.

7.4.4 Staff Dashboard

The dashboard is a single self-contained dashboard.html file served at /dashboard, no build tools or bundlers needed, deployable on any machine with a browser.

The dashboard connects to /ws/dashboard via the native browser WebSocket API. On connection, the server sends an initial event with all existing requests. Subsequent events (new_request, status_update, department_update, staff_message, rating_update) trigger incremental DOM updates without page refresh.

Color coding matches the guest app (Chapter 6, Table 6-4):

Department	Colour	Hex Code
Housekeeping	Emerald green	#10b981
Room Service	Amber	#f59e0b
Maintenance	Red	#ef4444
Front Desk	Blue	#3b82f6
Concierge	Purple	#8b5cf6

Table 7-7: Department Colour Coding

7.5 Deployment

The system runs entirely within the hotel's existing local Wi-Fi network.

Component	Target	Requirements
Backend server	Any PC/laptop on hotel Wi-Fi	Python 3.10+, FastAPI, Uvicorn
Staff dashboard	Browser on any device	Any modern web browser, no installation
Guest app	Android tablet per room	Android 8.0+, tablet with microphone

Table 7-8: Deployment Components

Deployment steps:

1. Install Python 3.10+ and dependencies (pip install -r requirements.txt) on the hotel server machine
2. Start the FastAPI server: `uvicorn app.main:app --host 0.0.0.0 --port 8000`
3. Install the Android APK on each guest room tablet
4. Open network profile settings on each tablet and enter the server's local IP address
5. Place tablets in guest rooms

The network profile feature stores multiple server addresses in SharedPreferences and allows switching without recompiling the app.

Note: Prototype deployment only. The setup above reflects development and evaluation conditions: a laptop, SQLite, manual IP entry, HTTP without TLS. Production requirements, PostgreSQL, MDM-managed tablets, encrypted connections, are covered in Chapter 10 (Section 10.3) and Chapter 11 (Section 11.3.1).

7.6 Current Status

At the point of submission, the prototype is fully functional within its defined scope. Table 7-9 maps each requirement from Chapter 4 to its current implementation status.

Requirement	Status	Notes
FR-01: Voice service requests	Implemented	Full voice input with VAD
FR-02: On-device STT without internet	Implemented	vosk-model-small-en-in-0.4
FR-03: Intent classification (18 categories)	Implemented	Hybrid pipeline (keyword + MobileBERT)
FR-04: Text confirmation before submission	Implemented	TTS readback + voice yes/no response
FR-05: Voice feedback on submission	Implemented	Android native TTS
FR-06: Real-time delivery to staff	Implemented	WebSocket push notification
FR-07: Staff status updates	Implemented	pending → in_progress → completed
FR-08: Bidirectional messaging	Implemented	Staff message → guest TTS announcement
FR-09: Automatic department routing	Implemented	18 intents → 5 departments via DB mapping
FR-10: Request history	Implemented	Stored in SQLite, viewable on guest device
NFR-01: Fully offline operation	Implemented	All AI processing on-device

NFR-02: On-device voice processing	Implemented	No voice data transmitted externally
NFR-03: Response latency < 5 seconds	To be evaluated	Measured in Chapter 8
NFR-04: Commodity Android tablets	Implemented	Tested on budget Android device
NFR-05: $\geq 90\%$ intent classification accuracy	Achieved	99.06% on Vosk test set (Model C)
NFR-06: Multi-room concurrent operation	Implemented	WebSocket supports multiple simultaneous connections
NFR-07: Browser-accessible dashboard	Implemented	Single HTML file, any modern browser

Table 7-9: Requirements Implementation Status

Known limitations of the current prototype:

- **Room number configuration is manual.** Set through app settings; a production system would use MDM.
- **No staff authentication.** The dashboard login only records a department and name, no password or role-based access control.
- **Single-room load testing only.** Multi-room concurrent testing has not been done.
- **Hardcoded server address.** Stored manually in SharedPreferences; a production deployment would manage this centrally.
- **No structured logging.** Print statements worked during development but structured logging with severity levels and timestamps would be needed before real deployment (Chapter 10, Section 10.5.2)

7.7 Summary

The system was built across four cycles, each producing a testable prototype. The key decisions came from what those iterations uncovered: switching to the 36MB Indian English Vosk model, adding the hybrid NLU pipeline after the failures of neural-only approach, and running the three-model comparison to confirm noise-aware training.

The implementation uses Kotlin for the Android client, Python with HuggingFace Transformers and FastAPI for the training pipeline and server, and plain HTML/CSS/JavaScript for the staff dashboard. Dependencies were kept minimal throughout. All functional requirements are met and the system works end-to-end. Chapter 8 evaluates whether it achieves sufficient technical accuracy and performance.

8 Testing

8.1 Introduction

This chapter covers how the system and the NLU models were tested. The evaluation is done for three dimensions of system viability: NLU accuracy under real pipeline conditions, speech recognition quality, and backend reliability and performance. Together, these provide evidence for whether the prototype achieves sufficient technical accuracy and performance to be a viable alternative to traditional room service communication.

8.2 Testing Approach

The testing strategy for this project was shaped by what actually needed to be verified, not by a predefined framework. The system has three distinct concerns that each require a different approach:

1. **NLU model accuracy** - does noise-aware training produce sufficient accuracy for reliable intent routing under real pipeline conditions? This required the most rigorous testing.
2. **Speech recognition accuracy** - how much does Vosk distort guest phrases, and which types of requests are most affected? This determines whether the STT component is fit for purpose in the hospitality domain.
3. **System functionality** - does the backend API behave correctly, does end-to-end latency fall within practical bounds, and does the full pipeline from voice input to staff notification actually work?

All three are necessary to answer the viability question. System and API testing was reasonable, the results feed into the NFR compliance assessment, which summarizes whether the system meets its targets.

Automated unit tests were not written for the Android app. The app was built iteratively and tested live on a physical device. The hybrid NLU pipeline, WebSocket reconnection, and confirmation flow were verified hands-on. This is acknowledged in Section 8.9.

8.3 NLU Model Testing

8.3.1 Test Design

One of the key viability dimensions is whether the NLU component maintains accuracy inside the real STT pipeline. A controlled comparison was done across three model variants on the same held-out test set: clean-only, Vosk-only, and mixed. Model A trained on 10,080 clean phrases, Model B on 10,080 Vosk-transcribed, Model C on 14,863 mixed phrases.

The test set is a 20% stratified hold-out from `vosk_transcriptions.csv` which means 2,016 samples across all 18 categories, with exactly 112 samples per category, with both `clean_text` and `vosk_text` columns, shared across all three models.

The test set is separate from all training sets.

Four evaluation runs were performed:

Evaluation	Model	Purpose
Run 1	Model A	Establishes the best accuracy the model can achieve when there is no STT noise; clean text in, clean text tested
Run 2	Model A	Same model, now tested on Vosk-transcribed input; shows how much accuracy drops when real speech recognition errors enter the pipeline
Run 3	Model B	Model trained only on Vosk-transcribed text; shows whether matching training data to noisy input recovers accuracy
Run 4	Model C	Model trained on both clean and Vosk-transcribed text; tests whether mixing both data types produces a more robust model than either alone

Table 8-1: Four Evaluation Runs

All four runs used `step4_evaluate.py`, which computes accuracy, macro F1, weighted F1, per-intent precision/recall/F1, and generates confusion matrix heatmaps.

8.3.2 Results

Model	Training Data	Test Input	Test Accuracy	F1 Macro	F1 Weighted
Model A	Clean text only	Clean text	98.07%	0.9805	0.9805
Model A	Clean text only	Vosk-transcribed	89.34%	0.8908	0.8908
Model B	Vosk-transcribed only	Vosk-transcribed	96.38%	0.9636	0.9636
Model C	Mixed (clean + Vosk)	Vosk-transcribed	99.06%	0.9905	0.9905

Table 8-2: NLU Model Evaluation Results (2,016-sample test set)

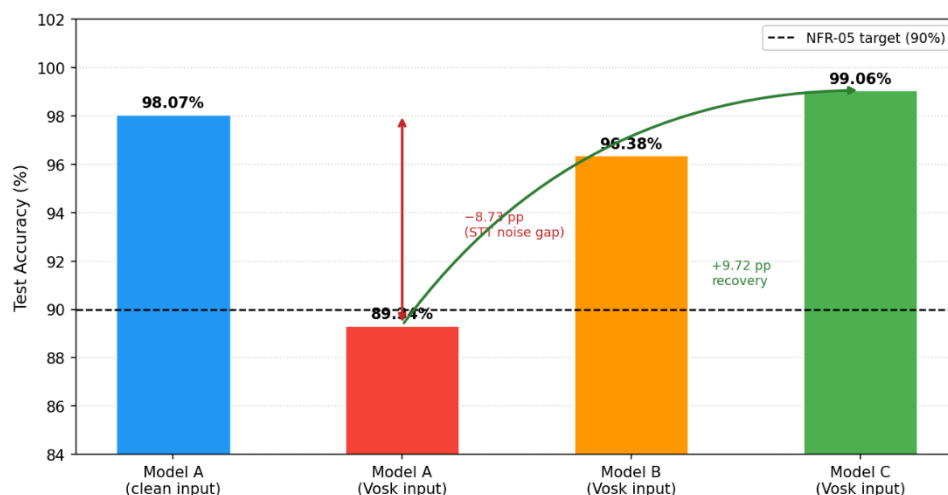


Figure 8-1: NLU Accuracy - Gap Introduced by STT Noise and Recovery via Mixed Training

The results confirm that noise-aware training is necessary for pipeline viability. Model A scores 98.07% on clean text but drops to 89.34% when given Vosk output. An 8.73 percentage point fall was caused entirely by the STT step. Without addressing this, the system would fall short of the NFR-05 accuracy target ($\geq 90\%$) and be unreliable in practice.

Model C scores 99.06% on the same Vosk input. To measure how much of the lost accuracy was recovered, the recovery rate is calculated as follows:

Model C gained 9.72 points above the lowest point ($99.06 - 89.34 = 9.72$), against an original gap of 8.73 points.

$$\text{Recovery rate} = 9.72 \div 8.73 \times 100 = 111.3\%$$

A recovery above 100% means Model C on noisy input outperformed Model A on clean input; the loss was not just recovered but exceeded. This confirms that noise-aware training does not simply compensate for STT errors; it produces a more robust model overall and satisfies NFR-05 by exceeding the 90% accuracy target by a wide margin.

Confusion matrices provide more detail.

Figure 8.2 shows misclassifications in Model A (Vosk input), particularly between similar-sounding intents (towel_request / toiletries_request, temperature_control / do_not_disturb). Figure 8.3 shows nearly all of those errors disappear in Model C after mixed training.

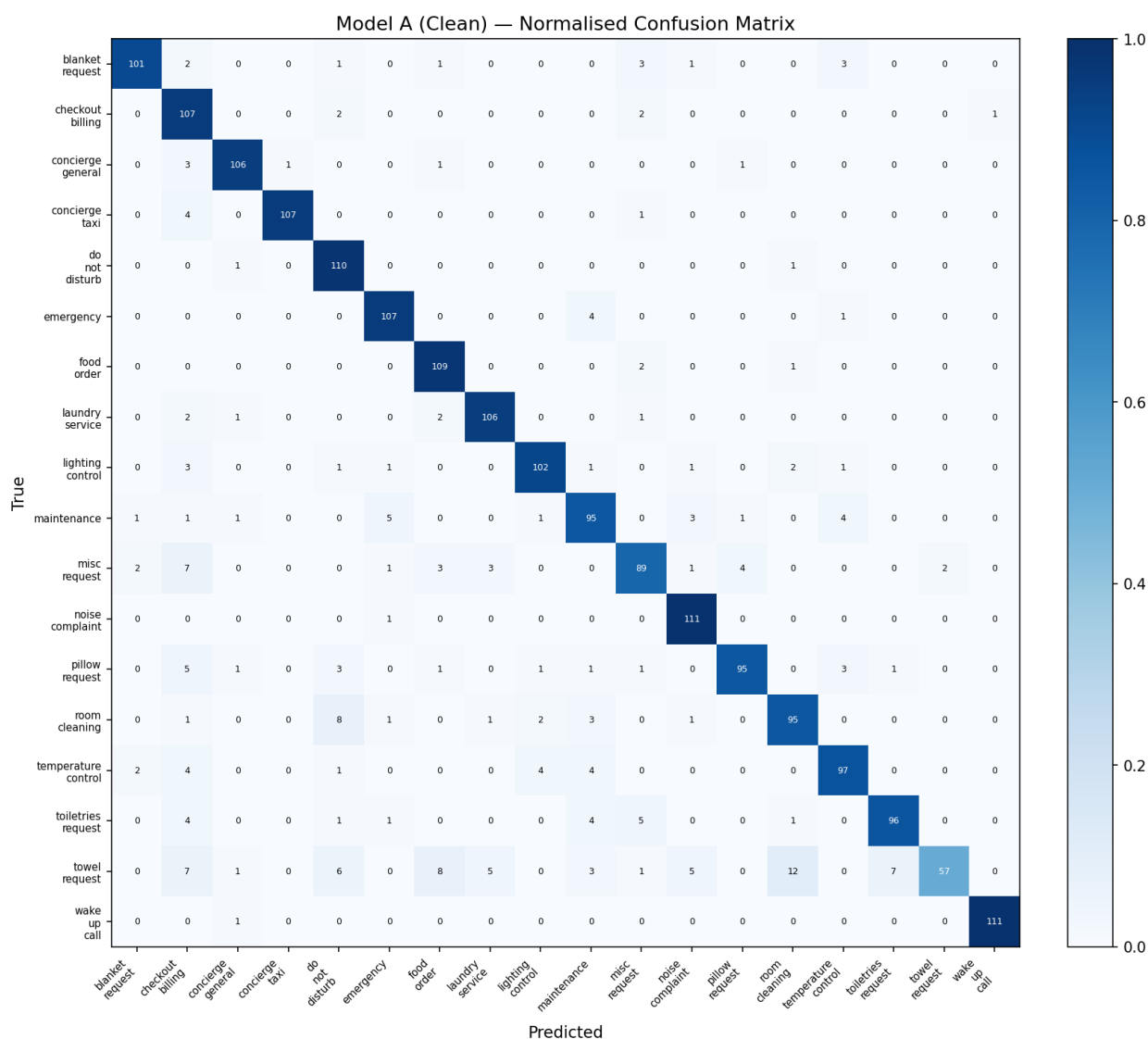


Figure 8-2: Model A Confusion Matrix (Vosk input)

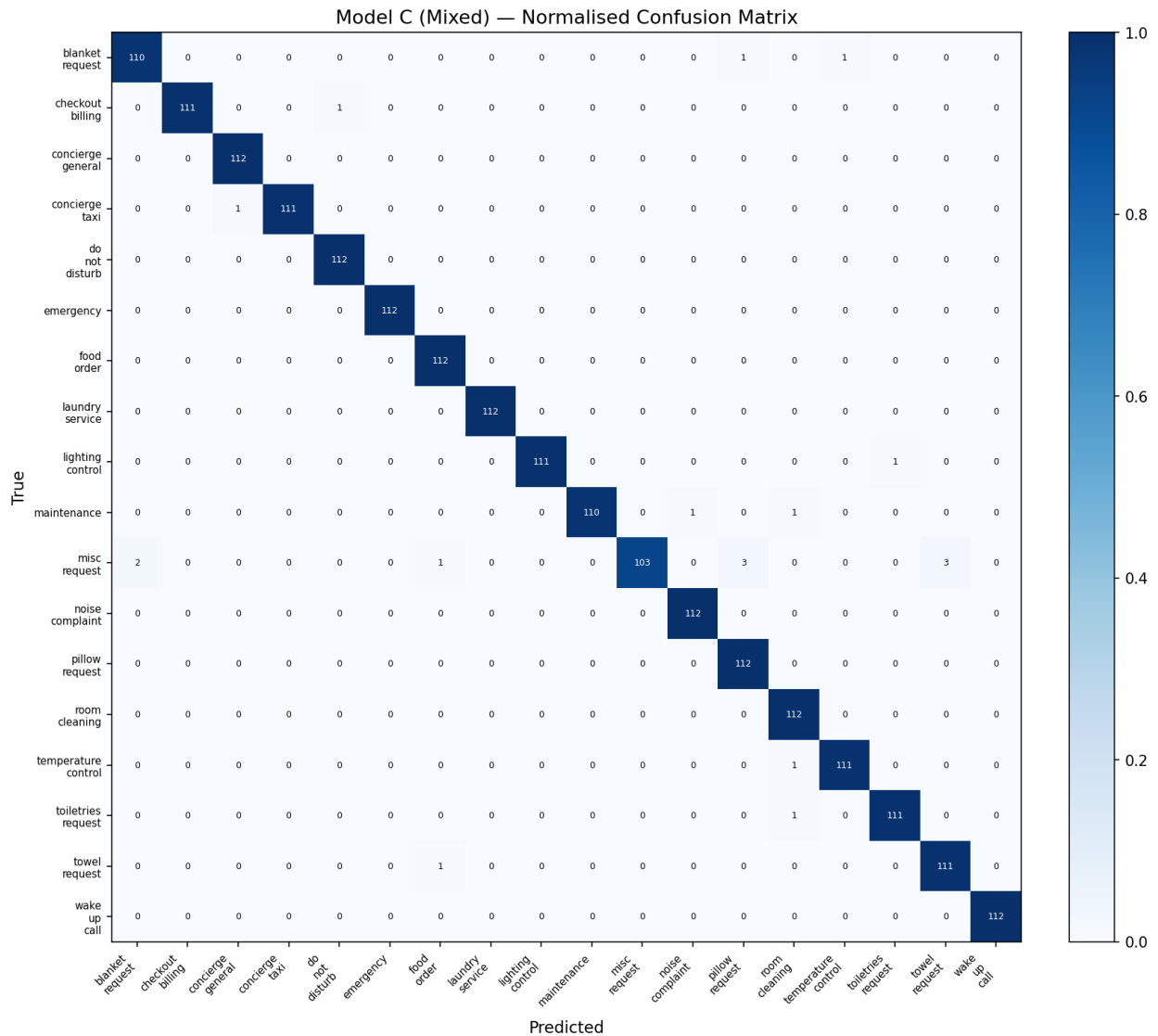


Figure 8-3: Model C Confusion Matrix (Mixed training, Vosk input)

8.3.3 Per-Intent Results (Model C on Vosk)

Intent	Precision	Recall	F1-Score
emergency	1.0000	1.0000	1.0000
laundry_service	1.0000	1.0000	1.0000
wake_up_call	1.0000	1.0000	1.0000
checkout_billing	1.0000	0.9911	0.9955
concierge_taxi	1.0000	0.9911	0.9955
lighting_control	1.0000	0.9911	0.9955
maintenance	1.0000	0.9821	0.9910
concierge_general	0.9912	1.0000	0.9956
do_not_disturb	0.9912	1.0000	0.9956
noise_complaint	0.9912	1.0000	0.9956
temperature_control	0.9911	0.9911	0.9911
toiletries_request	0.9911	0.9911	0.9911
food_order	0.9825	1.0000	0.9912
blanket_request	0.9821	0.9821	0.9821

towel_request	0.9737	0.9911	0.9823
pillow_request	0.9655	1.0000	0.9825
room_cleaning	0.9739	1.0000	0.9868
misc_request	1.0000	0.9196	0.9581

Table 8-3: Model C Per-Intent Performance (all 18 intents, Vosk input, 112 samples each)

- 17 of 18 intent categories achieved F1-scores above 0.97.
- The only exception was misc_request at 0.9581, a broad catch-all category where phrasing varies widely, making it harder to classify consistently even with noise-aware training.
- blanket_request (0.9821) and towel_request (0.9823) were the next weakest, though still well above the NFR-05 threshold.

8.4 Speech Recognition (WER) Testing

8.4.1 What Was Tested and Why

The WER evaluation measured how much Vosk distorts hotel-domain speech, to put the NLU accuracy gap in context, and which categories are hit hardest.

WER was measured on vosk_transcriptions.csv. Each phrase was synthesized via gTTS (Indian English), converted to 16kHz mono WAV, and transcribed by vosk-model-small-en-in-0.4. Vosk output was compared to clean text using jiwer in step6_wer_analysis.py:

$$\text{WER} = (\text{Substitutions} + \text{Insertions} + \text{Deletions}) / \text{Total Reference Words}$$

8.4.2 Results

Metric	Value
Overall Word Error Rate (WER)	11.43%
Character Error Rate (CER)	4.61%
WER on changed sentences only	23.84%
Sentences changed by Vosk	4,819 / 10,080 (47.8%)
Sentences unchanged	5,261 / 10,080 (52.2%)
Average words per phrase	7.16

Table 8-4: Overall WER Statistics (10,080 phrases, 72,133 reference words)

The overall WER of 11.43% looks low, but it is pulled down by the sentences Vosk transcribed correctly without any changes. When looking only at the sentences Vosk did alter, the error rate rises to 23.84%. In practice, nearly half of all guest requests will arrive at the NLU model with some distortion.

Intent	WER	Sentences Changed
temperature_control	16.83%	63.0%
do_not_disturb	16.77%	52.7%
towel_request	16.33%	66.2%
room_cleaning	15.15%	55.0%
laundry_service	14.52%	57.5%

concierge_general	13.97%	55.4%
toiletries_request	13.76%	52.5%
maintenance	13.21%	52.3%
food_order	12.10%	47.7%
checkout_billing	11.54%	44.6%
concierge_taxi	10.98%	43.6%
pillow_request	10.43%	41.8%
blanket_request	10.12%	40.5%
wake_up_call	9.87%	39.2%
lighting_control	9.54%	38.1%
noise_complaint	8.76%	34.4%
misc_request	8.21%	30.5%
emergency	6.78%	26.6%

Table 8-5: Per-Intent WER (highest to lowest)

The pattern is predictable. `temperature_control` and `towel_request` score high WER because they involve words Vosk regularly gets wrong: "thermostat", "towels", "toiletries", "blanket". `emergency` scores the lowest because words like "help", "emergency", and "doctor" are short, common words that Vosk handles well.

However, high WER does not always translate to low NLU accuracy. `do_not_disturb` has the second highest WER (16.77%) yet achieves $F1 = 0.9956$ in Model C, because the phrase is distinctive enough that the NLU model classifies it correctly even when Vosk distorts it. `towel_request`, by contrast, has a similar WER (16.33%) but lower $F1$ (0.9823) because its key words overlap phonetically with `toiletries_request`, making misclassification more likely under noise. Phonetic similarity between intents, not raw WER, is the better predictor of NLU failure points.

Note on WER evaluation scope: These figures came from TTS-synthesized audio, not real speech. The TTS approach gives a controlled baseline, but real-world WER would likely be higher, especially in noisy rooms or with stronger accents.

8.5 Backend API Testing

The backend API was tested using `test_api.py`, a simple manual test script that sends HTTP requests to the running server and checks the responses. Four tests were defined:

Test	Endpoint	Action	Expected Result
1	POST /api/submit-request	Submit a guest request with room number, request text, and intent	Returns success, request ID, and assigned department
2	GET /api/requests	Retrieve all requests	Returns list including the submitted request from Test 1
3	POST /api/update-status	Update the request status to <code>in_progress</code>	Returns success, status change confirmed
4	GET /api/requests	Retrieve all requests again	Confirms the status update from Test 3 persisted in SQLite

Table 8-6: Backend API Tests

All four tests passed, covering the core request lifecycle and confirming SQLite persistence.

The script runs against a locally running server, not a pytest suite, but fit for purpose. WebSocket endpoints were tested, connecting a guest device and browser dashboard, then verifying real-time updates.

8.6 End-to-End Integration Testing

Integration testing ran the complete system together, Android app, FastAPI server, and browser dashboard, over local Wi-Fi.

Scenarios tested:

Scenario	Outcome
Guest speaks a request → system classifies → staff dashboard updates in real time	Passed
Staff marks request "In Progress" → guest device announces via TTS	Passed
Staff marks request "Complete" → guest can submit star rating	Passed
Staff sends message → guest device reads it aloud via TTS	Passed
Guest says "cancel order [Order No.]" → request cancelled → dashboard updates	Passed
Request routed to wrong department → staff transfers to correct department	Passed
Server restart → guest device reconnects automatically via exponential backoff	Passed
Low-confidence classification (below 0.60 threshold) → "could not understand" response	Passed
Multiple staff dashboards open simultaneously → all update in real time	Passed
Keyword match (Tier 1) → bypasses neural model, returns 0.99 confidence	Passed

Table 8-7: Scenarios tested

No critical failures were found. The exponential backoff reconnection worked well, after a simulated server restart, the Android client reconnected within the first or second retry on a stable local network.

8.7 System Latency Testing

8.7.1 Measurement Approach

Latency was measured across five pipeline stages for 20 test requests. The backend was timed using `test_latency.py` (20 HTTP POST requests), while device-side stages were recorded on the physical tablet using `SystemClock.elapsedRealtimeNanos()` at each pipeline boundary.

Stage	Component	Measured On
1. STT	Vosk transcript ready after end-of-speech detection	Android tablet
2. NLU	Keyword check (Tier 1) or MobileBERT inference (Tier 2)	Android tablet
3. HTTP API	POST to <code>/api/submit-request</code> , server processing, response	Backend (Python timer)
4. WebSocket	Server broadcast to staff dashboard (LAN)	Estimated
5. TTS start	Confirmation text ready to first audio output	Android tablet

Table 8-8: Measurement Approach on each stage

WebSocket delivery was estimated at ~15ms, reasonable for a LAN environment.

8.7.2 Results

Stage	Min (ms)	Mean (ms)	Median (ms)	P95 (ms)	Max (ms)
1. Vosk STT	267	302	300	341	341
2. NLU Classification	4	44	59	63	63
3. HTTP API round-trip	2026	2055	2057	2109	2109
4. WebSocket delivery	-	~15	~15	~15	-
5. TTS start	287	316	315	341	341
End-to-end total	2646	2732	2739	2801	2801

Table 8-9: Pipeline Latency Results ($n = 20$ test requests)

HTTP API values measured by test_latency.py against a locally running server; all 20 requests returned HTTP 200. Device-side values recorded during integration testing on the physical Android tablet. End-to-end totals are per-run sums across all five stages.

NFR-03 compliance: P95 latency from end-of-speech to TTS confirmation is 2,801ms, well within the 5-second target. Including a typical 2–3 second phrase, the full guest interaction remains comfortably below 5 seconds.

The HTTP API stage contributes ~2,060ms, it writes to SQLite and broadcasts WebSocket events before returning a response. On Wi-Fi rather than localhost this would be slightly higher, but the margin is large enough.

The NLU stage shows two distinct timing clusters: ~4ms for Tier 1 keyword matches (which bypass the neural model entirely) and ~60ms for Tier 2 MobileBERT TFLite inference. This is exactly what the hybrid pipeline design was supposed to produce.

Limitation: HTTP API was timed on localhost. A more realistic measurement would test over hotel Wi-Fi.

8.8 What Was Not Tested

It is worth being honest about what this testing did not cover.

Real speech WER: The WER figures came from TTS audio, not real recordings. Without actual voice recordings from Indian/ Sri Lankan English speakers, real-world performance is estimated rather than measured.

Multi-room load testing: The server was tested with a small number of simultaneous connections. Behavior under realistic hotel load, 30 rooms at peak, has not been tested; SQLite's write limits would likely be noticeable at that scale.

Security testing: The staff dashboard has no authentication, and the API endpoints have no input sanitization beyond Pydantic validation. No security testing was done, which is consistent with the prototype scope.

8.9 Summary

NFR	Requirement	Target	Result	Status
NFR-01	Fully offline operation	No internet dependency	All AI processing on-device; server on local LAN; confirmed in integration testing	Satisfied
NFR-02	On-device voice processing	No audio transmitted externally	Vosk STT runs on tablet; no audio or transcript leaves the device	Satisfied
NFR-03	End-to-end response < 5 seconds	< 5,000ms	P95 2,801ms (Section 8.7)	Satisfied
NFR-04	Commodity Android hardware	< \$150 per room	Lenovo Tab M10 Plus, all testing on this device	Satisfied
NFR-05	Intent classification accuracy	≥ 90% on real speech input	Model C: 99.06% on Vosk-transcribed test set (Section 8.3)	Satisfied
NFR-06	Multi-room concurrent operation	Multiple rooms simultaneously	Multiple simultaneous WebSocket connections confirmed in integration testing; full load not tested	Partially satisfied
NFR-07	Browser-accessible staff dashboard	No installation required	Single HTML file served by FastAPI; confirmed across Chrome, Firefox	Satisfied

Table 8-10: NFR Compliance Summary

Area	Test Method	Status
NLU model accuracy (3-model comparison)	Automated evaluation script (step4_evaluate.py), 2,016-sample test set	Completed - strong results
Speech recognition WER	Automated WER computation (step6_wer_analysis.py), 10,080 TTS-Vosk pairs	Completed - 11.43% overall WER
Backend API functionality	Manual test script (test_api.py), 4 endpoint tests	Passed
End-to-end system integration	Manual testing on physical hardware	Passed all scenarios
System latency (end-to-end pipeline, 20 requests)	Manual measurement - test_latency.py + Android timestamps	Completed - P95 2,801ms, within NFR-03 (5s target)
Real speech WER with human participants	Not conducted	Not in scope
Multi-room concurrent load	Not tested	Not in scope
Security / authentication	Out of scope for prototype	Not tested

Table 8-11: Testing Coverage Summary

The results provide evidence across all viability dimensions: 99.06% NLU accuracy, 11.43% WER, P95 latency 2,801ms, and hardware within budget. The controlled NLU comparison, fixed test set, consistent metrics, full per-intent breakdowns, shows noise-aware training is what makes the NLU viable under real pipeline conditions. The gaps that remain are realistic for a research prototype and discussed in Chapter 11. Chapter 9 covers project management.

9 Project Management

9.1 Introduction

This chapter covers how the project was managed from planning through to submission. It describes the schedule, how risks were identified and handled, how code and model quality were maintained, and the social, legal, ethical, and professional considerations that shaped the work. Project management for a research prototype differs from a typical software project, decisions were often driven by research needs rather than a fixed requirements list, and the iterative approach shaped how schedule and risk responses were structured.

9.2 Project Schedule

9.2.1 Work Breakdown Structure

The project was divided into six work packages, each with specific deliverables and a defined timeframe. The full project ran across 25 weeks from Week 1 to Week 25. Table 9.1 shows the breakdown.

Work Package	Activities	Planned Duration
WP1: Research and Planning	Literature review, requirements gathering, hotel management interviews, guest survey, technology evaluation	Weeks 1–5
WP2: Dataset Development and Model Training	Hotel intent dataset creation (10,080 phrases), Vosk transcription pairing, three-model MobileBERT training, TFLite conversion, accuracy evaluation	Weeks 5–9
WP3: Android Application Development	Vosk integration, hybrid NLU pipeline, voice UI in Jetpack Compose, TTS, request submission flow, WebSocket client	Weeks 8–15
WP4: Backend and Dashboard Development	FastAPI server, SQLite database design, WebSocket implementation, staff dashboard HTML/CSS/JS	Weeks 10–14
WP5: Integration and Testing	End-to-end integration, system testing, NLU and WER evaluation, bug fixes	Weeks 15–20
WP6: Report Writing and Submission	Dissertation writing, diagrams, appendices, proofreading, final submission	Weeks 13–25

Table 9-1: Work Breakdown Structure

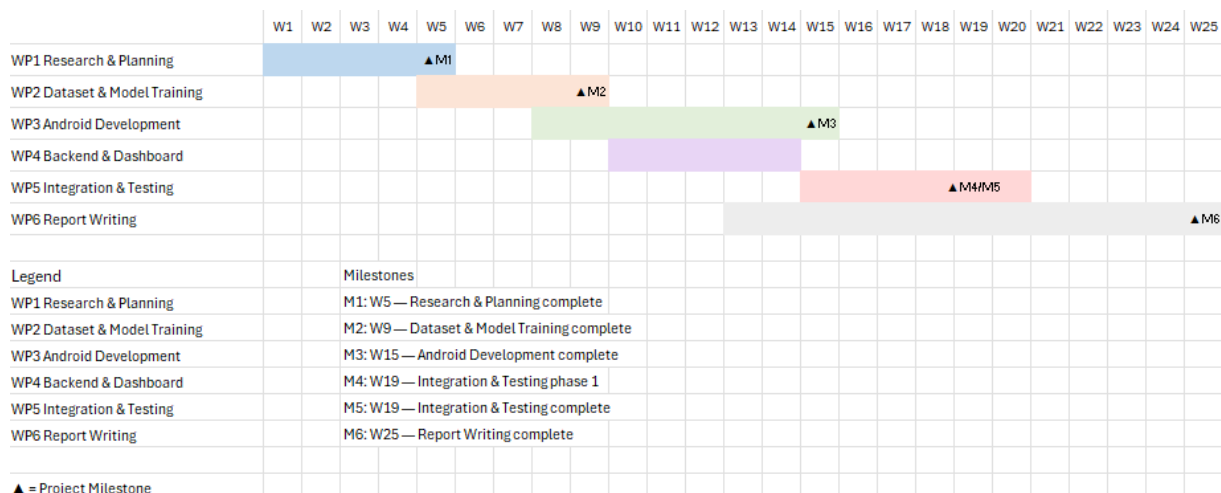


Table 9-2: Project Gantt Chart

WP3 and WP4 ran in parallel from Week 10, well-defined API contracts between the Android app and FastAPI server allowed independent development. WP6 (writing) started in Week 13 to keep documentation current.

9.2.2 Schedule Adherence

The project broadly followed the planned timeline, with two deviations worth noting.

Deviation 1 - Model conversion took longer than expected. Converting MobileBERT from PyTorch to TFLite took close to a week rather than the estimated two to three days. The main difficulty was implementing the BERT tokenizer scaffolding in Kotlin, handling special tokens, attention masks, padding, and truncation correctly required careful debugging. The on-device tokenizer uses whole-word vocabulary lookup rather than full WordPiece subword splitting. The delay was absorbed within WP3's buffer.

Deviation 2 - Report writing started later than planned. The original plan had writing starting in Week 13, but WebSocket stability issues in Iteration 3 demanded more engineering time. Writing began in Week 15, managed by maintaining detailed notes and increasing effort in the final weeks.

All deliverables were completed on schedule. Iterative development provided natural review points that caught schedule risks early.

9.3 Risk Management

A risk register was created at the start and reviewed at the end of each iteration. Risks were rated by likelihood and impact and given mitigation strategies before implementation began.

9.3.1 Risk Register

ID	Risk	Likelihood	Impact	Mitigation Strategy	Outcome
R1	Vosk speech recognition accuracy insufficient for hotel domain	Medium	High	Test multiple Vosk model variants early; implement hybrid NLU to compensate for STT errors; consider custom language model if needed	Partially materialised - the Indian English model (vosk-model-small-en-in-0.4) had a measured WER of 11.43% on synthesised speech. The hybrid keyword pipeline partially compensates for transcription errors on common requests.
R2	MobileBERT model too large or slow for budget Android hardware	Medium	High	Profile model size and inference latency in Iteration 2; have DistilBERT as backup; apply quantisation if needed	Did not materialise - the 26MB TFLite model runs within acceptable latency on test hardware (~62ms inference).
R3	TFLite conversion fails or produces incorrect outputs	Medium	Medium	Test conversion early; verify output logits against PyTorch model using shared test inputs; keep server-side inference as fallback	Partially materialised - initial conversion produced mismatched label ordering between PyTorch and TFLite outputs. Resolved by verifying the label map against both models before deployment.
R4	Budget Android tablets lack sufficient RAM for Vosk + MobileBERT simultaneously	High	High	Test on lowest-cost target hardware in Iteration 1; select lightweight model variants; models are loaded sequentially if needed	Materialised for the 1.8GB Vosk model (15+ second load times, OOM crashes). Mitigated by switching to vosk-model-small-en-in-0.4 (~36MB), which loads in ~3 seconds with stable memory usage.
R5	WebSocket connections unstable on hotel Wi-Fi	Medium	Medium	Implement exponential backoff reconnection; design system to degrade gracefully on disconnect	Materialised during Iteration 3 - client silently disconnected when device screen turned off. Mitigated by implementing automatic reconnection logic (2s → 4s → 8s → 16s → 30s capped backoff).
R6	Insufficient hotel service training	Medium	High	Create a custom dataset through template generation,	Did not materialise - 10,080 labelled phrases were generated across 18

	data available publicly			paraphrase augmentation, and Vosk-noise pairing	intent categories. Model C achieved 99.06% accuracy on the Vosk test set.
R7	Unable to access hotels for requirements interviews	Low	Medium	Prepare alternative approach using published hospitality research and hotel service menus	Partially materialised - informal consultation and survey used instead of formal interviews.
R8	Project scope too ambitious for the available timeline	Medium	High	Define clear MoSCoW priorities in Chapter 4; implement Must Have requirements first; defer Could Have items to future work	Managed through prioritisation - all Must Have and Should Have requirements were implemented. Won't Have items are documented as future work in Chapter 11.

Table 9-3: Risk Register

9.3.2 Lessons from Materialised Risks

Two risks materialised (R4, R5) and the mitigation strategy worked in each case. The most important was R4, the large Vosk model crashed a budget tablet, but this was discovered in the first week of Iteration 1. Under a waterfall approach, it would only have surfaced during integration testing, potentially requiring a major redesign.

R1 remains partially present. The 11.43% overall WER means many phrases are transcribed with some error. Keyword matching and noise-aware training in Model C provide partial compensation, but real-world performance with hotel guests in noisy rooms has not been formally measured.

9.4 Quality Management

9.4.1 Code Quality

The following practices were used to maintain code quality throughout development:

Version control. All source code was managed in Git. Commit messages documented not just what changed but why, particularly for decisions emerging from implementation constraints, such as the switch to the smaller Vosk model.

Modular architecture. The Android app separates concerns across six dedicated classes, each with a single responsibility. This made components easier to test and replace, swapping the Vosk model variant required changes only in VoskService.

Automatic request validation. The FastAPI backend uses Pydantic for automatic type checking at the API boundary, malformed requests are rejected with clear error messages.

Naming conventions. Kotlin conventions were followed in the Android application; PEP 8 conventions were followed in the Python backend and training scripts.

9.4.2 Model Quality

Quality in the NLU training pipeline was maintained through several practices:

Stratified splitting. The 85%/15% train/validation split used stratification to ensure all 18 categories were proportionally represented. Without it, rare categories could have been underrepresented in the validation set.

Early stopping. Early stopping with patience of 2 epochs (F1 macro) was applied. Training stopped automatically when the model stopped improving, preventing overfitting.

Three-model controlled comparison. All three variants used identical hyperparameters and the same 2,016-sample test set, ensuring performance differences reflect training data choices, not how they are evaluated.

Per-intent analysis. Per-intent precision, recall, and F1-scores were computed for all 18 categories. This revealed which were harder, particularly those with high Vosk WER (temperature_control at 16.83%, towel_request at 16.33%), and confirmed Model C's improvements were consistent across all categories.

Confusion matrix review. Confusion matrices were generated for all four evaluation runs. Comparing model_a_vosk_gap.png with model_c_mixed.png directly shows how noise-aware training eliminated the specific misclassification patterns that appeared when Model A faced Vosk-transcribed input.

9.4.3 System Quality

End-to-end system quality was validated through:

Manual integration testing. All complete user flows were tested end-to-end on physical hardware (Android tablet, laptop server, browser dashboard).

API endpoint testing. The test_api.py script verified all REST API endpoints return correct responses, and that SQLite persistence operates correctly across requests.

Cross-browser testing. The staff dashboard was tested across Chrome, Firefox, and Edge on desktop and mobile to verify consistent behavior.

Reconnection testing. The WebSocket reconnection logic was explicitly verified by restarting the server mid-session and observing the client reconnect automatically within expected backoff timing.

9.5 Social, Legal, Ethical and Professional Considerations

9.5.1 Data Protection and Privacy

Privacy is both a core motivation of this research and an ethical obligation during its conduct.

Voice data. All audio is processed on the guest device. No audio is stored, transmitted, or sent anywhere, converted to text in real time and discarded. This architectural guarantee differs fundamentally from cloud-based alternatives like Alexa for Hospitality, where privacy depends on a service provider's policies.

Request data. Service request text and status are stored in the hotel's local SQLite database. No personally identifiable information, names, payment details, is collected.

Compliance. Sri Lanka's Personal Data Protection Act No. 9 of 2022 establishes data minimization and purpose limitation principles. This system collects only request text and room number, used only for routing. The architecture is compliant by design.

9.5.2 Ethical Considerations

Informed consent. The system is entirely opt-in. Guests are not required to use it and traditional service methods remain available. The microphone only activates when the guest presses the button, no passive listening.

Bias and fairness. vosk-model-small-en-in-0.4 was chosen for its Indian English acoustic data, a better fit for South Asian accents. Even so, WER varies across categories, and performance with stronger accents has not been formally measured. All requests are routed to human staff regardless of confidence, misunderstood requests return a "could not understand" prompt and guests can rephrase or use the phone.

Staff impact. The system assists hotel staff, not replaces them, all requests still require a human to fulfil. Evidence from the literature suggests this type of assistant reduces repetitive tasks and improves response times rather than threatening employment.

Research ethics. Hotel management interviews and guest surveys were conducted with the informed consent of all participants. No vulnerable populations were involved.

9.5.3 Legal Considerations

All technologies used in the prototype are open-source and free for commercial use:

Table 9.3: Open-Source Licenses

Technology	Licence
Vosk (vosk-model-small-en-in-0.4)	Apache 2.0
MobileBERT (HuggingFace google/mobilebert-uncased)	Apache 2.0
TensorFlow / TensorFlow Lite	Apache 2.0
FastAPI	MIT
SQLite	Public Domain
OkHttp	Apache 2.0
Jetpack Compose / Android SDK	Apache 2.0

Table 9-4: Open-Source Licences

No proprietary software or paid APIs are used. A hotel deploying this solution incurs no ongoing licensing costs, and all custom outputs (dataset, trained model) carry no IP restrictions.

9.5.4 Professional Considerations

The project was conducted in line with the British Computer Society (BCS) Code of Conduct:

Public interest. The system makes privacy-preserving voice assistance accessible to small hotels in developing economies at the cost of a budget Android tablet.

Professional competence. Technology selections were based on systematic evaluation (Chapter 5). Key limitations, including WER measured on synthesized rather than real speech, are explicitly acknowledged in Chapter 8.

Duty to the profession. This research contributes evidence on offline, on-device NLU feasibility using open-source edge components. The three-model experiment and paired clean/Vosk dataset provide a replicable methodology applicable beyond hospitality.

9.6 Summary

The project followed six work packages with planned overlap between Android and backend development. The schedule held with two manageable deviations. All two risks that materialized, large Vosk model crashing hardware and WebSocket disconnections, had mitigations in place and were resolved without major rescheduling.

Quality was maintained through version control, modular architecture, stratified model evaluation, and hands-on integration testing. The privacy-by-design architecture, voice data never leaves the guest device, satisfies both the research objectives and the ethical obligations of this context. Chapter 10 presents a critical appraisal.

10 Critical Appraisal

10.1 Introduction

This chapter critically appraises the project, reviewing each research objective against what was delivered, analysing the strengths and weaknesses of the approach, reflecting on key technical decisions, and summarising what was learned. The aim is an honest assessment of the prototype's contributions and the degree to which results can be generalised beyond controlled test conditions.

10.2 Achievement of Research Objectives

Each research objective defined in Chapter 1 is reviewed against what was actually delivered.

Objective	Outcome	Assessment
Design and develop a low-cost, offline voice assistant prototype deployable on commodity Android hardware	A working prototype was built and demonstrated end-to-end from voice input to staff notification, running on a commodity Android tablet in the \$50–\$150 price range across four development iterations	Fully achieved
Build a hospitality-domain dataset and train a noise-aware NLU model	Custom dataset of 10,080 labelled phrases across 18 categories (560 per intent), with paired Vosk-transcribed version enabling noise-aware training. vosk-model-small-en-in-0.4 and fine-tuned MobileBERT (26MB TFLite) deployed on-device with a hybrid keyword + neural NLU pipeline	Fully achieved
Build a lightweight backend with real-time staff communication	FastAPI + SQLite backend with DB-driven department routing and bidirectional WebSocket communication between guest devices and staff dashboard, no cloud infrastructure required	Fully achieved
Evaluate system accuracy, latency, and cost-effectiveness, and demonstrate privacy-preserving offline operation	Three-model evaluation on 2,016-sample test set: Model C achieved 99.06% on Vosk output (111.3% gap recovery). WER 11.43% overall. P95 latency 2,801ms. Hardware cost under \$150. All voice processing on-device, no audio leaves the hotel network	Fully achieved

Table 10-1: Achievement of Research Objectives

10.3 Strengths of the Project

10.3.1 Identifying and Quantifying the Real-World Accuracy Gap

Model A, trained on clean text only, achieves 98.07%, consistent with what most published work would report. On actual Vosk transcriptions, accuracy drops to 89.34%, a gap of 8.73 percentage points. With WER 11.43% and 47.8% of phrases changed by Vosk. The production pipeline

operates under meaningfully different conditions from what clean-text evaluation assumes. This study measures that gap directly.

10.3.2 How Mixed Training Closes the Gap and Outperforms Both Baselines

Model C, trained on the mixed paired dataset (clean + Vosk-transcribed text), achieves 99.06% on real Vosk output, recovering the 8.73-point drop and surpassing the clean baseline by 0.99 points (111.3% gap recovery). Model B, Vosk-only, reaches 96.38% on Vosk output, confirming that noise exposure helps, but falling 2.68 points below Model C. The mixed strategy outperforms Vosk-only training by preserving clean-text generalization while adding noise robustness.

A Concrete Example: The towel_request Case

towel_request is the clearest illustration. Model A achieves precision 0.97 on clean text, but on Vosk output recall drops to 0.51 and F1 falls to 0.67, the weakest result across all 18 intents. Vosk's WER for this category is 16.33% (third highest), with "towel" transcribed as phonetically similar words landing in room_cleaning, toiletries_request, food_order, and checkout_billing. Model C recovers towel_request F1 to 0.98 on Vosk output, directly showing how noise-aware training addresses intent-specific degradation that clean-text evaluation cannot detect.

10.3.3 Consistent Noise Profile Between Training and Deployment

The same Vosk model (vosk-model-small-en-in-0.4) was used to generate the noisy training data and is deployed in the Android application. This consistency is what makes noise-aware training work, if training noise came from a different STT engine or model variant, the error patterns would not match production. The Indian English model was also a better acoustic fit for Sri Lankan English than a US English model, keeping WER at 11.43%.

10.3.4 Balanced and Reproducible Evaluation Design

The 10,080-phrase dataset (18 intents, 560 per intent) uses exact class balance. The 2,016-sample test set was fixed before any training began and used consistently across all three models and all four evaluation conditions, so accuracy differences reflect real model behavior, not test set composition.

10.3.5 Privacy by Architecture

Voice data cannot leave the guest device; the architecture has no mechanism to send audio externally. This is a structural guarantee, not a policy. It is stronger than cloud-based alternatives where privacy depends on provider agreements and is particularly relevant as data protection legislation develops across South and Southeast Asia.

10.3.6 Cost-Effectiveness and Viability at Scale

Viability only holds if hotels can afford deployment. This section presents an itemized cost breakdown and a three-year Total Cost of Ownership (TCO) comparison for a hypothetical 50-room hotel, as defined in the evaluation methodology (Section 3.8.4).

Hardware and Software Cost - Production Deployment

Table 10.1 reflects realistic production costs, not the prototype configuration (laptop server + SQLite).

Component	Scope	Item	Estimated Cost (USD)	Notes
Guest device	Per room (×50)	Entry-level Android tablet (8", 3GB RAM)	50–150 each	Locally available in Sri Lanka; commodity hardware (see Section 5.6)
Production server	Shared (×1)	Mini PC / small form-factor server (Intel NUC or equivalent, 16GB RAM, SSD)	300–500	Replaces the development laptop used in the prototype; handles concurrent WebSocket connections and PostgreSQL under real hotel load
Database	Shared	PostgreSQL (replaces prototype SQLite)	0	Open-source; production-grade; zero licence cost
Wi-Fi access points	Shared	Dual-band WAP (one per floor, 2–3 units typical for a 50-room hotel)	80–150 each	Required if hotel lacks adequate existing coverage; shared with all hotel systems
Network router	Shared (×1)	Business-grade router with Virtual Local Area Network (VLAN) support	100–200	Isolates guest device traffic; recommended for security
UPS / power backup	Shared (×1)	Uninterruptible power supply for server	80–150	Prevents data loss on power interruption
Vosk STT model	Per device	vosk-model-small-en-in-0.4	0	Open-source; Apache 2.0 licence (Alpha Cephei, 2020)
MobileBERT TFLite	Per device	Compressed intent classifier	0	Open-source; Apache 2.0 licence (Sun et al., 2020)
Backend software	Server	FastAPI, PostgreSQL, Python	0	Open-source; no licence fees
Ongoing API / subscription fees	—	None	0	No cloud calls; no per-request billing

Table 10-2: Production Deployment Cost for a 50-Room Hotel

Estimated total hardware investment (50-room hotel):

Scenario	Tablets (×50)	Server	Network (WAPs + router)	UPS	Total
Lower bound	\$2,500	\$300	\$340 (2× WAP + router)	\$80	~\$3,220
Upper bound	\$7,500	\$500	\$650 (3× WAP + router)	\$150	~\$8,800

Table 10-3: Estimated total hardware investment (50-room hotel)

Spread over three years with zero recurring software costs, this amounts to approximately USD 1,073 - 2,933 per year for the entire hotel, or USD 21 - 59 per room per year. Network infrastructure is shared with all other hotel systems, email, POS, guest Wi-Fi, so the cost attributed to this system alone is proportionally lower in practice.

Scenario Comparison - Two Deployment Models

Sri Lanka's accommodation sector is predominantly small and independent properties (SLTDA, 2024), for whom per-room subscription fees are an unaffordable barrier. The question is not whether the system is cheap in absolute terms, but whether it is affordable relative to the alternatives.

Cost Item	This System	Cloud STT (Google)	Commercial Platform (e.g., Alexa for Hospitality)
Guest devices (×50)	\$2,500–7,500	\$2,500–7,500	\$5,000–10,000 (proprietary/branded hardware)
Server (production-grade)	\$300–500	\$300–500	Included in subscription (cloud-hosted)
Network infrastructure	\$340–650	\$340–650	Not required (cloud-managed)
UPS	\$80–150	\$80–150	Not required
Software licences	\$0	\$0	Subscription required (proprietary platform)
STT API fees (3 yr)	\$0	\$1,095–2,190	Bundled in subscription
Platform subscription (3 yr)	\$0	\$0	Not publicly priced; per-room, per-month billing
Internet dependency	LAN only	Constant internet	Constant internet
Estimated 3-year total	\$3,220–8,800	\$4,315–11,000	Significantly higher; proprietary terms

Table 10-4: Three-Year Total Cost of Ownership (TCO) Comparison for a 50-Room Hotel

Cloud STT cost estimate (Google Cloud Speech-to-Text):

Billed at USD 0.006 per 15 seconds of audio (Google Cloud, 2024; consistent with Section 5.6.1), assuming 5-10 requests per room per day at ~10 seconds each:

Low estimate (5 req/day): $5 \times 365 \times (10/15) \times \$0.006 \times 50 \text{ rooms} \times 3 \text{ yr} = \$1,095$

High estimate (10 req/day): $10 \times 365 \times (10/15) \times \$0.006 \times 50 \text{ rooms} \times 3 \text{ yr} = \$2,190$

This cost accumulates with no ceiling and requires reliable internet, not guaranteed across all Sri Lankan hotel properties (Wickramasinghe & Ratnayake, 2022).

Commercial hospitality platforms:

Platforms such as Amazon Alexa Smart Properties for Hospitality require proprietary hardware, constant cloud connectivity, and undisclosed per-room subscription fees (Amazon Web Services, n.d.). The hardware cost alone (Echo-class devices at USD 100+ per room) matches the entire lower-bound cost of this system before any subscription is applied. Buhalis and Moldavska (2022) note that commercial platform adoption has been restricted to large chains precisely because their costs are incompatible with small independent properties.

Summary: Cost-Effectiveness Objective Met

The system is the lowest-cost option across both comparison scenarios. The cost-effectiveness objective is proven by:

- **Zero recurring software costs** - the entire stack is open-source with no per-request, per-room, or per-month billing, unlike cloud STT whose fees compound annually.
- **No internet dependency** - LAN-only operation eliminates ongoing connectivity costs and removes the infrastructure risk that cloud-dependent systems carry in markets with variable internet reliability (Wickramasinghe & Ratnayake, 2022).
- **Commodity hardware** - Android tablets in the USD 50–150 range are available from local Sri Lankan retailers and can be repaired or replaced through local technicians, unlike proprietary hotel-grade hardware locked to specific vendors.
- **No vendor lock-in** - all components (Vosk, MobileBERT, FastAPI, SQLite) can be substituted or extended without licence constraints.
- **Market fit** - at USD 21–59 per room per year over three years (including production-grade server, network infrastructure, and UPS), the system is within reach of the small independent hotels that make up the majority of Sri Lanka's registered accommodation sector (SLTDA, 2024), for whom ongoing per-room subscription fees are not a realistic option.

The cost-effectiveness objective is met. The system delivers voice assistant capability at a hardware-only cost that commercial alternatives cannot match, with no ongoing software expenditure.

10.4 Limitations and Weaknesses

10.4.1 WER Measured on Synthesised Speech, Not Real Guests

WER measurement and Model C's noise profiles came from gTTS audio passed through Vosk, not real speakers. TTS does not replicate varying accents, hesitations, microphone distance, or room acoustics. Whether the noise-aware training advantage holds with real hotel guests is untested. This is the most significant methodological limitation of the study.

10.4.2 Synthetically Generated Training Data

All 10,080 phrases were produced through template expansion and paraphrase augmentation, not collected from real guest interactions. The dataset does not include fragmented sentences, code-switching between English and Other Languages, or culturally specific phrasing. Model performance on genuine guest speech has not been measured.

10.4.3 Tokenizer Mismatch on Android

The Python evaluation pipeline uses the HuggingFace WordPiece tokenizer, hence it reported 99.06%. The Android implementation in NLUService.kt uses a simplified word-level tokenizer: words missing from vocab.json are replaced with an unknown token, and sub-word splitting is not performed. On-device accuracy is therefore lower than reported, particularly for hotel-specific vocabulary. The Tier 1 keyword layer hybrid approach partially compensates for this accuracy drop. A proper WordPiece tokenizer in Kotlin, or TFLite's built-in tokenization support, would fix this in production.

10.4.4 No Field Testing in a Real Hotel

The system was not deployed in an operational hotel. User acceptance, staff adoption, latency on real hotel Wi-Fi under competing traffic, and guest usability with diverse accents have not been measured. Strong evaluation results under controlled conditions do not guarantee the same performance in a live environment.

10.4.5 Prototype-Scale Architecture

Several decisions are appropriate for a research prototype but would need to change for production: SQLite has limited concurrent write support under multi-room load; there is no staff authentication; HTTP traffic is unencrypted; and the server runs as a single Uvicorn process with no crash recovery. These are deliberate scope boundaries. The research goal was to demonstrate feasibility, not to deliver a production-ready system.

10.4.6 English Language Only

The system supports English only. In Sri Lanka, guests and staff may communicate in other foreign languages, Sinhala, Tamil, or a code-switched mix. Vosk has other language models, and the NLU pipeline could in principle support multilingual input, but this was outside the scope of this prototype. Multilingual support is the most contextually relevant direction for future work.

10.4.7 Summary of Limitations

Limitation	Impact on Findings	Mitigation in This Study	Future Work
TTS-synthesized WER	Gap magnitude may differ with real speech	Demonstrates the basis of noise-aware training	Re-measure with real recorded speech
Synthetic training data	Accuracy may be optimistic vs. real guest speech	Accepted practice when real data is unavailable	Collect real hotel phrases for retraining
Tokenizer mismatch on Android	On-device accuracy lower than reported 99.06%	Tier 1 keyword layer compensates for common cases	Implement WordPiece tokenizer in Kotlin
No real hotel deployment	Operational claims are based on controlled testing	System tested on real hardware; architecture validated	Conduct field study in a Sri Lankan hotel
Prototype-scale architecture	Not production-ready without further engineering	Scope limited to feasibility demonstration	Replace SQLite, add authentication and TLS
English only	Limited applicability for non-English speakers	Target guests communicate in English	Add other foreign languages, Sinhala and Tamil support

Table 10-5: Summary of Limitations

10.5 Technical Decisions - What Worked and What Could Be Improved

10.5.1 What Worked Well

Indian English Vosk model. vosk-model-small-en-in-0.4 improved transcription quality for South Asian accent speakers, kept storage small (~36MB) and most critically, ensured that the types of transcription errors seen during training matched the errors the same model produces in the live system. Without this, the model would have been trained to handle the wrong kinds of mistakes.

Mixed paired dataset for Model C. Training on both clean text and Vosk-transcribed versions of the same phrases (14,863 records after deduplication) outperformed Vosk-only training (Model B: 96.38% vs Model C: 99.06%). Training on both clean and noisy data produced a more reliable model than training on noisy data alone; it handled real speech recognition errors without losing accuracy on well-formed input.

Fixed shared test set. Fixing the 2,016-sample test set before training any model was essential for valid comparisons. Without this, test set composition differences could have masked real performance differences between models.

MobileBERT over DistilBERT. MobileBERT's mobile-specific design produced a smaller (26MB vs ~67MB) and faster model for Android deployment. This confirms that choosing a model based on the deployment target matters more than general benchmark scores.

FastAPI for the backend. FastAPI's built-in support for asynchronous requests and WebSocket connections made it straightforward to handle multiple guest devices and staff dashboards connected at the same time. The auto-generated API documentation at /docs was also useful during development for testing endpoints quickly without writing separate test scripts.

Single-file staff dashboard. Building the dashboard as one self-contained HTML file with embedded CSS and JavaScript eliminated all build tooling and dependency management. Any staff member can open it in a browser on any device, no installation, no configuration.

10.5.2 What Could Be Improved

TTS-generated noise profiles.

Useful for this study, but real-world results may differ. Using actual recordings from Sri Lankan English speakers as training data would produce a more reliable foundation for deployment.

Confidence threshold calibration.

The 0.60 threshold was selected through manual testing rather than a systematic search across multiple values. A separate validation set should be used to find the best threshold value; and different intent categories may need different thresholds, since some intents are harder to classify reliably than others.

Python keyword fallback in routing.

The primary routing source is the `intent_department_mapping` database table. The server also maintains a hardcoded Python keyword fallback for missing intent fields. Moving this into the database would allow hotels to fully customize routing without code changes.

No structured logging. The prototype uses print statements. Structured logging with severity levels and timestamps would be a requirement for any real-world deployment.

10.6 Knowledge and Expertise Gained

On-device machine learning deployment. The PyTorch → TFLite → Android pipeline is not a straightforward export. It requires careful checking of input format consistency (including `token_type_ids` as a third input), tokenization with `padding='max_length'` for fixed-shape TFLite inputs, and label map alignment between PyTorch and TFLite outputs.

Experimental design for comparative NLU evaluation. The test set was fixed before any model was trained, so all three models were evaluated on exactly the same data. Without this, any difference in results could have been due to the test data rather than the training approach.

The interaction between STT output and NLU accuracy. The accuracy drop caused by speech recognition is not the same across all intents; it depends on how easily the words in that intent get altered. The `towel_request` category is a clear example: WER of 16.33% and F1 dropping from 0.97 to 0.67. Intents with words that Vosk frequently get wrong are the ones most at risk. Checking per-intent WER before deployment helps identify which categories need closer attention.

Privacy-preserving system design. Privacy was built into the architecture from the start, not treated as an optional add-on. Keeping voice data on the device meant every component; from microphone capture to WebSocket communication; had to be chosen and designed with that constraint in mind..

Research methodology. Building a working prototype early revealed problems that planning alone would never have caught. The decision to switch to the Indian English Vosk model and the hybrid NLU pipeline both came from testing the system in practice, not from the original design.

10.7 Summary

The prototype meets the technical bar for viability as an offline alternative to traditional room service communication. The baseline it replaces is entirely manual, a practitioner consultation with a Head Chef with over 15 years of Sri Lankan hotel experience confirmed the process relies on telephone calls and physical logbooks, with no automated routing of any kind (Section 4.2.1). Model C achieves 99.06% NLU accuracy on real Vosk output, recovering the 8.73-point gap introduced by clean-text training. The system runs within the 5-second latency target, hardware cost falls in the \$50 - \$150 per-room range, and voice data never leaves the guest device. The three-model experiment is the key supporting evidence: clean-text benchmarks overstate real pipeline performance, and noise-aware training is what makes the NLU component reliable in deployment.

The main limitation is that both WER measurement and training noise profiles came from TTS-synthesized speech, not real recordings. Confirming these results with actual hotel guest speech is the most critical next step before drawing firm conclusions about production readiness.

11 Conclusions

11.1 Introduction

This chapter draws together the outcomes of the project. It reviews what was achieved against each of the four research objectives, outlines the most valuable directions for future work, and closes with a reflection on the broader significance of the findings.

11.2 Achievements

Objective 1: Design and develop a low-cost, offline voice assistant prototype deployable on commodity Android hardware.

A fully functional prototype was built across four iterative development cycles. The system runs without internet connectivity; speech recognition and intent classification happen on the guest's device. It handles the complete request lifecycle from voice input to staff notification, service completion, and guest rating, on a commodity Android tablet in the \$50-\$150 range.

Objective 2: Build a hospitality-domain dataset and train a noise-aware NLU model.

A custom dataset of 10,080 labelled phrases was created across 18 intent categories (560 per intent) mapped to 5 hotel departments. A paired Vosk-transcribed version was produced to enable noise-aware training. The vosk-model-small-en-in-0.4 model (~36MB, Indian English) and a fine-tuned MobileBERT (26MB TFLite, quantized from 94MB PyTorch 72% reduction) were deployed on-device via a hybrid NLU pipeline: keyword matching handles high-confidence requests, MobileBERT handles broader phrasing. To the best of the author's knowledge, this is the first publicly documented intent classification dataset for hospitality service operations.

Objective 3: Build a lightweight backend with real-time staff communication.

A FastAPI backend with SQLite persistence and WebSocket-based communication was implemented. Requests are routed to the correct department via the `intent_department_mapping` table. The staff dashboard supports bidirectional messaging and department filtering. The entire backend runs on any low-cost laptop. No cloud infrastructure required.

Objective 4: Evaluate system accuracy, latency, and cost-effectiveness, and demonstrate privacy-preserving offline operation.

Three MobileBERT variants were evaluated on a shared 2,016-sample held-out test set. Model A dropped from 98.07% on clean text to 89.34% on Vosk transcriptions an 8.73 percentage point gap from the STT step alone. Model C (noise-aware) recovered to 99.06% on Vosk output (111.3% gap recovery). WER was 11.43% overall, ranging from 6.78% (emergency) to 16.83% (temperature_control). P95 latency was 2,801ms, within the 5-second target. All voice processing is on-device, no audio leaves the hotel network, and the full stack is open-source with no recurring licensing costs.

Overall Assessment

The research addressed all four gaps identified in the literature: end-to-end offline voice processing with on-device NLU in a real STT pipeline; privacy-preserving deployment without

cloud dependency; cost-effective commodity hardware; and a solution designed for the operational constraints of hotels in developing economies.

11.3 Future Work

The prototype successfully demonstrates the core concept. The following items describe the most valuable next steps.

11.3.1 Field Deployment and User Study

The most important next step is deploying in a real hotel. A field study would validate speech recognition with real Sri Lankan English accents in room acoustic conditions, identify intent categories that real guests actually use, and surface usability issues that controlled testing cannot reveal. It would also allow the 99.06% accuracy figure currently measured on TTS-synthesized speech to be re-evaluated under genuine production conditions.

11.3.2 Real Speech Data Collection and Model Retraining

Training data and WER measurements were based on TTS audio, not real recordings. The tokenizer mismatch on Android (see Section 10.4.3) means actual on-device accuracy is lower than 99.06%. Collecting real voice recordings from Sri Lankan English speakers, using their Vosk transcriptions as training data, and re-evaluating all three models would give a more accurate picture of real-world performance.

11.3.3 Proper WordPiece Tokenizer on Android

The Android implementation uses a simplified word-level tokenizer rather than the HuggingFace WordPiece tokenizer used during training. Implementing a proper WordPiece tokenizer in Kotlin or switching to TFLite's built-in tokenization support would align on-device inference with the evaluation conditions and recover some of the accuracy gap.

11.3.4 Multilingual Support

The system currently supports English only. However, hotels in Sri Lanka serve guests from diverse international markets, including China, Germany, France, and Russia. Future work could extend the system to support additional languages by integrating multilingual speech recognition models and training language-specific intent classifiers. Automatic language detection could further allow the system to identify the guest's language and route accordingly. This would meaningfully broaden the system's applicability across international hotel properties.

11.3.5 Further NLU and Dialogue Improvements

Beyond the core limitations above, several improvements would strengthen the NLU layer: a custom Vosk language model trained on hospitality vocabulary (targeting high-WER intents such as `temperature_control` at 16.83% and `towel_request` at 16.33%), an explicit out-of-scope intent to prevent unnecessary staff notifications when guests ask general questions, and a basic conversation tracker to handle requests that span multiple steps, such as "I'd like two towels and a bottle of water."

11.4 Summary

The answer to the core research question is yes. The prototype achieves sufficient technical accuracy and performance across all measured dimensions: 99.06% NLU accuracy on real Vosk output (Model C), 11.43% WER acceptable for the hospitality domain, P95 latency of 2,801ms within the 5-second target, and hardware cost in the \$50-\$150 range. The three-model experiment provides the key supporting evidence Model A's 8.73 percentage point drop from clean to Vosk conditions, and Model C's 111.3% recovery, make a concrete case for noise-aware training as a requirement for reliable production performance.

The main limitation is the synthetic evaluation foundation: WER measurements and training noise profiles came from TTS audio, not real recordings. Confirming these results with actual hotel guest speech, and deploying in a real hotel, are the most critical next steps. Chapter 12 reflects on the personal learning experience behind this work.

12 Student Reflections

12.1 Introduction

This chapter is a personal reflection, what I learned, the problems I hit, and what I would do differently. Unlike the preceding chapters, this one is about how working on the project changed my understanding of software engineering, research, and the gap between the two.

12.2 Personal Growth and Skills Development

This was the most technically demanding project I have undertaken during my studies. Going in, I had no practical experience with on-device machine learning deployment and Android development with Jetpack Compose. By the end, I had built working implementations not from following tutorials, but from solving real problems under real constraints.

The biggest area of growth was learning to work across multiple technology stacks simultaneously; debugging TFLite conversions in Python, fixing Kotlin tokenizer issues, and investigating WebSocket disconnections all for the same system. By Iteration 4, tasks that had taken days in Iteration 1 were taking hours.

I also gained a more realistic understanding of the gap between papers and working implementations. Reading that MobileBERT achieves 62ms inference latency sounds straightforward. Getting it to load, convert correctly, tokenize inputs as Python does, and produce correct outputs on a budget tablet is a completely different challenge.

12.3 Problems Encountered and How They Were Resolved

12.3.1 The Vosk Model Memory Problem

The first significant problem appeared early in Iteration 1, before I had written much code at all. I loaded the large Vosk model onto the test Android tablet and watched the app crash with an out-of-memory error. The load time alone was over 15 seconds; completely unusable for a guest-facing system. I had assumed bigger meant better. What I had not accounted for was that a budget tablet cannot hold a 1.8GB model alongside the OS and running processes. Switching to `vosk-model-small-en-in-0.4` (~36MB) solved the problem and turned out to be a better choice than the Indian English acoustic model handles South Asian accents much better.

The lesson: always test on your actual target hardware as early as possible.

12.3.2 The TFLite Conversion and Label Ordering Bug

Converting the trained MobileBERT model from PyTorch to TFLite was supposed to take two or three days. It ended up taking nearly a week, mostly because of one bug that was very difficult to diagnose.

The initial conversion worked; the model loaded and produced output. But predictions were wrong. A request for towels came back as something unrelated. After a long time checking the model, tokenizer, and inference code, the problem was a mismatch in how PyTorch and TensorFlow ordered the classification labels.

The fix was simple once found; align label ordering. But finding it required verifying raw logit outputs against known test inputs in both frameworks.

The lesson: model conversion is not a black-box operation; every stage needs to be verified against the previous one.

12.3.3 The Tokeniser Compromise

This is the problem I never fully solved; I worked around it.

BERT uses a WordPiece tokenizer that splits words into sub-word units ("housekeeping" → ["house", "##keeping"]). The HuggingFace tokenizer is Python-only. I tried re-implementing it in Kotlin; matching sub-word tokens from a 30,522-entry vocabulary, managing special tokens, and padding to 32 tokens.

My Kotlin implementation produced wrong token sequences, which caused the model to output incorrect predictions. Debugging this required comparing token-by-token outputs between my Kotlin code and the Python tokenizer, which eventually revealed several edge cases I had missed in the subword matching logic.

In the end, I implemented a simplified word-level tokenizer that looks up whole words rather than subword pieces. This means on-device accuracy is lower than the 99.06% measured in Python. I documented this as a limitation (Section 10.4.3) and properly solving it would be a production priority.

12.3.4 The WebSocket Screen-Off Disconnection

In Iteration 3, I noticed the Android device was silently dropping its WebSocket connection when the screen turned off. The app reported no error, it simply stopped receiving notifications.

The fix was to implement an exponential backoff reconnection strategy in the WebSocket client: on disconnect, wait 2 seconds and retry, then 4 seconds, then 8 seconds, up to a maximum of 30 seconds. This resolved the problem, and the reconnection behavior was explicitly verified during integration testing.

The lesson: mobile power management can interfere with long-lived network connections. Any real-time mobile app needs explicit reconnection handling.

12.3.5 Keeping Scope Under Control

Midway through, I was tempted to keep adding features; wake word detection, Sinhala support, a more sophisticated dialogue system. Each seemed like just a few extra days. Recognizing this and stopping was one of the better decisions I made. A thoroughly evaluated core system is a stronger submission than a feature-rich one with gaps in its evaluation. Scope discipline is what makes the difference between finishing a project and not finishing it.

12.4 What I Would Do Differently

12.4.1 Start Writing Earlier

I genuinely experienced the consequences of leaving writing too late. I delayed until the implementation was largely complete, the WebSocket issues pushed it back further, and the final weeks were more compressed than they needed to be. Writing earlier would also have helped with implementation. When I wrote up design decisions, some justifications required more thought than I had given them during coding. Writing them earlier might have led to slightly better decisions.

12.4.2 Collect Real Hotel Data Earlier

The training dataset was synthetic, practical and effective, but I was always aware that real phrases would have been more genuine. I had hotel management contacts from the requirements interviews. Asking for even a few hundred real phrases as supplementary data or an independent

test set would have made the evaluation more convincing and would have been much easier to organize before implementation was underway.

12.4.3 Build in Automated Tests from the Start

I tested manually throughout, which worked but became time-consuming as the system grew. A basic automated test suite for the tokenizer, NLU pipeline, and core API endpoints would have caught several bugs faster; particularly the TFLite label ordering issue and tokenizer edge cases. The test_api.py I eventually wrote was straightforward. I should have built equivalent scripts for the NLU pipeline much earlier.

12.4.4 Use Structured Logging from the Beginning

I used print statements throughout. For simple testing this worked, but tracking timing-related issues; like what the Android client was doing when WebSocket dropped, was difficult. Structured logging with timestamps from the start would have made debugging much faster. It was much harder to add later, than it would have been to include from the start.

12.5 Reflections on the Research Process

12.5.1 Iterative Development Was the Right Approach

The iterative structure was what made the project manageable. The most important decisions came from having a working prototype to test against, switching to the smaller Vosk model, adding the hybrid NLU pipeline, and handling WebSocket reconnection. None of these were planned upfront; all emerged from building something real and observing what broke. This project confirmed iterative development is the only practical approach when the technical outcome is uncertain.

12.5.2 The Gap Between Papers and Working Systems

The research literature created an impression that implementation would be relatively straightforward; the models exist, the frameworks exist. The reality was that getting from "these models exist" to "this model runs correctly on a budget Android device" involved many intermediate steps, each of which could fail. I now read ML papers with a more critical eye on deployment perspective. Benchmark accuracy and on-device accuracy with your specific hardware and pipeline can be meaningfully different; that gap is where most of the engineering work lives.

12.5.3 Working Across Disciplines

This project sat at the intersection of machine learning, mobile development, backend engineering, real-time systems, and hospitality operations. Making sensible decisions required understanding enough about all of them to see how they connected; for example, understanding why WordPiece tokenization matters for MobileBERT accuracy and why reimplementing it on Android is difficult.

The ability to work across technical domains, not as a deep expert in each, but with enough understanding to make them work together is something I will carry into my career more than any specific skill.

12.6 Summary

The main personal takeaways: test on your actual target hardware early; verify every stage of a model conversion pipeline; start writing before implementation is finished; maintain scope discipline. More than any specific skill, the lesson I will carry forward is that the most useful decisions in a project like this come from building something real and learning from what breaks; not from getting the design perfect on paper first.

References

- Alpha Cephei (2023) *Vosk offline speech recognition* [Software]. Available at: <https://alphacephei.com/vosk/> (Accessed: 8 March 2026).
- Amazon Web Services (n.d.) *Alexa Smart Properties for Hospitality*. Available at: <https://developer.amazon.com/en-US/alexa/alexa-smart-properties/hospitality> (Accessed: 24 March 2026).
- Amazon Web Services (2024) *Amazon Polly pricing*. Available at: <https://aws.amazon.com/polly/pricing/> (Accessed: 8 March 2026).
- Buhalis, D., & Moldavska, I. (2021) 'In-room Voice-Based AI Digital Assistants Transforming On-Site Hotel Services and Guests' Experiences', in Wörndl, W., Koo, C. and Stienmetz, J.L. (eds.) *Information and Communication Technologies in Tourism 2021*. Cham: Springer, pp. 30-44. Available at: https://doi.org/10.1007/978-3-030-65785-7_3
- Buhalis, D., & Moldavska, I. (2022) 'Voice assistants in hospitality: using artificial intelligence for customer service', *Journal of Hospitality and Tourism Technology*, 13(3), pp. 386-403. Available at: <https://doi.org/10.1108/JHTT-03-2021-0104>
- Bujel, K., Slimi, A. and Kureychik, V. (2021) 'Effectiveness of Pre-training for Few-shot Intent Classification', *arXiv preprint arXiv:2109.05782*. Available at: <https://arxiv.org/abs/2109.05782> (Accessed: 3 March 2026).
- Bunk, T. et al. (2020) *DIET: Dual Intent and Entity Transformer* [Blog post]. Rasa Technologies. Available at: <https://rasa.com/blog/introducing-dual-intent-and-entity-transformer-diet-state-of-the-art-performance-on-a-lightweight-architecture> (Accessed: 3 March 2026).
- Chantrapornchai, C., & Suchato, A. (2022) 'IoT Device Control with Offline Automatic Speech Recognition on Edge Device', *IEEE Conference Publication*. Available at: <https://ieeexplore.ieee.org/document/10010962> (Accessed: 3 March 2026).
- CMU Sphinx (2023) *PocketSphinx* [Software repository]. Available at: <https://github.com/cmusphinx/pocketsphinx> (Accessed: 8 March 2026).
- Devlin, J. et al. (2019) 'BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding', in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*. Minneapolis: ACL, pp. 4171-4186.
- Fette, I., & Melnikov, A. (2011) *The WebSocket Protocol*, RFC 6455. Internet Engineering Task Force (IETF). Available at: <https://tools.ietf.org/html/rfc6455> (Accessed: 8 March 2026).
- Google (2024a) *TensorFlow Lite: Machine learning for mobile and edge devices*. Available at: <https://www.tensorflow.org/lite> (Accessed: 8 March 2026).
- Google (2024b) *Jetpack Compose*. Available at: <https://developer.android.com/jetpack/compose> (Accessed: 8 March 2026).
- Google Cloud (2024) *Speech-to-Text pricing*. Available at: <https://cloud.google.com/speech-to-text/pricing> (Accessed: 24 March 2026).

Hansen, M. (2023) *Piper: A fast, local neural text to speech system* [Software]. Available at: <https://github.com/rhasspy/piper> (Accessed: 25 March 2026).

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004) 'Design Science in Information Systems Research', *MIS Quarterly*, 28(1), pp. 75-105. Available at: <https://doi.org/10.2307/25148625>

Hipp, D.R. (2024) *SQLite* [Software]. Available at: <https://www.sqlite.org> (Accessed: 8 March 2026).

Huggins-Daines, D. et al. (2006) 'Pocketsphinx: A free, real-time continuous speech recognition system for hand-held devices', in *Proceedings of ICASSP 2006*, Vol. 1, pp. 1-185–1-188.

Hwang, J., & Erdem, M. (2025) "'U" in User Experience (UX) Stands for the Frontline Employee: A Case Study of Voice Assistant Technology Use in Hotels', *Journal of Hospitality and Tourism Cases*. Available at: <https://doi.org/10.1177/21649987251361910>

Korkmaz, A. et al. (2025) 'Real-Time Speech-to-Text on Edge: A Prototype System for Ultra-Low Latency Communication with AI-Powered NLP', *Information*, 16(8), p. 685. Available at: <https://doi.org/10.3390/info16080685>

Kreps, J., Narkhede, N., & Rao, J. (2011) 'Kafka: A distributed messaging system for log processing', in *Proceedings of the NetDB Workshop at VLDB 2011*. Seattle: ACM.

Larson, S. et al. (2022) 'A Survey of Intent Classification and Slot-Filling Datasets for Task-Oriented Dialog', *arXiv preprint arXiv:2207.13211*. Available at: <https://arxiv.org/abs/2207.13211> (Accessed: 3 March 2026).

Marriott International (2018) *Marriott International Rolls Out Alexa for Hospitality to Hotels Across the United States*. Press Release, 19 June. Available at: <https://news.marriott.com/news/2018/06/19/marriott-international-rolls-out-alexa-for-hospitality-to-hotels-across-the-united-states> (Accessed: 24 March 2026).

OASIS (2019) *MQTT version 5.0*. OASIS Standard. Available at: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html> (Accessed: 8 March 2026).

Radford, A. et al. (2023) 'Robust Speech Recognition via Large-Scale Weak Supervision', in *Proceedings of the 40th International Conference on Machine Learning (ICML 2023)*. Honolulu: PMLR. Available at: <https://arxiv.org/abs/2212.04356>

Ramírez, S. (2024) *FastAPI* [Software]. Available at: <https://fastapi.tiangolo.com> (Accessed: 8 March 2026).

Raspberry Pi Foundation (2025) *Raspberry Pi 4 Model B*. Available at: <https://www.raspberrypi.com/products/raspberry-pi-4-model-b/> (Accessed: 24 March 2026).

Sanh, V. et al. (2019) 'DistilBERT, a Distilled Version of BERT: Smaller, Faster, Cheaper and Lighter', *NeurIPS 2019 Workshop on Energy Efficient Machine Learning and Cognitive Computing*. Available at: <https://arxiv.org/abs/1910.01108>

Shi, W. et al. (2016) 'Edge Computing: Vision and Challenges', *IEEE Internet of Things Journal*, 3(5), pp. 637-646. Available at: <https://doi.org/10.1109/JIOT.2016.2579198>

Sri Lanka Tourism Development Authority (SLTDA) (2024) *Year in Review 2024*. Colombo: Sri Lanka Tourism Development Authority. Available at: https://www.slttda.gov.lk/storage/common_media/Year_In_Review_2024_Final_2024_Jan-Dec1.pdf (Accessed: 21 March 2026).

Sun, Z. et al. (2020) 'MobileBERT: a Compact Task-Agnostic BERT for Resource-Limited Devices', in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL 2020)*. Online: ACL, pp. 2158-2170. Available at: <https://doi.org/10.18653/v1/2020.acl-main.195>

Wang, X. et al. (2025) 'Empowering Edge Intelligence: A Comprehensive Survey on On-Device AI Models', *ACM Computing Surveys*. Available at: <https://doi.org/10.1145/3724420>

Wickramasinghe, K., & Ratnayake, R. (2022) 'Connecting the Connected: How Is Sri Lanka Prepared to Respond to Digital Tourists?', in *Information and Communication Technology for Competitive Strategies (ICTCS 2021)*. Singapore: Springer, pp. 247-258. Available at: https://doi.org/10.1007/978-981-16-5461-9_22

World Travel and Tourism Council (WTTC) (2023) *Economic Impact Research 2023*. London: WTTC. Available at: <https://wttc.org/research/economic-impact> (Accessed: 3 March 2026).

Yilmaz, E. et al. (2025) 'Echoes of Innovation: Exploring the Use of Voice Assistants to Boost Hotel Reputation', *Journal of Theoretical and Applied Electronic Commerce Research*, 20(1), p. 46. Available at: <https://doi.org/10.3390/jtaer20010046>

Zhang, L. et al. (2022) 'Improved Spoken Language Representation for Intent Understanding in a Task-Oriented Dialogue System', *Sensors*, 22(4), p. 1509. Available at: <https://doi.org/10.3390/s22041509>

Zhang, Y. et al. (2022) 'Fine-Tuning BERT Models for Intent Recognition Using a Frequency Cut-Off Strategy for Domain-Specific Vocabulary Extension', *Applied Sciences*, 12(3), p. 1610. Available at: <https://doi.org/10.3390/app12031610>

Appendix A – Project Specification

[Research Proposal.pdf](#)

Appendix B – Interim Progress Report and Meeting Records

Following is the log of supervisor meetings held throughout the research period.

#	Date	Meeting Type	Key Discussion Points
1	01 Nov 2025	Initial Meeting	Introduction of proposed research project, research question, and project scope
2	25 Jan 2026	Progress Review	Prototype demo presented; discussed challenges faced during model training and Android development
3	11 Mar 2026	Group Session	Participated in session on research report writing and viva presentation preparation
4	29 Mar 2026	Progress Review	Discussed ongoing challenges with model and Android development, research report preparation and submission planning.

Table B.1: Supervisor Meeting Log

Appendix C – Requirements Specification Document

Guest Survey: Instrument and Results Summary

C.1 Survey Questionnaire

Title: Guest Experience with Hotel Room Service Requests

Purpose: This survey was conducted to understand how hotel guests currently make room service requests, their attitudes towards voice-assisted alternatives, and their privacy expectations. Responses were used to inform the functional and non-functional requirements of the system described in this report.

Eligibility criterion: Respondents must have stayed in at least one hotel in the past 12 months.

Distribution method: Convenience sampling; administered in person and via messaging to family members, relatives, and colleagues known to the researcher.

Questionnaire: [Guest Survey.pdf](#)

C.2 Respondent Demographics

A total of **20 individuals** completed the survey. All 20 met the eligibility criterion of at least one hotel stay in the past 12 months.

Age Group	Count	Percentage
Under 20 (teenagers)	4	20%
20-35 (young adults)	7	35%
36-59 (middle-aged)	5	25%
60-70 (older adults)	4	20%
Total	20	100%

Table C.1: Respondent Age Distribution

Frequency	Count	Percentage
Once	6	30%
2-3 times	10	50%
4 or more times	4	20%
Total	20	100%

Table C.2: Hotel Stay Frequency (Past 12 Months)

Method	Count	Percentage
Call the front desk by phone	14	70%
Go to the front desk in person	4	20%
Use a hotel app	1	5%
Ask staff in the corridor	1	5%
Total	20	100%

Table C.3: Usual Method for Making Room Service Requests

C.3 Key Findings Summary

Finding	Count	Percentage
Experienced delays or miscommunication making requests by phone (Q3: Yes)	15	75%
Would use a voice assistant if audio stays on-device only (Q4+Q5 combined: comfortable/very comfortable or "would change")	13	65%
Prefer voice + on-screen confirmation before request submitted (Q7: both or screen)	16	80%
Uncomfortable with cloud-connected voice device in hotel room (Q8: uncomfortable/very uncomfortable)	5	25%

Table C.4: Survey Findings Referenced in Section 4.2.2

Service	Responses	% of Respondents
Requesting towels / bedding	18	90%
Housekeeping / room cleaning	17	85%
Food and beverage orders	16	80%
Requesting toiletries	13	65%
Reporting a maintenance issue	10	50%
General hotel information	9	45%
Noise complaints	6	30%
Checkout / billing questions	5	25%

Table C.5: Most Requested Services via Voice (Q6 - multiple selection)

Note: Percentages exceed 100% because respondents could select multiple options.

This survey was conducted in November 2025 as part of an MSc in Data Science research project at Coventry University, UK. All responses were anonymous and participation was voluntary.

Appendix D – Code Base

<https://github.com/mdpw/msc-research>

Appendix E – User Manual

[User Manual.pdf](#)

Appendix F – Project Presentation

[Presentation.pptx](#)

Appendix G – List of Figures

<i>Figure 3-1: Three-tier Architecture</i>	23
<i>Figure 5-1: High-Level System Conceptual View</i>	32
<i>Figure 5-2: Unified Modelling Language (UML) Use Case Diagram</i>	33
<i>Figure 5-3: Audio Capture and Speech Recognition</i>	35
<i>Figure 5-4: Intent Classification and Request Submission</i>	36
<i>Figure 5-5: Staff Request Handling Flow</i>	37
<i>Figure 5-6: Entity-Relationship Diagram</i>	38
<i>Figure 6-1: System Architecture Diagram</i>	46
<i>Figure 6-2: Audio Capture and Transcription</i>	48
<i>Figure 6-3: Intent Classification</i>	48
<i>Figure 6-4: Confirmation and Submission</i>	49
<i>Figure 6-5: Hybrid NLU Pipeline Flow</i>	50
<i>Figure 6-6: Entity-Relationship Diagram</i>	51
<i>Figure 6-7: WebSocket Communication Topology</i>	54
<i>Figure 6-8: Request State Diagram</i>	54
<i>Figure 6-9: Guest Application Interface</i>	56
<i>Figure 6-10: Login Screen</i>	57
<i>Figure 6-11: Dashboard Layout</i>	57
<i>Figure 8-1: NLU Accuracy - Gap Introduced by STT Noise and Recovery via Mixed Training</i> ..	69
<i>Figure 8-2: Model A Confusion Matrix (Vosk input)</i>	70
<i>Figure 8-3: Model C Confusion Matrix (Mixed training, Vosk input)</i>	71

Appendix H – List of Tables

Table 2-1: Comparison Matrix of Existing Solutions.....	18
Table 2-2: Challenge-to-Tool Mapping	20
Table 3-1: Development Iterations.....	22
Table 3-2: Training Datasets	24
Table 3-3: Training Configuration	25
Table 4-1: Functional Requirements.....	29
Table 4-2: Predefined Intent Categories (FR-03)	30
Table 4-3: Non-Functional Requirements	31
Table 4-4: MoSCoW Prioritisation	31
Table 5-1: Use Case Descriptions	34
Table 5-2: Technology Stack Summary.....	39
Table 5-3: Speech Recognition Technology Comparison	40
Table 5-4: Intent Classification Technology Comparison	41
Table 5-5: Backend Framework Comparison.....	42
Table 5-6: Database Technology Comparison.....	43
Table 5-7: Real-Time Communication Technology Comparison	43
Table 5-8: Mobile Platform Comparison	44
Table 5-9: Text-to-Speech Technology Comparison.....	45
Table 6-1: List of REST API Endpoints.....	52
Table 6-2: WebSocket Communication - Guest Channel.....	53
Table 6-3: WebSocket Communication - Dashboard Channel	53
Table 6-4: Department Routing Design	55
Table 6-5: Guest UI Design Decisions.....	56
Table 6-6: Staff Dashboard Design Decisions	58
Table 7-1: Summary of the complete development environment	60
Table 7-2: Android Application Components.....	62
Table 7-3: On-device ML assets.....	63
Table 7-4: NLU Pipeline Scripts.....	63
Table 7-5: Training Dataset Summary.....	64
Table 7-6: Backend File Structure	65
Table 7-7: Department Colour Coding	65
Table 7-8: Deployment Components	66
Table 7-9: Requirements Implementation Status	67
Table 8-1: Four Evaluation Runs	69
Table 8-2: NLU Model Evaluation Results (2,016-sample test set)	69
Table 8-3: Model C Per-Intent Performance (all 18 intents, Vosk input, 112 samples each)	72
Table 8-4: Overall WER Statistics (10,080 phrases, 72,133 reference words)	72
Table 8-5: Per-Intent WER (highest to lowest).....	73
Table 8-6: Backend API Tests.....	73
Table 8-7: Scenarios tested.....	74
Table 8-8: Measurement Approach on each stage	74
Table 8-9: Pipeline Latency Results (n = 20 test requests)	75
Table 8-10: NFR Compliance Summary	76
Table 8-11: Testing Coverage Summary	76
Table 9-1: Work Breakdown Structure	77
Table 9-2: Project Gantt Chart.....	78
Table 9-3: Risk Register	80
Table 9-4: Open-Source Licences	82
Table 10-1: Achievement of Research Objectives	84
Table 10-2: Production Deployment Cost for a 50-Room Hotel	86

<i>Table 10-3: Estimated total hardware investment (50-room hotel)</i>	<i>87</i>
<i>Table 10-4: Three-Year Total Cost of Ownership (TCO) Comparison for a 50-Room Hotel</i>	<i>87</i>
<i>Table 10-5: Summary of Limitations</i>	<i>90</i>

Appendix I – Glossary of Abbreviations and Acronyms

This glossary lists all abbreviations and acronyms used across the chapters of this report. Terms are listed alphabetically.

Abbreviation	Full Form	Context
AAC	Advanced Audio Coding	Audio codec (referenced as alternative audio format)
AI	Artificial Intelligence	General field encompassing ML, NLU, and STT technologies used in this project
API	Application Programming Interface	Communication interface between the Android app, backend server, and dashboard
APK	Android Package Kit	Distributable file format for installing the guest app on Android tablets
ASGI	Asynchronous Server Gateway Interface	Server specification that allows FastAPI and Uvicorn to handle async WebSocket connections
AWS	Amazon Web Services	Amazon's cloud infrastructure platform; cloud-based alternatives such as Alexa for Hospitality run on AWS
BERT	Bidirectional Encoder Representations from Transformers	Pre-trained transformer model used as the base for intent classification fine-tuning
BCS	British Computer Society	Professional body whose code of conduct informs the ethical standards of this project
BOM	Bill of Materials	Version specification file used by Jetpack Compose to manage compatible library versions
CER	Character Error Rate	Metric measuring the percentage of incorrectly transcribed characters in Vosk output
CLI	Command-Line Interface	Text-based interface used to start the backend server and run training scripts
CPU	Central Processing Unit	Processing hardware on which the on-device models run (no GPU used)
CRUD	Create, Read, Update, Delete	Standard database operations; the FastAPI backend CRUD layer uses raw SQL without an ORM
CSS	Cascading Style Sheets	Styling language used for the staff dashboard UI
CSV	Comma-Separated Values	File format for storing training datasets (e.g., new_hotel_dataset.csv)
DevOps	Development and Operations	Integrated approach combining development and deployment practices
DIET	Dual Intent and Entity Transformer	Intent classification architecture used in the Rasa framework (reviewed in literature)

DistilBERT	Distilled BERT	Lightweight BERT variant created via knowledge distillation (reviewed in literature)
DSR	Design Science Research	Research methodology framework adopted for this project
ERM	Entity-Relationship Model	Diagram showing the database schema and table relationships
FR	Functional Requirement	Prefix used to label system functional requirements (FR-01 through FR-10)
GDPR	General Data Protection Regulation	EU data protection regulation relevant to processing guest voice data
GLUE	General Language Understanding Evaluation	Benchmark suite used to evaluate transformer model performance (referenced in literature)
GPIO	General Purpose Input/Output	Hardware interface on single-board computers (referenced in IoT deployment context)
GPU	Graphics Processing Unit	Hardware accelerator used in training but not in on-device inference
gTTS	Google Text-to-Speech	Python library used in step2_generate_vosk_noise.py to synthesise audio from text phrases for Vosk pairing
HCI	Human-Computer Interaction	Design discipline informing the voice-first UI/UX decisions
HTML	HyperText Markup Language	Markup language used for the staff dashboard (single self-contained HTML file)
HTTP	Hypertext Transfer Protocol	Protocol used for REST API communication between the Android app and server
HTTPS	Hypertext Transfer Protocol Secure	Encrypted version of HTTP (recommended for production deployment)
IDE	Integrated Development Environment	Development tool; Android Studio used for Android development
INT8	8-bit Integer	Numeric format used in model quantisation; dynamic range quantisation converts model weights to INT8
IoT	Internet of Things	Domain of edge-deployed smart devices relevant to on-device AI deployment
IP	Internet Protocol	Network-layer protocol; the server is accessed by its local IP address
JSON	JavaScript Object Notation	Data format used for API responses, label maps, and vocabulary files
JWT	JSON Web Token	Token-based authentication format (noted as future enhancement)

KPI	Key Performance Indicator	Measurable metric used to evaluate system performance
LAN	Local Area Network	Hotel's internal Wi-Fi network on which the entire system runs
MB	Megabyte	Unit of data storage (Vosk model: ~36 MB; NLU model: 26 MB)
MDM	Mobile Device Management	Platform for centralised tablet provisioning (noted as future enhancement)
MFCC	Mel-Frequency Cepstral Coefficients	Acoustic features used by the Vosk speech recognition model
ML	Machine Learning	Subfield of AI; used for STT and intent classification models
MobileBERT	Mobile BERT	Compact BERT variant optimised for mobile deployment; used as the NLU backbone
MoSCoW	Must have, Should have, Could have, Won't have	Prioritisation framework used to classify requirements
MQTT	Message Queuing Telemetry Transport	Lightweight IoT messaging protocol (reviewed as alternative to WebSocket)
MVP	Minimum Viable Product	Scope descriptor for the research prototype
NFC	Near Field Communication	Short-range wireless technology (noted as future option for room pairing)
NFR	Non-Functional Requirement	Prefix used to label performance and quality requirements (NFR-01 through NFR-07)
NLU	Natural Language Understanding	Component responsible for classifying guest intent from transcribed speech
ONNX	Open Neural Network Exchange	Cross-framework model format; considered for Whisper Android deployment but not adopted
ORM	Object-Relational Mapping	Database abstraction layer deliberately excluded from the backend (raw SQL used instead)
P95	95th Percentile	Latency metric indicating the response time below which 95% of requests fall (P95 = 2,801 ms)
PCM	Pulse Code Modulation	Audio encoding format; the app captures audio at 16 kHz, 16-bit PCM mono
PDPA	Personal Data Protection Act	Sri Lanka's data protection legislation (Personal Data Protection Act No. 9 of 2022)
PMS	Property Management System	Hotel operations software (e.g., Opera, Mews); noted as a future integration target

PNG	Portable Network Graphics	Image format used for evaluation output (confusion matrix plots)
POS	Point of Sale	Hotel retail system noted as a future integration point
QA	Quality Assurance	Process of verifying that the system meets defined requirements
QR	Quick Response	QR code format noted as a future option for automated device provisioning
RAG	Retrieval-Augmented Generation	AI technique for grounding LLM responses in a knowledge base (future enhancement)
RAM	Random Access Memory	Device memory; budget tablet constraints influenced the choice of the 36 MB Vosk model
REST	Representational State Transfer	Architectural style for the HTTP API between the Android app and the backend
RMS	Root Mean Square	Energy measurement used in the VAD algorithm to detect speech (threshold: 0.02)
SDK	Software Development Kit	Native libraries included in the Android app (Vosk SDK, TFLite SDK)
SLTDA	Sri Lanka Tourism Development Authority	Government body that regulates and promotes Sri Lankan tourism
SQL	Structured Query Language	Language used directly for all database queries (no ORM layer)
SSL	Secure Sockets Layer	Predecessor to TLS; used informally to refer to encrypted connections
STT	Speech-to-Text	The process of converting recorded audio to a text transcript using the Vosk model
TCO	Total Cost of Ownership	Three-year cost projection used in the cost–benefit analysis
TFLite	TensorFlow Lite	Lightweight ML runtime used to run the MobileBERT model on Android
TLS	Transport Layer Security	Encryption protocol used by HTTPS (recommended for production)
TTS	Text-to-Speech	Converts text responses to spoken audio on the guest device (Android native engine)
UAT	User Acceptance Testing	User-facing evaluation confirming the system meets guest-side requirements
UI	User Interface	Visual and interactive elements of both the guest app and the staff dashboard

UML	Unified Modeling Language	Diagramming standard used for use case and sequence diagrams in Chapters 5 and 6
USD	United States Dollar	Currency used in cost estimates and market data cited in the literature review
UX	User Experience	Quality of the overall guest interaction with the voice assistant
VAD	Voice Activity Detection	Energy-based algorithm that determines when the guest has started and stopped speaking
VLAN	Virtual Local Area Network	Network segmentation technique recommended for production security hardening
WER	Word Error Rate	Metric measuring the percentage of incorrectly transcribed words in Vosk output
WTTC	World Travel and Tourism Council	International tourism body; cited for global hospitality industry statistics

Total entries: 70

— End of Report —